



UNIVERSIDAD DE MÁLAGA



MÁSTER UNIVERSITARIO EN INGENIERÍA DEL SOFTWARE E INTELIGENCIA ARTIFICIAL

ANÁLISIS EXPERIMENTAL DE ORQUESTACIÓN DE AGENTES BASADOS EN LLMS PARA LA REPARACIÓN AUTOMÁTICA DE SOFTWARE

EXPERIMENTAL ANALYSIS OF LLM-BASED AGENT ORCHESTRATION FOR AUTOMATED SOFTWARE REPAIR

Realizado por
Adrián Torremocha Doblas

Tutorizado por
Gabriel Jesús Luque Polo
Francisco Javier Servant Cortes

Departamento
Lenguajes y Ciencias de la Computación

MÁLAGA, JUNIO DE 2026



UNIVERSIDAD
DE MÁLAGA



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA
INFORMÁTICA
MÁSTER UNIVERSITARIO EN INGENIERÍA DEL SOFTWARE
E INTELIGENCIA ARTIFICIAL

**ANÁLISIS EXPERIMENTAL DE ORQUESTACIÓN
DE AGENTES BASADOS EN LLMS PARA LA
REPARACION DE SOFTWARE**

**EXPERIMENTAL ANALYSIS OF LLM-BASED AGENT
ORCHESTRATION FOR AUTOMATED SOFTWARE
REPAIR**

Realizado por
Adrián Torremocha Doblas

Tutorizado por
Gabriel Jesús Luque Polo
Francisco Javier Servant Cortes

Departamento
Lenguajes y Ciencias de la Computación

UNIVERSIDAD DE MÁLAGA
MÁLAGA, JUNIO DE 2026

Fecha defensa: junio de 2025

Resumen. La reparación automática de software basada en modelos de lenguaje de gran tamaño, o LLMs (del inglés, *Large Language Models*), ha pasado de la generación directa de parches a sistemas agénticos capaces de leer código, ejecutar tests y corregir sus propios intentos. Este TFM estudia cómo influye la orquestación de esos agentes en un sistema de reparación automática de software, o APR (del inglés, *Automated Program Repair*), guiado por tests para proyectos Python. Para ello se ha desarrollado una plataforma experimental que ejecuta cada intento de reparación en un contenedor aislado, registra las acciones del agente y permite comparar arquitecturas bajo un entorno de ejecución controlado y reproducible. Se evalúan tres arquitecturas (mono-agente, Planner-Executor y orquestador con sub-agentes) combinadas con modelos locales y un modelo propietario, sobre los 40 bugs de QuixBugs y una muestra estratificada de BugsInPy. Más allá de la tasa de tests pasados, se incorpora una validación semántica que separa reparaciones correctas, plausibles y con sobreajuste al test. Los resultados muestran que QuixBugs ofrece poco margen de discriminación para los modelos más capaces, mientras que BugsInPy pone de manifiesto que pasar los tests no equivale necesariamente a corregir el fallo real. En ese escenario, el orquestador obtiene los mejores resultados, aunque el análisis también muestra que el contexto proporcionado al agente y la infraestructura de ejecución condicionan el rendimiento tanto como la arquitectura elegida.

Palabras clave: reparación automática de software, modelos de lenguaje de gran tamaño, sistemas multi-agente, pruebas de software, benchmarks.

Abstract. Automatic software repair based on large language models, or LLMs, has evolved from direct patch generation to agentic systems able to read code, run tests and correct their own attempts. This master's thesis studies how the orchestration of those agents affects a test-guided automated software repair system for Python projects. To do so, an experimental platform has been developed that executes each repair attempt inside an isolated container, records the agent's actions and enables architecture comparison under a controlled and reproducible runtime environment. Three architectures are evaluated (mono-agent, Planner-Executor and an orchestrator with sub-agents), combined with local models and a proprietary model, on the 40 bugs of QuixBugs and on a stratified sample of BugsInPy. Beyond the rate of passing tests, the evaluation incorporates a semantic validation stage that separates correct, plausible and test-overfitted repairs. The results show that QuixBugs leaves little room to differentiate the most capable models, whereas BugsInPy makes clear that passing the tests does not necessarily mean fixing the real fault. In that setting, the orchestrator obtains the best results, although the analysis also shows that the context provided to the agent and the execution infrastructure shape performance as much as the chosen architecture.

Keywords: Automated Program Repair, Large Language Models, Multi-Agent Systems, Software Testing, Benchmarks.

Table of Contents

1	Introducción	7
2	Antecedentes y trabajos relacionados	8
2.1	Reparación automática de software (APR)	8
	Evolución y enfoques clásicos.	8
	APR basado en tests: ventajas y limitaciones.	9
	El problema de los parches plausibles.	9
	Benchmarks de evaluación en APR.	10
2.2	De los modelos de lenguaje a los agentes autónomos	10
	Fundamentos de los modelos de lenguaje.	10
	La API de chat y el modelo de turnos.	11
	Modelos de lenguaje y generación de código.	12
	El salto al paradigma agéntico.	12
	Agent harness: infraestructura para la autonomía.	13
	Context engineering.	13
	Arquitecturas multi-agente.	14
2.3	LLMs aplicados al APR	15
	Enfoques basados en generación directa.	15
	Sistemas agénticos de reparación iterativa.	15
2.4	Trabajos relacionados	16
3	Objetivos y preguntas de investigación	18
3.1	Preguntas de investigación	18
3.2	Objetivos específicos	18
3.3	Alcance del sistema	19
4	Planteamiento del problema	20
4.1	El ciclo de reparación automática basado en tests	20
4.2	Retos de diseño	21
	Localización y exploración del código.	21
	Sobreajuste a los tests.	21
	Control de iteraciones, herramientas y restricciones.	22
4.3	Requisitos del sistema	22
5	Diseño e implementación	24
5.1	Visión arquitectónica general	24
5.2	Entorno de ejecución aislado	25
	Ejecución con Docker y contenedores temporales.	25
	Integración con datasets.	26
5.3	Herramientas del agente	26
	Herramientas de exploración.	26
	Herramientas de edición.	27
	Herramientas de ejecución y validación.	27
	Herramientas de diagnóstico de cambios.	28
	Herramientas delegadas por sub-agentes.	28
5.4	Bucle iterativo de reparación	28
	Fases de una iteración.	28
	Propagación de contexto entre iteraciones.	29
	Validación, reintentos y criterios de parada.	30
5.5	Arquitecturas de agentes implementadas	31
	Mono-agente.	31

	Multi-agente con handoff.	32
	Planner-Executor.	33
	Orchestrator con sub-agentes.	34
5.6	Observabilidad y trazabilidad de resultados	34
6	Refinamiento del harness iterativo.	36
6.1	Sobreajuste a la batería de tests	36
6.2	Continuidad de contexto entre iteraciones	37
6.3	Efecto acumulado de los refinamientos	38
6.4	Validación semántica asistida de parches	39
7	Evaluación experimental	41
7.1	Diseño experimental	41
	Variables del estudio.	41
	Benchmarks.	42
	Procedimiento de ejecución y análisis.	42
7.2	Métricas de evaluación	43
7.3	Validación semántica frente a métricas léxicas	44
7.4	Resultados globales	44
7.5	Resultados en QuixBugs	46
7.6	Resultados en BugsInPy.	47
7.7	Uso de herramientas, tokens y coste del procedimiento	48
7.8	Iteraciones, tiempos y causas de fallo	51
	Reparaciones según la iteración de terminación.	52
	Coste temporal de los intentos sin éxito.	52
	Causas de terminación de los intentos fallidos.	53
7.9	Análisis comparativo de arquitecturas y modelos	53
8	Conclusiones y trabajo futuro	56
8.1	Conclusiones	56
8.2	Trabajos futuros	57
A	Material auxiliar de validación semántica	58
A.1	Prompt de validación	58
A.2	Protocolo de la <i>skill</i> apr-validator	58
A.3	Caso ilustrativo de clasificación semántica	59
B	Prompts e instrucciones de los agentes	62
B.1	Mono-agente	62
B.2	Orquestador	63
	Agente principal.	63
	Sub-agente explorador.	64
	Sub-agente de tests.	64
B.3	Planner-Executor.	65
	Planner.	65
	Executor.	66

1 Introducción

La reparación de errores de software es una de las tareas más habituales en el desarrollo del software y uno de los problemas clásicos de APR [12,11,22]. En muchos casos, el punto de partida no es una descripción clara del problema, sino un test que falla, una traza de error y un fragmento de código que no se comporta como debería.

Durante años, los enfoques clásicos de APR abordaron este problema con estrategias de *generate-and-validate*: generar muchas variantes y quedarse con las que superan la batería de pruebas. Ese enfoque demostró que la reparación automática era viable, pero también dejó claras dos limitaciones. La primera es combinatoria, porque el espacio de cambios posibles crece muy rápido. La segunda es de validación, porque pasar tests no siempre equivale a corregir el bug real, ya que puede aparecer sobreajuste a los casos concretos probados [22].

En los últimos años, los modelos de lenguaje de gran tamaño han cambiado de forma notable este escenario. Estos modelos no solo pueden generar código, sino que también pueden leer ficheros, interpretar mensajes de error y proponer cambios a partir del contexto de un repositorio. Cuando se combinan con herramientas de ejecución, edición y validación, dejan de ser un generador de texto aislado y pasan a comportarse como agentes, es decir, sistemas que combinan un modelo, un historial de mensajes y un conjunto de herramientas para actuar sobre el repositorio [32,31]. En ese formato pueden seguir un ciclo de trabajo parecido al de un desarrollador. Observan el fallo, buscan el código relevante, editan, ejecutan tests y vuelven a intentarlo si la primera solución no funciona [30].

Este TFM parte de esa idea y estudia hasta qué punto la forma de organizar esos agentes afecta a la reparación automática de software. Para ello se ha desarrollado una plataforma experimental de APR orientada a proyectos Python. La plataforma ejecuta cada intento de reparación en un entorno aislado, registra las acciones del agente y permite comparar varias arquitecturas bajo las mismas condiciones. En concreto, la evaluación principal compara una arquitectura mono-agente, una arquitectura Planner-Executor y un orquestador con sub-agentes. Además, se prueban modelos locales y un modelo propietario, ya que el comportamiento del sistema no depende únicamente de la arquitectura, sino también de la capacidad del modelo que la ejecuta.

Las aportaciones principales del trabajo son las siguientes:

- una plataforma reproducible para ejecutar reparaciones sobre benchmarks Python;
- tres arquitecturas de agentes evaluadas sobre el mismo bucle de ejecución;
- una evaluación con modelos locales y un modelo propietario sobre QuixBugs y BugsInPy;
- una validación que separa tests pasados, parches plausibles y reparaciones correctas.

La última aportación es especialmente importante. En APR es habitual medir si el parche hace pasar los tests, pero ese indicador puede ser insuficiente. Por este motivo, la evaluación separa la tasa de tests pasados de una validación semántica posterior, donde los parches aceptados por tests se comparan con la solución de referencia del desarrollador. Así se evita contar como reparación completa un cambio que solo resuelve el caso probado.

El documento se organiza de la siguiente manera. En primer lugar, el Capítulo 2 presenta los antecedentes de APR, modelos de lenguaje y sistemas agénticos. Después, el Capítulo 3 fija los objetivos y las preguntas de investigación. A continuación, el Capítulo 4 describe el problema de diseño que plantea un sistema APR guiado por tests. Los Capítulos 5 y 6 explican la plataforma desarrollada y las mejoras introducidas durante su construcción. Finalmente, el Capítulo 7 presenta la evaluación experimental, y el último capítulo resume las conclusiones y las líneas de trabajo futuro.

2 Antecedentes y trabajos relacionados

La reparación automática de software no es una idea nueva. A comienzos de los 2000 ya aparecieron propuestas de reparación automática, como la reparación de estructuras de datos de Demsky y Rinard [3] o el enfoque *failure-oblivious* de Rinard et al. [23]. Más adelante, sistemas como GenProg [12] o PAR [11] trasladaron esa idea al parcheo automático de programas completos. Sin embargo, durante mucho tiempo estos sistemas dependieron de reglas, plantillas o búsquedas sobre espacios de cambios muy limitados. La llegada de los modelos de lenguaje de gran tamaño no elimina esos problemas, pero sí introduce una capacidad nueva: el modelo puede leer código, interpretar errores y usar información contextual que antes era muy difícil de incorporar [1,31].

Este capítulo sitúa el trabajo dentro de esa evolución. En primer lugar, se revisan los enfoques clásicos de APR y se explica por qué los tests son un indicador útil pero incompleto. Posteriormente, se introducen los modelos de lenguaje y el paso hacia agentes que pueden usar herramientas. Por último, se analizan los trabajos más cercanos y se aclara qué hueco ocupa el sistema desarrollado en este TFM, que no propone una técnica aislada de generación de parches, sino una comparación controlada de distintas formas de organizar agentes de reparación.

La idea central del capítulo es simple. La capacidad del modelo importa, pero el sistema que lo rodea también. En APR, esa diferencia se nota especialmente porque el agente no solo tiene que escribir código, sino decidir qué información leer, cuándo ejecutar tests y cuándo dejar de insistir en una hipótesis equivocada.

2.1 Reparación automática de software (APR)

La reparación automática de software estudia cómo generar parches correctivos para un programa defectuoso sin intervención humana. El planteamiento del problema es siempre el mismo: dado un programa que falla una o más pruebas, el objetivo es producir una versión candidata que las supere. Lo que ha cambiado a lo largo de los años es cómo se genera esa versión candidata y qué criterio se usa para decidir si es aceptable. Las subsecciones siguientes recorren los enfoques históricos que sentaron las bases del campo, el papel de los tests como criterio de corrección y sus límites, y la distinción entre parche plausible y parche correcto que estructura la metodología de evaluación de este TFM.

Evolución y enfoques clásicos. Durante más de dos décadas, la estrategia dominante en APR fue la de *generate-and-validate*: el sistema genera variantes del programa mediante mutaciones y las evalúa contra los tests disponibles, conservando las que los superan. El problema central de este enfoque no es técnico sino combinatorio, pues el espacio de modificaciones posibles es enorme y la mayor parte de ellas no tiene sentido desde el punto de vista de la lógica del programa.

Tres trabajos resumen bien esa etapa:

- **GenProg** [12] demostró que esta idea podía funcionar sobre bugs reales a escala. El sistema aplicaba programación genética directamente sobre el AST (del inglés, *Abstract Syntax Tree*) del programa, es decir, su representación estructural en forma de árbol. Operar sobre él permitía explorar inserciones, borrados y reemplazos de sentencias de manera sistemática.
- **RSRepair** [21], en cambio, mostró que una búsqueda aleatoria sobre ese mismo espacio producía resultados comparables a los de GenProg, sin mantener una población evolutiva. Por tanto, el cuello de botella no parecía estar en el algoritmo de búsqueda, sino en la calidad del espacio de mutaciones disponible.
- **PAR** [11] trató de mejorar precisamente ese punto. En lugar de mutar el código al azar, extraía plantillas de reparación a partir del historial de commits humanos, como ilustra la Figura 1. El enfoque resulta intuitivo, ya que muchos errores se corrigen de formas recurrentes, aunque su alcance queda limitado por los patrones que el sistema ya conoce.

Antes de la llegada de los LLMs, la siguiente generación de sistemas entrenó redes neuronales directamente sobre pares de código defectuoso y código corregido extraídos de repositorios públicos. DeepFix [7] y

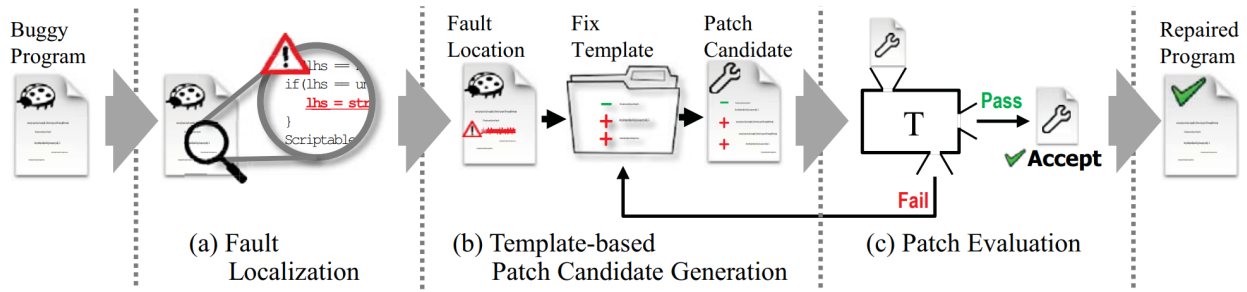


Fig. 1. Flujo general de la técnica clásica de APR basada en plantillas PAR: localización del fallo, generación de candidatos a partir de plantillas extraídas del historial de commits y evaluación contra los tests disponibles. Fuente: Kim et al. [11].

SequenceR [2] demostraron que los modelos podían aprender a corregir ciertos errores sintácticos y de compilación con una precisión notable, aunque su capacidad estaba limitada por el contexto local que podían procesar, normalmente unas pocas líneas de código. Eso los hacía poco efectivos en bugs que requerían razonar sobre funciones enteras o módulos interconectados.

APR basado en tests: ventajas y limitaciones. Una de las razones por las que el APR puede automatizarse es que los tests del proyecto sirven de criterio de corrección. Si el programa reparado supera todos los tests, el parche se considera plausible y puede pasar a una revisión más exigente. Este enfoque es práctico porque proporciona un criterio objetivo, evaluable de forma inmediata y sin intervención humana, lo que hace posible que un sistema explore muchos candidatos en un tiempo razonable.

Esta ventaja, sin embargo, no agota la dificultad del APR. El reto principal sigue siendo encontrar un cambio correcto dentro de un espacio de búsqueda enorme, mientras que la cobertura incompleta de tests es una limitación adicional de la validación, no la causa única de la dificultad del problema.

La limitación es que los tests solo cubren los casos que el desarrollador decidió verificar en su momento, no el comportamiento esperado del programa en todos los escenarios posibles. Un ejemplo concreto lo ilustra bien: supóngase una función que debe devolver el mínimo de una lista de números pero que por un error devuelve el máximo. Si los tests del proyecto solo verifican que para la lista [1, 3, 2] se devuelve 1, un parche que devuelva siempre el primer elemento pasaría esos tests. En este contexto, decir que un parche "encaja" con los tests significa justo eso: que satisface los casos concretos comprobados, aunque no represente todavía una corrección general del comportamiento esperado. Sin embargo, seguiría fallando en cualquier otro caso, porque no corrige el error de confundir el mínimo con el máximo.

Conviene matizar que el ejemplo anterior describe una batería de pruebas mala a propósito, precisamente para hacer visible el problema. En proyectos reales, el riesgo viene de algo más difícil de evitar. No es viable comprobar todos los caminos, todos los tamaños de entrada ni todas las interacciones entre módulos. Aumentar la cobertura reduce el riesgo, pero no lo elimina por completo.

El problema de los parches plausibles. Esta diferencia entre lo que los tests verifican y lo que el programa debería hacer fue recogida en [22], que distinguió entre parche *plausible* y parche *correcto*. El primero supera la batería de tests disponible, mientras que el segundo preserva también el comportamiento intencionado del programa en escenarios no cubiertos por esas pruebas. El fenómeno por el que un sistema APR genera parches plausibles pero incorrectos se conoce como sobreajuste a las pruebas, o *test overfitting*. En lugar de corregir el fallo, el sistema aprende a satisfacer los casos concretos que las pruebas verifican, es decir, se sobreajusta a ellos.

Los experimentos del trabajo demostraron que una parte significativa de los parches que superaban los tests en los sistemas de la época eran semánticamente incorrectos. Los sistemas no habían entendido el error sino que habían encontrado una modificación que satisfacía los tests. Aquí, "entender" no significa

comprensión humana completa del programa, sino disponer de información suficiente para producir un cambio que preserve la intención funcional más allá de los ejemplos de la batería de pruebas. Medir solo la tasa de tests superados ofrece, por tanto, una imagen parcial y a veces engañosa de la calidad real de los parches generados.

Por esta razón, la metodología de evaluación de este TFM no se limita al criterio de éxito o fracaso en los tests, sino que clasifica cada parche aceptado por la batería de pruebas como correcto, plausible o sobreajustado. De este modo se puede distinguir entre los sistemas que realmente corrigen el fallo y los que se limitan a satisfacer los tests como si fueran un oráculo completo del comportamiento esperado.

Benchmarks de evaluación en APR. El progreso en APR se mide sobre conjuntos de bugs estandarizados, los *benchmarks*. Cada benchmark proporciona, para cada bug, una versión defectuosa del programa, una batería de tests que reproduce el fallo y, habitualmente, el parche con el que un desarrollador lo corrigió. Esa estructura común permite que distintos sistemas se comparen sobre los mismos casos. Aun así, los benchmarks disponibles difieren mucho en lenguaje, escala y tipo de fallo, y esas diferencias determinan qué se está midiendo realmente.

Defects4J [9] es el conjunto de referencia histórico del campo. Reúne cientos de bugs reales de proyectos Java maduros y se convirtió en el banco de pruebas estándar de los sistemas *generate-and-validate*. Su influencia explica que buena parte de la literatura clásica de APR esté ligada a Java. QuixBugs [13], por el contrario, ocupa el extremo opuesto en escala. Contiene 40 programas pequeños, cada uno con la implementación defectuosa de un algoritmo clásico y disponible en Python y Java, donde el bug se concentra casi siempre en una sola línea. Su dificultad no está en el tamaño del repositorio sino en el razonamiento algorítmico, lo que lo ha convertido en una referencia habitual para evaluar la capacidad de reparación de los LLMs.

Entre ambos extremos se sitúa BugsInPy [27], que traslada la idea de Defects4J al ecosistema Python. Recopila fallos reales del historial de proyectos de uso extendido, con la versión exacta en la que se produjeron y los tests que los reproducen, de modo que combina realismo con un tamaño todavía manejable. Más allá de la reparación clásica, SWE-bench [8] amplía la escala hasta la resolución de *issues* de repositorios de GitHub, donde el sistema debe localizar y modificar el código a partir de una descripción en lenguaje natural en lugar de un fallo de test acotado. Es, por tanto, un problema más amplio que la reparación guiada por tests sobre la que se centra este trabajo.

Estos benchmarks no son intercambiables. Un mismo sistema puede destacar en bugs algorítmicos acotados y rendir mucho peor en fallos reales con dependencias de proyecto. Por ello, la elección del benchmark condiciona tanto las cifras publicadas como las conclusiones que pueden extraerse de ellas, un aspecto que se retoma al justificar los conjuntos empleados en la evaluación experimental.

2.2 De los modelos de lenguaje a los agentes autónomos

Los modelos de lenguaje a gran escala han transformado la forma en que se aborda la reparación de software. Para este TFM, su aportación principal no es solo que puedan generar código, sino que pueden integrarse en sistemas que leen un repositorio, ejecutan pruebas y corrigen su propio trabajo de forma iterativa. En este documento se llamará agente a uno de esos sistemas completos: un LLM con historial de mensajes, herramientas y un bucle de ejecución que le permite actuar sobre el entorno.

Fundamentos de los modelos de lenguaje. Un LLM es un modelo neuronal de lenguaje entrenado con grandes volúmenes de texto procedentes de internet, libros, repositorios de código y otras fuentes escritas. Su tarea fundamental es predecir, dado un fragmento de texto como entrada, cuál es el siguiente elemento más probable. Ese elemento no es una palabra sino un *token*, la unidad básica con la que el modelo trabaja internamente, que puede corresponder a una palabra completa, a parte de ella o a un signo de puntuación.

La generación de texto se realiza de forma secuencial. El modelo genera un primer token, lo incorpora a la entrada y vuelve a predecir el siguiente a partir de ese contexto ya ampliado. Este proceso, conocido

como generación autoregresiva, se repite hasta alcanzar una marca de fin de respuesta o el límite de tokens predefinido.

La arquitectura que hace posible este comportamiento a escala es el Transformer [26], cuya pieza central es el mecanismo de atención. A diferencia de las redes neuronales recurrentes, que procesaban el texto de forma lineal y tenían dificultades para conservar dependencias lejanas, el mecanismo de atención permite que cada token calcule, en paralelo, cuánta importancia dar a cada uno de los demás tokens del contexto. En código, esta propiedad resulta especialmente útil porque permite relacionar una declaración con su uso posterior, asociar un mensaje de error con la función que lo ha producido o detectar que una corrección afecta a otra función que la llama desde un módulo distinto.

Cuando este proceso se entrena a escala suficiente, el modelo aprende patrones generales del lenguaje natural y del código. Sin embargo, predecir texto no equivale por sí solo a seguir instrucciones. Por eso, los modelos que se usan como asistentes suelen pasar por etapas adicionales de ajuste, entre ellas RLHF (del inglés, *Reinforcement Learning from Human Feedback*), donde respuestas generadas por el modelo se comparan con preferencias humanas para favorecer comportamientos más útiles y controlables.

La API de chat y el modelo de turnos. En la práctica, los LLMs no se invocan directamente como funciones. Se accede a ellos a través de una API, tanto a nivel local como remoto. La API de *chat completions*, introducida por OpenAI y adoptada como estándar por la práctica totalidad de proveedores de modelos, expone un único punto de entrada que recibe una lista de mensajes y devuelve la respuesta del modelo. Cada mensaje tiene un rol y un contenido de texto. En el intercambio básico aparecen tres roles principales.

system Contiene las instrucciones generales sobre cómo debe comportarse el modelo durante la sesión.

user Representa las entradas del usuario o del sistema que invoca al modelo.

assistant Corresponde a las respuestas anteriores del propio modelo.

El conjunto formado por un mensaje de usuario y la respuesta del modelo constituye un *turno* de la conversación. Varios turnos encadenados forman el historial que el sistema mantiene y transmite en cada llamada a la API.

Un aspecto que no es evidente a primera vista es que el LLM no tiene estado entre llamadas. No recuerda nada de una petición anterior, porque cada llamada a la API es completamente independiente. Lo que parece memoria conversacional no es una propiedad del modelo sino del sistema que lo llama. Para que el modelo tenga en cuenta lo que dijo en pasos anteriores, quien lo invoca tiene que incluir esas respuestas pasadas explícitamente, bajo el rol **assistant**, en la lista de mensajes de la siguiente petición. El historial completo de la conversación viaja en cada llamada.

Esto tiene consecuencias directas sobre el diseño de agentes. Si en una iteración de reparación el modelo propone un parche incorrecto, la siguiente llamada tendrá que incluir, además del nuevo mensaje con el resultado de los tests, todos los mensajes del intercambio anterior. Con cada iteración la lista crece, y cuando se acerca al límite de la ventana de contexto el sistema tiene que decidir qué mensajes descartar o resumir. Esa decisión afecta directamente a lo que el modelo puede razonar en el paso siguiente.

El mecanismo de *function calling* se integra de forma natural en este modelo de mensajes. Cuando el modelo quiere ejecutar una herramienta, en lugar de responder con texto libre devuelve una estructura con el nombre de la función y los argumentos que quiere usar. El sistema que lo invoca detecta esa estructura, ejecuta la función correspondiente, ya sea leer un fichero, ejecutar los tests o buscar en el repositorio, y devuelve el resultado como un nuevo mensaje de rol **tool**. Este cuarto rol aparece cuando la conversación incorpora herramientas, de modo que el modelo recibe ese resultado dentro de su contexto y continúa su razonamiento. Este mecanismo, junto con la gestión del historial, es la base técnica sobre la que se construyen todos los agentes de APR descritos en este capítulo, como ilustra la Figura 2.

La secuencia es simple. El sistema mantiene el historial, ejecuta las herramientas y devuelve el resultado al modelo, mientras que el modelo decide cuándo pedir una acción y cuándo cerrar la respuesta. La Figura 2 resume ese intercambio.

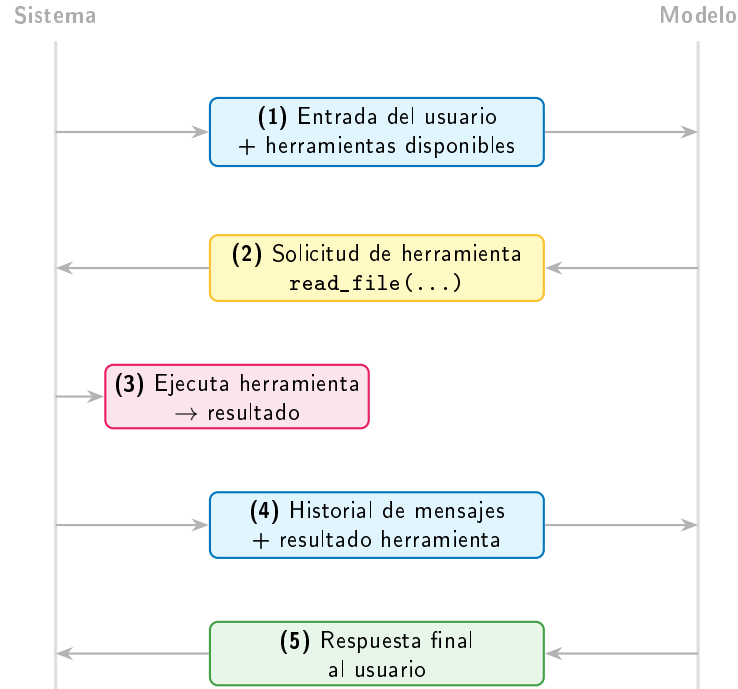


Fig. 2. Ciclo básico de *function calling* entre sistema y modelo.

Modelos de lenguaje y generación de código. La aplicación de estas capacidades al código fue una evolución natural. Los primeros modelos grandes, como GPT-2 o BERT, ya podían manejar fragmentos de código, pero no estaban especializados en tareas de programación y sus resultados eran limitados. Uno de los primeros hitos importantes llegó con Codex [1], un modelo que OpenAI entrenó específicamente sobre decenas de miles de repositorios de GitHub. La evaluación sobre HumanEval, un benchmark de 164 problemas de programación en Python, reveló que Codex obtenía un 28.8% en la métrica *pass@1*, lo que significa que en casi tres de cada diez problemas era capaz de generar de primeras una solución que pasase todos los tests.

Desde la perspectiva del APR, lo más interesante no es ese porcentaje sino cómo el modelo adquiere capacidades de reparación de forma indirecta. El entrenamiento sobre historiales de commits incluye, de forma natural, muchos ejemplos del patrón código defectuoso → código corregido. Cada vez que un desarrollador sube un commit con el mensaje “fix bug in X”, el modelo puede ver ese par de versiones durante el preentrenamiento. No aprende a reparar de forma explícita, pero incorpora esos patrones como parte del aprendizaje general del lenguaje del código. Desde Codex, modelos como StarCoder, Code Llama o Qwen-Coder han continuado especializando esta capacidad, entrenándose sobre corpus más grandes y con técnicas de fine-tuning orientadas a tareas de programación.

El salto al paradigma agéntico. Que un modelo sepa generar código no lo convierte en un agente. Para reparar un bug en un proyecto real necesita algo más: explorar el repositorio, ejecutar los tests, leer el resultado y, a partir de todo eso, decidir qué cambiar y dónde. Nada de esto es posible con un modelo que solo recibe texto y devuelve texto.

El salto al paradigma agéntico se produjo cuando el mecanismo de *function calling* comenzó a utilizarse no solo para completar una tarea puntual, sino para que el modelo tomara decisiones dentro de un bucle. ReAct [32] propuso combinar razonamiento explícito en lenguaje natural con llamadas a herramientas en un ciclo de observación, pensamiento y acción. En lugar de generar directamente una respuesta, el modelo razona sobre la situación, decide qué acción tomar, la ejecuta y observa el resultado antes de continuar. El efecto es un agente más fácil de depurar que un sistema que solo devuelve una respuesta final.

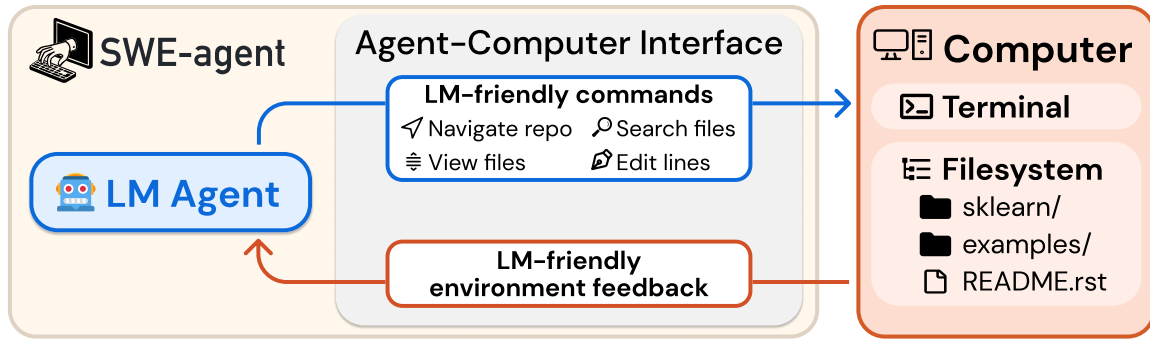


Fig. 3. Relación entre el agente, la interfaz agente-ordenador y el entorno del repositorio en SWE-agent. Fuente: Yang et al. [31].

Este modelo se ha materializado también en herramientas de desarrollo. GitHub Copilot Workspace [5], por ejemplo, se presentó como una forma de describir una tarea en lenguaje natural y delegar en el agente la lectura del repositorio, la propuesta de cambios y la preparación de una *pull request*. Lo relevante para este trabajo no es una herramienta concreta, sino el patrón común que aparece en todas ellas. El modelo deja de ser un generador aislado de texto y pasa a participar en un ciclo de edición, prueba y revisión del código.

Agent harness: infraestructura para la autonomía. Una diferencia importante entre un modelo de lenguaje y un agente operativo es la presencia de un *harness*, es decir, la infraestructura de ejecución que envuelve al modelo y le da acceso a herramientas, mantiene el historial de la conversación y gestiona el bucle de ejecución. Sin esa capa, el modelo es solo una función de texto a texto. Con ella, puede actuar de forma continuada sobre un entorno real y adaptar su comportamiento en función del resultado de sus acciones.

El diseño de este harness afecta directamente al comportamiento del agente. En concreto, obliga a tomar varias decisiones prácticas:

- qué herramientas ofrecer al modelo y cómo describirlas;
- cómo gestionar el historial de la conversación a medida que crece;
- qué hacer cuando el modelo devuelve una llamada a herramienta con argumentos incorrectos;
- cuándo cortar el bucle si el agente no converge.

SWE-agent [31] pone de manifiesto la importancia de estas decisiones. Introduce el concepto de *Agent-Computer Interface* (ACI), un conjunto de herramientas diseñado específicamente para que el modelo pueda interactuar con el sistema de ficheros, el terminal y los tests de manera eficiente, como se muestra en la Figura 3. Una de las conclusiones más relevantes del trabajo es que variaciones en el diseño del harness producen diferencias de rendimiento mayores que cambiar el modelo subyacente. En este TFM, el harness es el componente que construye el contexto de cada iteración, invoca al modelo, captura las herramientas que este solicita y decide cuándo continuar o detener el proceso.

Context engineering. A medida que los sistemas agénticos se han vuelto más complejos, ha cobrado relevancia una práctica complementaria al diseño de *prompts*, denominada *context engineering*. Karpathy popularizó el término para referirse a la selección de la información que entra en la ventana de contexto antes de cada llamada al modelo [10]. La idea es sencilla de formular, aunque difícil de aplicar bien. El prompt fija las instrucciones generales, mientras que el contexto decide qué sabe el modelo en un paso concreto. En la práctica, esto significa escoger qué parte del historial conservar, qué indicadores del entorno priorizar, cómo presentar el resultado de los tests fallidos o qué herramientas describir y cuáles omitir.

En el caso de un sistema de APR, las cuestiones que surgen son muy concretas: ¿se incluye el código completo del fichero afectado o solo las líneas relevantes? ¿Se muestran los mensajes de error de todos los

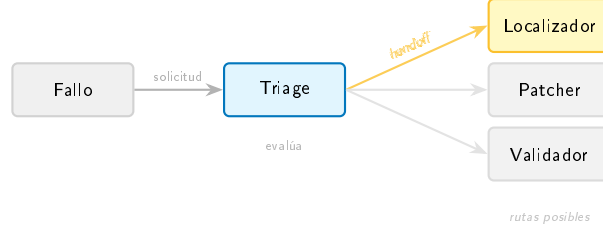


Fig. 4. Patrón *handoff* con transferencia de control a un especialista.

tests fallidos o solo los más informativos? ¿Se conserva el historial de los últimos intentos de reparación o solo el más reciente? En la práctica, durante el desarrollo de este sistema se observó que muchos errores del agente no procedían necesariamente de que el modelo fuese incapaz de generar el parche correcto, sino de que el contexto recibido no le daba la información necesaria para razonar con precisión. Gran parte del trabajo de mejora del sistema desarrollado en este TFM ha consistido precisamente en ajustar qué información recibe el agente en cada paso.

Arquitecturas multi-agente. Un agente único equipado con todas las herramientas necesarias puede, en principio, abordar cualquier tarea de reparación. En la práctica, concentrar todo el proceso en una única instancia tiene un coste. Las instrucciones tienen que ser genéricas para cubrir cada fase del ciclo, el contexto acumula información de naturalezas muy distintas y el razonamiento sobre qué paso dar compete con el razonamiento sobre el fallo concreto que se está analizando. Las arquitecturas multi-agente descomponen el problema en subtarefas con responsabilidades acotadas y las asignan a instancias especializadas.

La coordinación multiagente suele materializarse de dos maneras. Una de ellas es el *handoff*, representado en la Figura 4. En este patrón, un agente enrutador evalúa la solicitud entrante y transfiere el control de la conversación al agente especializado más adecuado para ese tipo de tarea. Como la transferencia es unidireccional, el agente receptor asume el control completo de la interacción sin que el emisor intervenga en el resultado final.

La otra posibilidad consiste en exponer un agente como herramienta invocable por otro, siguiendo el patrón *agent as tool*, representado en la Figura 5. En este caso, un agente coordinador conserva la autoridad de decisión a lo largo de todo el proceso y, en vez de realizar todas las acciones por su cuenta, delega ciertas capacidades en subagentes que actúan como si fueran funciones. De este modo, puede llamarlos cuando lo necesite, recibir su resultado y decidir si volver a invocarlos con información adicional antes de continuar. A diferencia del *handoff*, el coordinador mantiene el control del flujo en todo momento.

La diferencia entre ambos mecanismos no es solo terminológica. Afecta a cómo se reparte la responsabilidad durante la reparación:

- en un *handoff*, el agente que recibe la tarea asume el control y continúa la interacción.
- en un esquema *agent as tool*, el coordinador conserva el control y usa subagentes para obtener información o validaciones parciales.
- en ambos casos, la coordinación consume contexto y tokens, por lo que más agentes no implica automáticamente una reparación mejor.

En este TFM se implementan varias arquitecturas para estudiar esos compromisos sobre una misma infraestructura experimental, incluyendo una línea base mono-agente.

Las Figuras 4 y 5 muestran ambos patrones de forma esquemática. En el *handoff* el control cambia de manos cuando se transfiere la conversación. En cambio, en el patrón *manager* el agente coordinador conserva el control y solo delega subtarefas concretas.

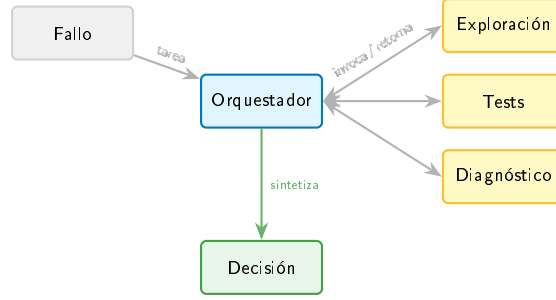


Fig. 5. Patrón manager con orquestador centralizado y subagentes invocables.

2.3 LLMs aplicados al APR

Los trabajos que se describen a continuación combinan las dos líneas revisadas anteriormente. Todos usan LLMs para generar candidatos de reparación y se evalúan sobre benchmarks de APR, pero no conceden la misma autonomía al modelo. Algunos generan un parche en un único paso, mientras que otros incorporan *feedback* de los tests o permiten que el agente use herramientas. Esa diferencia es importante porque determina qué información puede consultar el sistema antes de editar y cuánto coste asume durante la reparación.

Enfoques basados en generación directa. En [20] se llevó a cabo una de las primeras evaluaciones sistemáticas de un LLM sobre QuixBugs. Usando Codex en modo *zero-shot*, es decir, sin ejemplos previos ni ajuste específico para el problema, el sistema reparó 27 de los 40 programas Python del conjunto. El proceso era sencillo. Se pasaba al modelo el código del programa defectuoso junto con el resultado del test que fallaba, y se esperaba que el modelo devolviera una versión corregida. Que un sistema tan directo funcionara en casi el 70% de los casos confirmó que los LLMs habían aprendido patrones de reparación suficientemente generales como para resolver bugs algorítmicos sin ejemplos específicos.

Es importante matizar que, en este tipo de enfoque directo, el modelo no busca por sí mismo qué partes del repositorio leer. El contexto relevante se selecciona fuera del modelo y se le entrega ya acotado en el prompt de entrada.

Otros trabajos posteriores reforzaron la misma lectura desde configuraciones más amplias. En [29] se compararon distintos modelos preentrenados sobre varios conjuntos de datos y lenguajes, mostrando que los modelos grandes podían superar a técnicas APR clásicas incluso sin estar entrenados específicamente para reparación. En QuixBugs, estos trabajos se acercan mucho a saturar el benchmark cuando se proporciona localización perfecta o un contexto muy acotado. El análisis de ChatGPT en [25] obtuvo 31 reparaciones de 40 en el mismo benchmark mediante interacción con información adicional. En conjunto, estos resultados ayudan a interpretar QuixBugs como una referencia útil, aunque no necesariamente como el escenario que mejor distingue entre arquitecturas de agentes.

También conviene interpretar estas cifras con cierta cautela. QuixBugs es un benchmark público y, como ocurre con otros conjuntos muy difundidos, no puede descartarse por completo que parte de sus casos haya aparecido durante el preentrenamiento de algunos modelos. Esa posible contaminación no invalida la comparación, pero sí hace aconsejable apoyarla después con un benchmark más realista como BugsInPy.

La limitación de este enfoque es estructural. El modelo recibe el contexto una vez, genera un candidato y no puede hacer nada más si ese candidato falla. No puede consultar otros ficheros del proyecto, no puede ver el resultado de los tests ni corregir su propia propuesta. Es una aproximación útil como línea base, pero sus posibilidades de mejora están acotadas por su propio diseño.

Sistemas agénticos de reparación iterativa. Para superar esa limitación, ChatRepair [30] propuso aprovechar el historial conversacional para que el modelo pudiera iterar. Después de generar un parche, el sistema devuelve al modelo el resultado de los tests, incluyendo los mensajes de error, y le pide que intente

corregirlo. Con este mecanismo, sencillo en su concepción, se consiguió reparar 162 de los 337 bugs del conjunto Defects4J a un coste medio de 0.42 \$ por bug. El resultado muestra que el feedback del entorno puede mejorar de forma notable una reparación basada en LLMs, especialmente cuando la primera propuesta no es correcta.

SWE-agent [31] fue un paso más allá, equipando al agente con una interfaz completa de interacción con el repositorio, con lectura de ficheros, búsqueda, edición y ejecución de comandos. Esto le permite no solo iterar sobre su propio parche, sino también explorar el código del proyecto antes de proponer ningún cambio. En SWE-bench, que consiste en resolver incidencias reales de proyectos de GitHub, obtuvo una tasa de resolución del 12.5 %. La conclusión más relevante para este TFM no es ese número aislado, sino que el diseño del harness influye mucho en el rendimiento final.

Agentless [28] representa una propuesta en sentido contrario. En lugar de dar al modelo autonomía completa para decidir futuras acciones, organiza la resolución en tres fases más cerradas, que son localización, reparación y validación del parche. Con esta estructura obtiene resultados competitivos en SWE-bench con menor coste que muchos agentes más complejos. Esto apunta directamente a una idea importante para este TFM. La complejidad de la arquitectura no garantiza mejor rendimiento, y la eficiencia del procedimiento es una dimensión de evaluación tan relevante como la tasa de éxito.

La evolución de estos sistemas refleja un reconocimiento creciente de que el rendimiento de un agente de APR depende no solo del modelo subyacente sino del sistema que lo orquesta. Sin embargo, la comparación sistemática entre patrones de coordinación distintos, aplicados al mismo conjunto de bugs y bajo las mismas condiciones, sigue siendo escasa en la literatura.

En paralelo, han aparecido trabajos recientes que usan BugsInPy para estudiar el papel del contexto en APR con LLMs. HAFixAgent [24] incorpora información del historial del repositorio para orientar la reparación. Ehsani et al. [4] proponen una inyección jerárquica de conocimiento, desde información local del bug hasta contexto de repositorio y proyecto. Parasaram et al. [19] formulan, por su parte, el problema de seleccionar qué hechos incluir en el *prompt*, mostrando que añadir más información no siempre mejora el resultado. Estos trabajos son importantes porque evitan una lectura demasiado simple de BugsInPy. No se trata de un benchmark sin explorar por la literatura reciente, sino de un escenario donde el diseño del contexto y del procedimiento de reparación tiene un peso claro.

2.4 Trabajos relacionados

Los trabajos descritos hasta aquí comparten el objetivo de reparar software de forma automática, pero no todos intentan responder a la misma pregunta. Algunos se centran en mejorar la selección del contexto, otros en usar historial del repositorio y otros en reducir el coste de la autonomía. En este TFM, en cambio, la pregunta principal es más acotada. Se estudia qué cambia cuando se mantiene fija la plataforma de ejecución y se varía la forma de coordinar al agente reparador.

Los trabajos más directamente relacionados con la comparación que se presenta en este TFM son los siguientes.

De todos ellos, el trabajo más cercano en planteamiento general es ChatRepair [30], porque también estudia un ciclo iterativo en el que el modelo propone un parche, recibe el resultado de los tests y vuelve a intentarlo. Si se mira solo el rendimiento publicado en QuixBugs, la referencia más fuerte es [29], que se acerca a saturar el benchmark cuando el contexto está muy guiado o la localización del fallo es prácticamente perfecta.

- En ChatRepair [30] el modelo recibe el resultado de cada ejecución de tests como parte del diálogo y la reparación avanza iterativamente. Es exactamente el mismo ciclo que usan las arquitecturas evaluadas en este TFM.
- SWE-agent [31] concluye que el diseño del harness puede tener más impacto en el rendimiento que el modelo subyacente, una hipótesis que este trabajo contrasta de forma sistemática entre varias arquitecturas.
- Agentless [28] es relevante por el contraste entre enfoques: fases acotadas sin autonomía completa pueden ser competitivas con agentes más libres y a un coste menor, lo que obliga a no asumir que más complejidad arquitectónica implica mejor resultado.

- Sobre QuixBugs ya existe bibliografía con LLMs [20,29,25] que sitúa el benchmark cerca de la saturación cuando el contexto está muy guiado. Eso condiciona cómo interpretar las cifras en ese conjunto.
- Los trabajos recientes sobre BugsInPy estudian qué contexto incluir y cómo estructurarlo [19,4,24]. Son especialmente relevantes porque usan el benchmark principal de este TFM y permiten situar la propuesta frente a estrategias centradas en la selección de contexto.

En QuixBugs, los resultados publicados permiten situar el rendimiento del sistema frente a un benchmark muy conocido. Sin embargo, muchas configuraciones de la literatura usan localización perfecta o generación directa sobre fragmentos ya acotados, por lo que sus cifras no son equivalentes a las de un agente que tiene que explorar, editar y validar mediante herramientas. En BugsInPy ocurre algo distinto. Los trabajos recientes de reparación con LLMs se centran en técnicas concretas de inyección o selección de contexto. Por eso son una referencia útil para interpretar los resultados, aunque no constituyen una comparación directa arquitectura por arquitectura.

La evaluación incorpora además una dimensión semántica explícita. Medir solo el porcentaje de tests superados no distingue entre un agente que corrige el fallo y otro que ajusta el parche a las aserciones del test. Esta limitación está relacionada con trabajos recientes de evaluación de parches y detección de sobreajuste [34,33], que muestran que la aceptación por tests no basta para estimar la corrección real. Por otro lado, métricas léxicas como BLEU [18] tampoco resuelven el problema, ya que miden solapamiento textual y no equivalencia funcional. En tareas de código se ha observado que la similitud sintáctica puede correlacionar mal con la corrección semántica [15].

Por ese motivo, este TFM separa explícitamente los indicadores de eficacia, calidad semántica y coste. El detalle de esos indicadores se fija en los objetivos específicos (Sección 3.2), para que la evaluación posterior use criterios estables y comparables entre arquitecturas y modelos.

En consecuencia, la comparación que se presenta más adelante no se reduce a preguntar qué modelo obtiene más tests superados. La comparación es más amplia. Se estudia qué configuración repara más bugs, con qué coste y con cuántos parches que solo parecen correctos a primera vista.

3 Objetivos y preguntas de investigación

Este capítulo fija qué se quiere comprobar con el trabajo y bajo qué límites deben interpretarse sus resultados. La comparación que se persigue no consiste solo en lanzar varios modelos sobre unos cuantos bugs, ya que para atribuir una diferencia a la arquitectura o al modelo también hay que mantener constantes el entorno, el flujo de reparación y la forma de validar los parches.

El estudio se apoya, por tanto, en tres condiciones básicas:

- disponer de una plataforma común que ejecute todos los intentos bajo las mismas condiciones;
- comparar arquitecturas de agentes que resuelven la misma tarea con formas distintas de organizar el trabajo;
- evaluar los resultados separando la superación de tests de la corrección real del parche.

En primer lugar, se formulan las preguntas de investigación que se responderán en la evaluación experimental (ver Sección 3.1). Posteriormente, se presentan los objetivos específicos que guiaron el desarrollo (ver Sección 3.2). Por último, se delimita el alcance del sistema para dejar claro en qué contexto son válidos los resultados obtenidos (ver Sección 3.3).

Con ello, el capítulo fija una comparación concreta entre configuraciones ejecutadas bajo un mismo entorno y deja preparados los criterios que se usarán más adelante para interpretar las diferencias observadas.

3.1 Preguntas de investigación

Las dos preguntas de investigación responden a los dos ejes de variación del experimento, que son la estrategia de orquestación (RQ1) y el modelo LLM empleado (RQ2). Mantenerlas separadas permite analizar si las diferencias observadas se explican mejor por la arquitectura del agente, por el modelo que la ejecuta o por la combinación de ambos factores.

RQ1. *¿Cómo afecta la estrategia de orquestación (mono-agente frente a multi-agente con roles) a la eficacia de la reparación automática y a la calidad de los parches generados?*

La pregunta compara arquitecturas que difieren en cómo distribuyen la responsabilidad del proceso de reparación. La *eficacia* se mide por la tasa de bugs reparados con éxito. La *calidad* del parche evalúa si la corrección es semánticamente coherente con el comportamiento esperado del módulo, más allá de que los tests simplemente pasen.

Además de contar cuántos bugs corrige cada arquitectura, se observa si una organización del trabajo tiende a explorar más código, a necesitar más iteraciones o a producir parches que pasan los tests pero no corrigen del todo el fallo. De este modo, la comparación no depende únicamente del resultado final, sino también del proceso seguido para alcanzarlo.

RQ2. *¿Qué diferencias se observan al usar distintos LLMs en términos de eficacia, eficiencia y coste?*

La pregunta compara el rendimiento de modelos con distintas características, incluyendo modelos locales de distintos tamaños y un modelo propietario accesible a través de API. En este caso no solo interesa saber qué modelo corrige más bugs, sino también cuánto trabajo necesita para hacerlo. Por ello se comparan la tasa de reparación, el consumo de tokens, el número de iteraciones, la estabilidad durante el uso de herramientas y el coste económico estimado.

3.2 Objetivos específicos

El objetivo principal del TFM es estudiar de forma experimental la influencia de la orquestación de agentes y del modelo LLM en la reparación automática de errores en software Python. Para ello, se han definido cuatro objetivos concretos:

OB-1. Diseñar y desarrollar una plataforma experimental de reparación automática apoyada en LLMs y agentes. Las plataformas APR existentes están orientadas a técnicas clásicas o fijan un único flujo de ejecución, por lo que no permiten comparar con facilidad distintas arquitecturas de agentes. La plataforma desarrollada debe recibir un fallo reproducible, permitir que el agente explore el código, aplicar la corrección propuesta y verificarla mediante tests dentro de un contenedor Docker temporal.

- OB-2.** Diseñar e implementar distintas arquitecturas de orquestación de agentes que puedan compararse experimentalmente. Cada arquitectura evaluada representa una forma distinta de organizar la reparación, desde un único agente con acceso a todo el contexto hasta esquemas donde el trabajo se reparte entre agentes con responsabilidades más acotadas. Esta variación permite estudiar la orquestación como una variable del experimento. Además, se implementó una variante con *handoff* para explorar el espacio de diseño, aunque no se incluyó en la comparación formal por la inestabilidad observada durante las pruebas preliminares.
- OB-3.** Definir un conjunto de métricas para evaluar tanto la eficacia de la reparación como la eficiencia del procedimiento. Superar los tests es una condición necesaria, pero no siempre suficiente, ya que un parche puede ajustarse a las aserciones sin resolver la causa real del fallo. Por este motivo, la evaluación considera tres tipos de información:
- la tasa de bugs cuyos tests pasan tras la reparación;
 - la clasificación semántica del parche como correcto, plausible o sobreajustado;
 - el coste del proceso, medido mediante tokens, iteraciones y coste estimado.
- OB-4.** Evaluar el sistema sobre benchmarks establecidos de APR, ejecutando el experimento bajo distintas combinaciones de arquitectura de agentes y modelo LLM. Usar QuixBugs [13] y BugsInPy [27] sitúan los resultados en benchmarks conocidos y reduce el riesgo de escoger bugs que favorezcan a una arquitectura concreta.

3.3 Alcance del sistema

Los resultados de este trabajo deben leerse dentro del marco experimental en el que se han obtenido. El sistema se ha diseñado para un tipo concreto de reparación, con proyectos Python, bugs reproducibles mediante tests y ejecución controlada en contenedores. Dentro de ese marco, el sistema sí contempla los siguientes elementos:

- repositorios Python con tests fallidos que reproducen el bug;
- ejecución aislada de cada intento de reparación;
- comparación entre varias arquitecturas de agentes y varios modelos LLM;
- evaluación sobre QuixBugs [13] y BugsInPy [27].

Quedan **fuera del alcance** los siguientes aspectos:

- La localización de fallos en situaciones donde no existe una traza de error ni tests que fallen de forma reproducible.
- La reparación de proyectos en lenguajes distintos de Python. En ese mismo punto queda fuera Defects4J, que es el benchmark histórico del área pero está centrado en Java y exigiría otra cadena de preparación, otra instrumentación y otro conjunto de herramientas.
- Los fallos relacionados con configuración de entornos, dependencias de sistema o condiciones de carrera.
- El uso de herramientas de localización automática de fallos independientes del LLM. La localización del error la realiza el propio modelo a partir de la información disponible en cada ejecución.
- La navegación web y el uso de servicios externos durante la reparación. Toda la ejecución tiene lugar en un contenedor Docker temporal sin conectividad de salida.

Por tanto, la evaluación se limita a las arquitecturas y modelos estudiados en este trabajo. Los resultados son útiles para comparar estas configuraciones bajo el diseño experimental descrito en el Capítulo 7. Estas exclusiones pueden afectar a los resultados, pues incorporar navegación web, localización de fallos independiente o benchmarks de otros lenguajes podría alterar las tasas absolutas de reparación, porque cambiaría tanto la evidencia disponible como el coste del procedimiento. Precisamente por eso se dejan fuera, ya que el objetivo del experimento no es maximizar el rendimiento, sino comparar arquitecturas y modelos dentro de un entorno controlado y reproducible. Por ejemplo, un sistema orientado a resolver incidencias abiertas de GitHub tendría que trabajar con descripciones en lenguaje natural, repositorios de mayor tamaño y necesidades de exploración distintas a las de un benchmark guiado por tests, por lo que requeriría una evaluación propia.

4 Planteamiento del problema

Antes de construir la plataforma fue necesario concretar qué significa reparar un bug en este trabajo. La entrada no es una descripción redactada por una persona, sino una batería de tests que falla, una traza de error y un repositorio que el agente puede explorar. En ese escenario, reparar consiste en localizar el código relevante, aplicar un cambio justificable y dejar suficiente evidencia para saber si el resultado es realmente aceptable.

En primer lugar, se describe el ciclo de reparación guiado por tests que constituye el mecanismo central del sistema (ver Sección 4.1). A continuación, se identifican los retos de diseño que aparecen al llevar ese ciclo a una plataforma real, especialmente la exploración del código, el sobreajuste a tests y el control del proceso iterativo (ver Sección 4.2). Por último, esos retos se traducen en requisitos funcionales y no funcionales del sistema (ver Sección 4.3).

4.1 El ciclo de reparación automática basado en tests

El punto de partida de cualquier ejecución es un fallo reproducible: un conjunto de tests que no pasan y una traza de error, o *traceback*, que los acompaña. La traza ofrece un primer punto de entrada para investigar el repositorio y formular una hipótesis inicial sobre la causa del fallo. A partir de ahí, el sistema pone en marcha un bucle iterativo de cuatro fases:

1. **Comprensión.** El agente lee el *traceback* para identificar el fichero, la función y la línea donde se origina el error. A continuación consulta el código fuente del módulo afectado para entender el comportamiento esperado y construir una hipótesis sobre qué está mal y por qué.
2. **Localización.** Con la hipótesis como guía, el agente determina qué ficheros y fragmentos concretos deben modificarse. Si el error apunta a un problema más amplio, puede consultar módulos relacionados para acotar el alcance real del cambio antes de editar nada.
3. **Edición.** El agente aplica los cambios mediante herramientas de escritura estructurada. Cada modificación queda registrada como una operación de herramienta verificable y trazable.
4. **Validación.** El sistema ejecuta los tests sobre el repositorio modificado y captura el resultado. Si todos pasan, el ciclo puede concluir. En caso contrario, la salida de los tests se incorpora como entrada de la siguiente iteración para que el agente pueda ajustar su hipótesis.

Estas cuatro fases resumen el ciclo a nivel de problema. En el capítulo siguiente, la implementación se descompone en pasos operativos más pequeños para preparar la iteración, ejecutar el agente, observar los cambios y decidir si el intento debe continuar.

El cierre de una ejecución puede darse de tres formas:

- si los tests pasan, el sistema conserva el parche generado en formato *unified diff*, que es la representación textual estándar de los cambios añadidos y eliminados por fichero;
- si los tests siguen fallando pero el repositorio ha cambiado, se inicia una nueva iteración con el estado actualizado y la salida de tests más reciente;
- si se alcanza el límite de iteraciones o se encadenan intentos sin cambios observables en el código fuente, con el mismo resultado compacto de tests y una hipótesis que se repite, la ejecución termina de forma controlada y deja constancia del parche parcial, los *logs* y los resultados obtenidos.

Una característica que distingue a este enfoque del APR clásico es el papel que juega el *traceback* como punto de partida para la localización. En los sistemas tradicionales, localizar el fallo es un paso explícito y costoso, ya que se recurre a técnicas de análisis de cobertura, *program slicing* o diagnóstico estadístico para identificar qué líneas son candidatas al parche. En un sistema de *test-guided APR*, en cambio, la traza suele indicar dónde se manifiesta el error y permite empezar por una zona concreta del repositorio. Esto reduce la exploración inicial. A partir de ese punto, el agente debe comprobar si la zona visible coincide con la causa real del bug o si solo es el lugar donde el fallo acaba apareciendo [30,31].

4.2 Retos de diseño

Aunque el ciclo anterior se puede explicar como una secuencia ordenada, en la práctica no funciona de forma tan limpia. El sistema se enfrenta a tres dificultades acumulativas:

- el agente no sabe de antemano qué ficheros importan;
- los tests solo muestran una parte del comportamiento;
- cada iteración consume tiempo y tokens.

Por tanto, el problema no consiste únicamente en generar un parche, sino en guiar la exploración y la validación sin convertir cada intento en una búsqueda abierta por todo el proyecto.

En concreto, el diseño del sistema tuvo que responder a tres preguntas:

- qué parte del repositorio debe leer el agente antes de editar;
- cómo evitar que el test fallido se confunda con la especificación completa del comportamiento;
- qué límites necesita el bucle para que el coste y los resultados sean comparables.

Localización y exploración del código. Un agente autónomo no dispone de ningún mapa del repositorio, por lo que para localizar el código relevante tiene que explorar activamente: leer ficheros, seguir importaciones, buscar las funciones mencionadas en el *traceback* y entender cómo se relacionan con el resto del módulo. Esta exploración tiene un coste directo en tokens de contexto y, además, existe una tensión real entre dos extremos indeseables. Explorar demasiado implica consumir contexto en código irrelevante, mientras que explorar demasiado poco puede llevar a proponer una corrección sin comprender el contrato completo de la función afectada.

Una forma de reducir esa incertidumbre consiste en proporcionar al modelo una localización previa del fallo, de modo que reciba directamente el fragmento donde debe actuar. Este trabajo adopta una opción distinta. El *traceback* se usa como punto de entrada, pero el agente debe decidir qué consultas adicionales necesita mediante las herramientas disponibles. Así, cada arquitectura se evalúa también por su forma de administrar el contexto durante la exploración.

Sobreajuste a los tests. La batería de tests es el único indicador automático de corrección del que dispone el sistema durante la reparación. Como se explicó al tratar el problema de los parches plausibles en el capítulo de antecedentes, ese indicador es parcial por naturaleza. Un conjunto de tests cubre solo una parte del comportamiento del módulo, de modo que el agente puede producir un parche *plausible* que supera la validación sin restaurar el comportamiento intencionado de la función.

Este riesgo aumenta en un sistema iterativo. Después de cada intento, el agente no recibe una descripción completa de lo que sigue estando mal, sino el resultado de los tests ejecutados. Si esos tests no ejercitan una rama, una función invocante o un uso alternativo de la misma interfaz, el agente tampoco recibe una pista clara para revisarlo.

Este fenómeno se ha observado directamente durante el desarrollo, al comparar las correcciones del sistema con los parches canónicos de BugsInPy [27]. En varios casos, el agente produjo correcciones semánticamente incompletas. Resolvían el síntoma visible que ejercitaban los tests, pero omitían cambios de propagación que sí estaban presentes en la corrección real del desarrollador. El ejemplo más ilustrativo es un bug de FastAPI en el que la causa raíz afectaba a cuatro ficheros. El agente detectó el punto de fallo visible en uno de ellos, lo corrigió y los tests pasaron. Sin embargo, no había ningún indicio automático de que los otros tres ficheros requerían cambios paralelos, porque los tests no ejercitaban esos caminos. En otro caso, la corrección requería lanzar una nueva excepción en la función que fallaba y capturarla en la función invocante. El agente resolvió la primera parte, pero la segunda quedó sin tratar porque ningún test llegaba a ejercitar ese camino de propagación.

Este comportamiento ha llevado a tomar dos decisiones de diseño complementarias. Por una parte, las instrucciones del agente deben orientarlo hacia la comprensión de la función y de su contrato, no solo hacia satisfacer las aserciones del test. Además, el sistema incorpora una capa de validación semántica que clasifica

los parches como correctos, plausibles o sobreajustados, más allá del resultado binario de los tests. En la evaluación experimental, esta clasificación separa los parches que pasan tests de los que además reparan la causa raíz del fallo.

Control de iteraciones, herramientas y restricciones. Sin restricciones, un agente puede iterar indefinidamente, llamar a la misma herramienta repetidas veces con los mismos argumentos, o explorar código que ya ha leído sin producir ningún avance. Estas restricciones de comportamiento, a menudo denominadas *guardrails*, son los mecanismos que limitan esas situaciones. En un contexto experimental, además, un agente sin control aumenta el coste por ejecución y hace los resultados difícilmente comparables entre arquitecturas y modelos, ya que el consumo de tokens depende tanto de la capacidad del modelo como de la forma en que administra su contexto.

Por ello, el sistema opera con un límite máximo de iteraciones configurable. Cuando en iteraciones consecutivas los tests siguen fallando sin que se hayan producido cambios en el repositorio, el sistema detecta la ausencia de progreso y puede terminar anticipadamente. Con este control, las ejecuciones mantienen un presupuesto comparable incluso cuando un agente insiste en explorar sin editar. Además, cada iteración recibe el resultado de los tests de la anterior, de modo que el agente dispone de información actualizada sin reconstruir desde cero lo que ya se ha intentado.

El conjunto de herramientas disponibles también está deliberadamente acotado. El agente opera sobre el repositorio mediante operaciones locales (lectura y escritura de ficheros, ejecución de tests y utilidades de control de versiones), sin acceso a búsqueda en internet durante la ejecución. Esto garantiza que los resultados no dependan de recursos externos y que cada ejecución sea reproducible.

Cuando la tarea se reparte entre varios agentes, estos problemas no desaparecen. En algunos casos se hacen más visibles. Cada subagente trabaja con su propio contexto y no siempre conoce todo lo que han hecho los demás, de modo que el coordinador tiene que decidir qué información conserva y cómo la resume. Además, coordinar agentes también consume tokens. Esta tensión forma parte de la comparación experimental entre arquitecturas.

4.3 Requisitos del sistema

Los retos anteriores se traducen en tres necesidades del sistema:

- permitir que el agente lea y modifique el repositorio mediante operaciones controladas;
- distinguir entre pasar tests y reparar la causa raíz del fallo;
- ejecutar cada intento con límites claros, resultados auditables y configuración reproducible.

Requisitos funcionales

RF-1. Aislamiento de ejecución. Cada ejecución se realiza en un contenedor temporal e independiente, de modo que el entorno del repositorio objetivo no contamina la máquina anfitriona y los intentos son reproducibles en cualquier máquina con Docker.

RF-2. Interfaz basada en herramientas. El agente aplica los cambios sobre el repositorio mediante operaciones de lectura, escritura y reemplazo de líneas. El parche final procede del estado real del repositorio, con cambios aplicados y verificables.

RF-3. Soporte multiarchivo. El agente puede modificar múltiples archivos en la misma ejecución, sin restricciones sobre cuántos ficheros puede tocar por iteración. Limitar las ediciones a un único fichero introduciría un sesgo sistemático que llevaría al agente a producir parches parciales incluso cuando la causa raíz requiere cambios propagados a varios módulos.

RF-4. Bucle iterativo con resultado de tests. El resultado de los tests de cada iteración se incorpora como entrada de la siguiente, de modo que el agente dispone de información actualizada sobre qué fallos permanecen sin necesidad de repetir lo que ya se ha intentado.

RF-5. Validación automática. Antes de aceptar un resultado como exitoso, el sistema verifica que los tests relevantes pasan sobre el repositorio modificado. Esta validación actúa como primer filtro de corrección antes de la revisión semántica posterior.

RF-6. Generación de parche final. Al terminar la ejecución, el sistema genera un *unified diff* desde el estado real del repositorio, tanto si el resultado es éxito como si es un fallo parcial.

Requisitos no funcionales

RNF-1. Trazabilidad. Cada ejecución registra las llamadas a herramientas, el número de iteraciones, el consumo de tokens y la duración total. Estos datos constituyen la base de la comparación entre arquitecturas y modelos en la evaluación experimental.

RNF-2. Modularidad. Las arquitecturas de agentes son intercambiables sobre la misma base experimental. Cambiar de mono-agente a orquestador no requiere modificar la infraestructura de ejecución, los datasets ni el sistema de registro de resultados. Esta separación es una condición necesaria para que la arquitectura actúe como variable independiente del experimento y los resultados de distintas configuraciones sean comparables entre sí.

RNF-3. Reproducibilidad. La configuración completa de cada ejecución, incluyendo modelo, arquitectura y versión de prompts, queda versionada junto a los resultados, de modo que cualquier intento puede auditarse o reproducirse con posterioridad.

RNF-4. Control de coste. El número máximo de iteraciones y el tiempo máximo de ejecución son parámetros configurables, lo que permite ajustar el presupuesto según el modelo o el benchmark sin modificar el código del sistema.

5 Diseño e implementación

Tras definir el problema y los requisitos, el siguiente paso fue diseñar una plataforma que permitiera comparar arquitecturas de agentes bajo las mismas condiciones. Para que esa comparación fuera válida, la preparación del caso, la validación de los parches y el registro de resultados debían permanecer constantes entre variantes. Solo así una diferencia de rendimiento puede atribuirse a la arquitectura o al modelo, y no a variaciones en el entorno.

La solución fue separar la parte común del flujo de los factores que cambian entre experimentos. Esa separación es el hilo conductor que organiza todo el capítulo, desde la preparación de los casos hasta el registro de los resultados.

En primer lugar, se presenta la visión arquitectónica general y la separación entre partes comunes y factores experimentales del sistema (ver Sección 5.1). Posteriormente, se describe el entorno de ejecución aislado, necesario para mantener la reproducibilidad en benchmarks heterogéneos (ver Sección 5.2). A continuación, se detalla el conjunto de herramientas APR, que es la interfaz mediante la que el agente observa y modifica el repositorio (ver Sección 5.3). También se explica el bucle iterativo, incluyendo cómo se construye el contexto entre iteraciones y qué criterios de parada se aplican (ver Sección 5.4). Después, se analizan las arquitecturas implementadas y sus compromisos de diseño (ver Sección 5.5). Por último, se presenta el sistema de observabilidad, que permite auditar cada ejecución y comparar resultados con evidencia (ver Sección 5.6).

5.1 Visión arquitectónica general

La plataforma se diseñó separando la parte común de la ejecución de los factores que cambian entre experimentos. La parte común agrupa la preparación del caso, el bucle de iteraciones, la validación de tests y la persistencia de artefactos. En cambio, la parte variable queda limitada a los factores de estudio, es decir, la arquitectura de agentes, el modelo y la selección de bugs. Gracias a esta separación, una diferencia de resultados puede analizarse sin mezclarla con cambios accidentales en el entorno o en la forma de validar los parches.

En la práctica, cada ejecución se compone de dos grupos de elementos:

- **Elementos fijos.** Preparación del caso, entorno de ejecución, herramientas APR, validación de tests, generación del parche final y registro de artefactos.
- **Elementos variables.** Arquitectura que toma las decisiones, modelo LLM usado y bug preparado que recibe el sistema como entrada.

Sin esa separación, una diferencia de resultados podría proceder de la coordinación entre agentes o de cualquier variación accidental en la preparación, la validación o el registro. La distinción entre elementos fijos y variables es lo que convierte el experimento en una comparación controlada. La Figura 6 resume esta estructura.

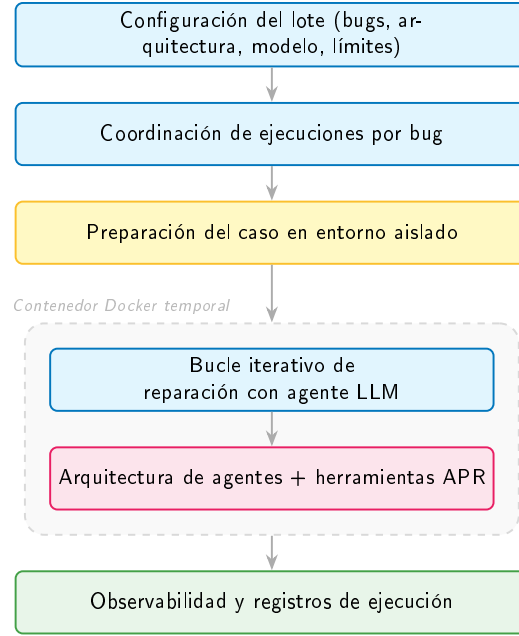


Fig. 6. Arquitectura general de la plataforma APR. La parte común prepara cada caso, ejecuta el bucle de reparación dentro de un contenedor temporal y registra la información necesaria para comparar arquitecturas.

Registrar información durante todo el flujo evita que el análisis dependa únicamente del resultado final. Dos arquitecturas pueden terminar en fallo, pero una puede haber localizado el problema y otra puede haberse quedado explorando ficheros irrelevantes. Sin esa diferencia, ambos casos quedarían mezclados en la misma categoría.

5.2 Entorno de ejecución aislado

Cada ejecución se realiza en un contenedor temporal que arranca desde una imagen limpia y se destruye al finalizar. Sin ese aislamiento, dos ejecuciones del mismo bug podrían depender del estado previo del host, lo que comprometería la comparación de arquitecturas. El ciclo sigue siempre el mismo orden: preparar el entorno, preparar el caso, ejecutar el bucle de reparación, guardar los resultados y destruir el contenedor.

Ejecución con Docker y contenedores temporales. El ciclo operativo de una ejecución se puede resumir en cuatro pasos:

1. Preparar el espacio de trabajo del bug dentro del contenedor,
2. Ejecutar iteraciones del agente y validación de tests,
3. Persistir artefactos (diff, trazas, métricas, salidas),
4. Destruir el contenedor.

En BugsInPy, parte de los fallos potenciales no proceden del código que debe reparar el agente, sino de desajustes de intérprete Python. Por ello, el sistema separa explícitamente el entorno del benchmark y el entorno de la plataforma dentro del contenedor, en vez de confiar en la configuración del host. Sin esa separación, varios falsos negativos habrían distorsionado la comparación de arquitecturas.

Integración con datasets. QuixBugs y BugsInPy persiguen el mismo objetivo, pero no se preparan igual. QuixBugs suele requerir una preparación más directa. BugsInPy, en cambio, fuerza a trabajar con repositorios reales, revisiones concretas y entornos más sensibles.

Si el motor de iteraciones conociera esos detalles internamente, el diseño quedaría acoplado a cada benchmark. Para evitarlo, el sistema normaliza cada entrada a un caso preparado con un contrato común formado por identificador de bug, comando de test y espacio de trabajo listo para ejecutar.

Así, cuando se corrige un problema de preparación en BugsInPy, no se toca la lógica del bucle iterativo. Del mismo modo, cuando se refina la política de iteraciones, no se reescribe la capa de adaptación del dataset. Este desacoplamiento permite evolucionar cada parte sin degradar la estabilidad del flujo completo.

5.3 Herramientas del agente

El comportamiento del agente no depende solo de las instrucciones del prompt, ya que las herramientas disponibles condicionan tanto qué puede hacer como cómo lo hace. Sin límites de tamaño, una herramienta de lectura puede saturar el contexto con ruido. Sin restricciones de alcance, una de edición puede producir cambios difíciles de auditar. Por ello, el conjunto de herramientas APR se concibió como una interfaz controlada entre el modelo y el repositorio, de modo que cada acción queda dentro de operaciones observables y reversibles.

Table 1. Herramientas APR y función de cada grupo.

Grupo	Acciones típicas	Para qué se usa en APR
Exploración	inspección del workspace, listados, lecturas y búsquedas	del localizar código relevante a partir del traceback y reducir exploración ciega
Edición	reemplazos localizados y reescrituras y aplicación de diffs	aplicados, evitando sobrescrituras destructivas
Validación	ejecutar tests y comandos auxiliares	medir efecto real del parche y detectar regresiones pronto
Control de cambios	inspeccionar estado diff del repositorio	y entender qué cambio introdujo el agente y evitar iteraciones repetitivas

En el perfil completo, el toolkit expone doce tools directas. Se agrupan del siguiente modo:

- exploración: `get_workspace_info`, `list_files`, `read_file` y `search_files`,
- edición: `write_file`, `replace_in_file`, `replace_lines` y `apply_unified_diff`,
- ejecución: `execute_command` y `run_test_target`,
- control de cambios: `git_status_summary` y `git_diff_summary`.

No todas las arquitecturas reciben las doce. El sistema define perfiles más cortos según el rol de cada agente y, en el caso del orquestador, añade además dos tools delegadas, `explore_code` y `run_tests`.

Herramientas de exploración. El primer grupo de herramientas permite al agente orientarse dentro del repositorio. En la práctica, el toolkit de exploración se reduce a cuatro operaciones. Corresponden a `get_workspace_info`, `list_files`, `read_file` y `search_files`, con las que el agente puede inspeccionar la raíz de trabajo, listar rutas, leer rangos concretos de un fichero y buscar fragmentos de texto o símbolos. Esta decisión no es accidental. En APR, casi todas las necesidades de localización caben en esa combinación, mientras que abrir un terminal libre para explorar desde el principio multiplica el ruido y hace más difícil auditar qué evidencia vio realmente el modelo.

La exploración queda además acotada por límites comunes a todas las arquitecturas. Por defecto, una lectura no devuelve más de 12 000 caracteres, un listado no supera 400 rutas, una búsqueda textual no devuelve más de 50 coincidencias y no se leen ficheros mayores de 512 KB. Estos valores son parámetros del harness, no decisiones que cambien con cada modelo o con cada repositorio durante la comparación. Su papel es mantener utilizable la ventana de contexto, de modo que el agente pueda seguir el *traceback* hacia el código afectado sin volcar directorios completos ni salidas muy largas. Se fijaron durante los lotes preliminares como umbrales estables: eran lo bastante amplios para no bloquear la localización normal del fallo y, al mismo tiempo, evitaban que una sola lectura o búsqueda consumiera una parte desproporcionada del contexto disponible.

Estas herramientas cumplen tres funciones principales:

- localizar el código mencionado por el fallo inicial,
- ampliar el contexto cuando el error depende de funciones o módulos relacionados,
- evitar lecturas masivas que consuman tokens sin aportar información nueva.

Esta última función se controla con reglas explícitas. Mientras el agente no haya editado nada, el sistema solo permite ocho búsquedas textuales por iteración y bloquea la repetición exacta de una consulta ya lanzada. En otras palabras, la plataforma considera que la exploración deja de aportar información nueva cuando el agente vuelve a pedir la misma búsqueda o consume todo el presupuesto de búsquedas sin haber transformado esa evidencia en una edición concreta.

Herramientas de edición. Las herramientas de edición se reparten entre `write_file`, `replace_in_file`, `replace_lines` y `apply_unified_diff`. En la práctica, las tres primeras cubren la mayor parte de los parches, mientras que la cuarta permite aplicar un diff unificado cuando el agente ya ha razonado el cambio en ese formato. El objetivo no es ofrecer al modelo toda la libertad posible, sino obligarlo a anclar cada cambio sobre evidencia concreta del código ya leído. Sin ese anclaje, resulta muy fácil que una edición generada por el modelo sobrescriba más código del necesario o pierda partes del archivo al reemitirlo completo.

Por eso, estas operaciones incorporan controles específicos frente a los problemas que aparecen con más facilidad en APR agéntico:

- sobrescribir más código del necesario,
- modificar la batería de tests para hacer pasar el caso.

La plataforma aplica dos reglas concretas. En primer lugar, rechaza cualquier ruta que parezca un archivo de tests antes de escribir sobre ella. Además, cuando la edición implica reescribir un fichero completo, comprueba si el archivo existente tiene más de 50 líneas y si la nueva versión recorta más de un tercio de su tamaño. Si se cumplen ambas condiciones, la operación se bloquea y se obliga al agente a volver a un reemplazo localizado. A esto se suma un control adicional sobre los reemplazos: si el anclaje no aparece el número de veces esperado, la edición se rechaza para evitar que un parche pensado para una zona termine aplicándose sobre otra. Cabe destacar que, al igual que los umbrales de exploración, estos límites se fijaron durante los lotes preliminares y son configurables; si se detecta que restringen en exceso la capacidad de edición del agente, pueden relajarse sin modificar la lógica del bucle iterativo.

Por esta razón, la edición no se trata como generación libre de texto. Cada cambio queda asociado a una llamada de herramienta y después se contrasta con el estado real del repositorio. Esto permite saber qué fichero se tocó, qué tamaño tuvo el parche y si el agente intentó actuar sobre archivos que no debía modificar.

Herramientas de ejecución y validación. El tercer grupo ejecuta comandos dentro del entorno aislado. Aquí intervienen dos tools con papeles distintos. `run_test_target` se usa para la batería de tests y `execute_command` para los comandos auxiliares. Se reservó una herramienta específica para los tests porque validar un parche no es lo mismo que lanzar un comando auxiliar. La diferencia se refleja incluso en el presupuesto temporal. El runner de tests tiene un límite por defecto de 120 segundos, mientras que los comandos

auxiliares se ejecutan con 30. La salida de tests se compacta antes de volver al agente, ya que una traza completa puede ocupar mucho contexto y desplazar información más útil para la siguiente iteración.

La compactación no es solo una cuestión de formato. Antes de devolver la salida al modelo, la plataforma elimina cabeceras de pytest, advertencias repetitivas de Python y bloques duplicados, y recorta el resultado si supera el presupuesto de caracteres fijado para comandos. Así el agente ve el síntoma relevante del fallo sin arrastrar páginas enteras de ruido.

Además, la validación se controla desde la plataforma. El agente puede solicitar que se ejecuten los tests, pero la decisión de aceptar el resultado, rechazar un cambio prohibido o detener la ejecución se toma con reglas externas al modelo. Antes de dar por válida una continuación, cada iteración comprueba tres cosas: que los cambios observados en disco tengan un diff consistente, que no se hayan tocado ficheros de tests y que el resultado nuevo no sea una regresión respecto a la línea base. Si falla una regla recuperable, como una edición accidental sobre tests, el sistema revierte el repositorio y concede un único reintento con una explicación explícita. Esta separación evita que una respuesta convincente del agente sustituya a la evidencia producida por el repositorio modificado.

Herramientas de diagnóstico de cambios. El diagnóstico de cambios conecta la acción del agente con el estado observable del repositorio. Aquí entran `git_status_summary` y `git_diff_summary`, que resumen el estado y el parche vigente sin devolver toda la salida cruda de Git. No basta con que el modelo diga que ha aplicado un parche, ya que una llamada de herramienta puede fallar o no encontrar el fragmento esperado. Por eso, al final de cada iteración se consulta el estado de versiones y se obtiene una vista resumida del diff. Antes y después de la iteración, la plataforma toma una instantánea del repositorio. Si esa comparación detecta archivos rastreados modificados, entonces exige que `git diff` muestre precisamente esos cambios. En este contexto, que el diff sea coherente significa que el parche textual refleje las modificaciones realmente observadas en disco. Si la instantánea cambia pero el diff queda vacío, la iteración falla por un problema de integridad y no se acepta como evidencia válida.

Esta información se usa con dos propósitos. Primero, permite comprobar que realmente hubo una edición. Segundo, ayuda a detectar iteraciones repetitivas, por ejemplo cuando el agente vuelve a proponer el mismo cambio o cuando modifica una zona que no afecta al fallo observado.

Herramientas delegadas por sub-agentes. En la arquitectura orquestador aparecen dos herramientas adicionales, `explore_code` y `run_tests`, implementadas mediante el mecanismo de `Agent as_tool()`. No editan código, sino que devuelven resúmenes accionables al agente principal. La diferencia respecto a las herramientas directas es que la llamada encapsula trabajo de otro agente, de modo que el orquestador recibe una síntesis en vez de todo el historial interno. Esta compresión resulta útil cuando una salida extensa de pytest o una exploración larga añadiría más ruido que información útil.

5.4 Bucle iterativo de reparación

En cada iteración, el agente recibe el estado actual del espacio de trabajo, actúa con sus herramientas y obtiene el resultado de los tests antes de que comience la siguiente vuelta. Esta estructura permite corregir la dirección a mitad de reparación. Sin ese bucle, el sistema dependería de que la primera hipótesis del modelo fuese correcta, algo poco realista en bugs donde el síntoma visible no coincide exactamente con la causa raíz.

El presupuesto de iteraciones es un parámetro del caso preparado, común a la arquitectura que lo ejecute. Si no se especifica otra cosa, el runtime usa tres iteraciones y solo admite valores entre 1 y 30. Por tanto, el límite no depende del modelo ni cambia dinámicamente según el repositorio, aunque cada lote experimental puede fijarlo de antemano en su configuración.

Fases de una iteración. El flujo operativo se articula en cinco fases, resumidas en la Figura 7 y el Pseudocódigo 9:

1. **Preparación.** Se registra la identidad de iteración, la instantánea del espacio de trabajo y el reinicio de contadores de edición y llamadas a herramientas.
2. **Ejecución del agente.** Se construye la entrada de iteración y se invoca al agente con su perfil de herramientas.
3. **Observación.** Se inspeccionan los cambios reales, se guarda el parche de iteración y se ejecutan los tests.
4. **Validación y decisión.** Se aplican las comprobaciones de integridad y la política de parada.
5. **Aplicación de la decisión.** El sistema continúa, reintenta tras restaurar el repositorio o detiene la ejecución.

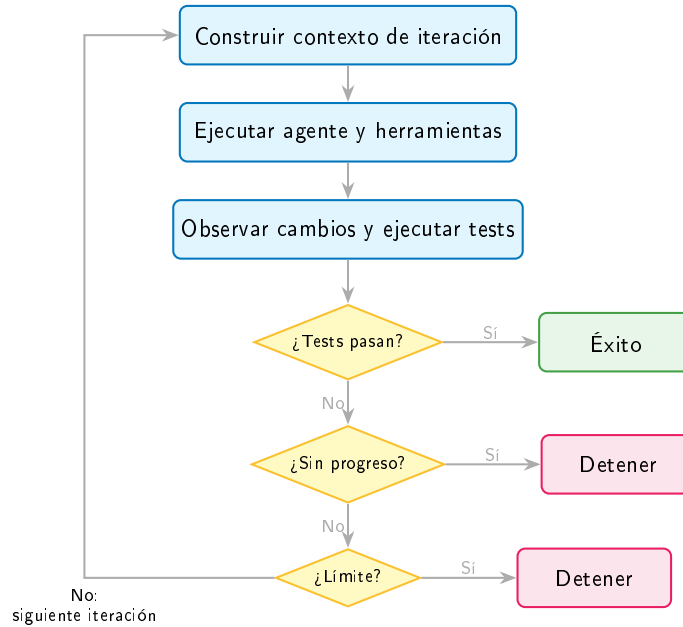


Fig. 7. Lógica de control del bucle iterativo. Tras ejecutar el agente y observar los tests, el sistema termina si el parche pasa, si no hay progreso o si se alcanza el límite configurado. Solo cuando ninguna condición de parada se cumple se construye una nueva entrada de iteración.

Separar las causas de parada es importante para el análisis posterior. Un fallo por límite de iteraciones indica que el agente siguió proponiendo acciones hasta agotar su presupuesto, mientras que una parada por falta de progreso suele señalar que el sistema ya no observa cambios útiles entre intentos. Ambos terminan sin reparación aceptada, pero no describen el mismo comportamiento.

Propagación de contexto entre iteraciones. El contexto de una iteración no se limita a reenviar todo el historial conversacional del agente. El sistema construye una nueva entrada a partir de lo que se ha observado en el repositorio, de modo que la siguiente vuelta no dependa solo de lo que el modelo afirma haber hecho. Esta decisión importa porque una respuesta puede describir una edición que no ha llegado a aplicarse en disco, por ejemplo si una herramienta falla o si el agente agota sus turnos antes de modificar código.

La entrada de continuación combina dos tipos de información, como muestra la Figura 8:

- **Información declarada por el agente.** Incluye explicación de la hipótesis, cambios intentados y notas que el propio agente deja para la siguiente vuelta.
- **Información observada por la plataforma.** Incluye resultado compacto del test, ficheros modificados, vista parcial del diff y avisos generados durante la iteración.

En la primera iteración, el sistema parte del fallo inicial y añade una vista compacta del repositorio junto con la función fuente localizada a partir del *traceback*. En las iteraciones posteriores, en cambio, la entrada se concentra en lo que ha cambiado: resultado de tests, diff y avisos de la plataforma. Si la vuelta anterior no produjo ninguna edición real, se activa un mecanismo de recuperación que vuelve a localizar la función afectada y obliga a replantear el siguiente intento desde evidencia observada, no solo desde la conversación previa.

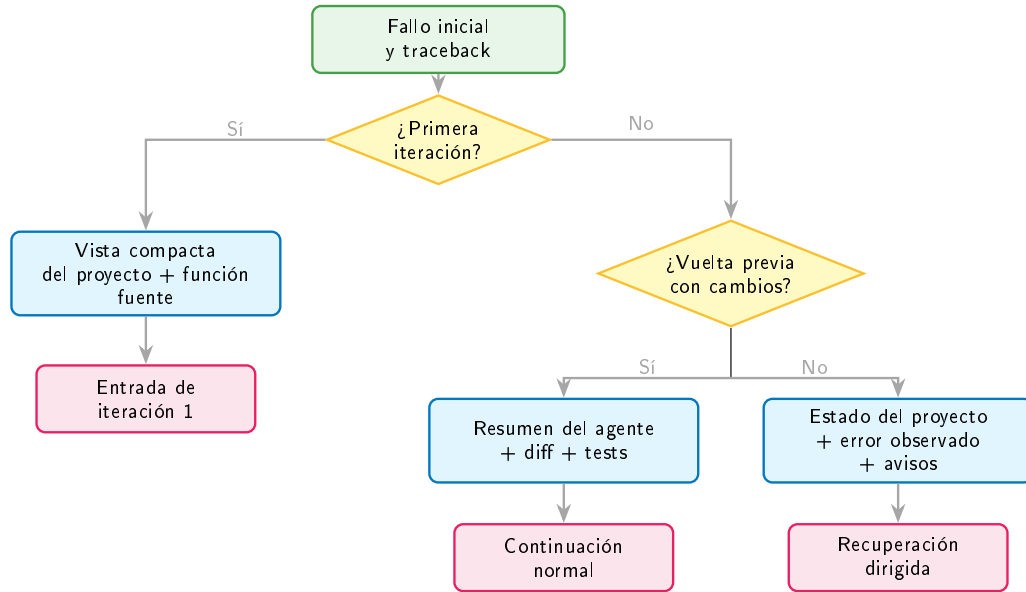


Fig. 8. Lógica de selección de contexto por iteración. La primera vuelta recibe una vista compacta del repositorio y la función fuente localizada. Después, el sistema decide entre una continuación normal basada en cambios observados o un mecanismo de recuperación cuando la iteración previa no produjo una edición real.

Este mecanismo evita que una iteración sin cambios se trate como un intento equivalente a los demás. Si no hubo edición, la siguiente vuelta recibe una indicación explícita de que debe cambiar de estrategia. Si hubo edición pero el fallo persiste, la continuación se centra en el efecto real del parche. El capítulo siguiente analiza con más detalle cómo se refinó este paso durante el desarrollo, especialmente el orden entre código fuente, salida de tests e información acumulada.

Validación, reintentos y criterios de parada. La decisión de continuar o detener no depende del agente, sino de políticas aplicadas sobre el estado observable del repositorio. Al finalizar cada iteración se comprueban tres condiciones:

- que el diff generado sea coherente con los cambios aplicados;
- que no se hayan modificado ficheros de test;
- que el parche no introduzca una regresión respecto a la línea base.

Cuando alguna comprobación falla por una causa subsanable, como un cambio accidental en los tests, el sistema restaura el espacio de trabajo y concede un reintento con una indicación explícita del problema. Si la incidencia no puede resolverse dentro del flujo normal, la ejecución termina. La falta de progreso tampoco se decide a partir de una sola señal. Para estimarla, la plataforma revisa al cierre de cada iteración si ha habido cambios reales en los archivos fuente, si la firma compacta del resultado de tests coincide con la iteración anterior, si el agente repite la misma propuesta textual y si la confianza declarada mejora o no.

Con esa información, la ejecución puede detenerse por falta de progreso en dos situaciones. La más clara aparece cuando no hay cambios en disco, el fallo observado sigue siendo el mismo y el propio agente se declara atascado. En otros casos, el sistema exige una coincidencia más fuerte: tampoco hay cambios, se repite la firma de tests, la propuesta no varía y la confianza no mejora. Así se evita tanto cortar demasiado pronto una ejecución que aún podría reconducirse como alargar ciclos que, en la práctica, ya no están moviendo el estado del problema.

La configuración de la ejecución (modelo, arquitectura, límites) es inmutable durante la ejecución, mientras que el estado (instantánea, resultados previos, tokens acumulados, información de validación) evoluciona en cada iteración. Cuando una ejecución falla de modo inesperado, esa separación acota la causa.

Pseudocódigo 1. Bucle iterativo de reparación a alto nivel

Entrada: caso preparado, configuración de la ejecución

Salida: resultado final y parche generado

1. Inicializar estado de la ejecución (línea base, resultados previos, contadores).
2. Para cada iteración hasta el límite:
 - 2.1 Construir entrada con evidencia observada y contexto acumulado.
 - 2.2 Ejecutar agente con su perfil de herramientas.
 - 2.3 Inspeccionar cambios y ejecutar tests.
 - 2.4 Validar integridad, regresión y cambios prohibidos.
 - 2.5 Decidir si se continúa, se reintenta con restauración o se detiene.
3. Emitir resultado final y artefactos de la ejecución.

Fig. 9. Pseudocódigo conceptual del bucle iterativo.

5.5 Arquitecturas de agentes implementadas

La plataforma implementa cuatro arquitecturas sobre el mismo flujo de iteración. Todas reciben el mismo tipo de caso preparado, usan el mismo sistema de herramientas y generan los mismos artefactos, pero cambian la forma en que se reparte la responsabilidad entre agentes. La comparación se centra así en la coordinación, mientras que el entorno de ejecución y la validación permanecen constantes. Las instrucciones de sistema completas de cada arquitectura se recogen en el Apéndice B.

No todas las variantes tienen el mismo papel en la memoria. Mono-agente, la variante de planificación y ejecución, y el orquestador forman parte de la evaluación formal del capítulo experimental. La arquitectura con *handoff*, en cambio, se conserva en esta sección porque fue implementada y ayuda a entender el espacio de diseño, pero se descarta antes de la comparación final por la inestabilidad observada en las pruebas preliminares.

La Tabla 2 resume la diferencia principal entre las arquitecturas. La clave no es únicamente cuántos agentes intervienen, sino quién conserva el control del flujo, qué información se transfiere entre fases y qué herramientas puede usar cada rol.

Mono-agente. La arquitectura mono-agente actúa como línea base del sistema. Un único agente recibe el contexto de iteración, decide qué leer, aplica el parche y valida el resultado. Al no existir transiciones entre roles, no se pierde información por trasposos intermedios y tampoco aparece sobre coste de coordinación. La Figura 10 muestra este flujo.

Esa sencillez tiene una contrapartida clara. El mismo agente debe decidir cuándo dejar de explorar y empezar a editar, por lo que el diseño depende mucho de que el conjunto de herramientas oriente bien el proceso. En esta variante se le entrega el perfil completo de herramientas, pero el flujo exige tres hábitos básicos:

- leer antes de editar;

Table 2. Resumen de las arquitecturas implementadas.

Arquitectura	Papel en la comparación
Mono-agente	Línea base simple. Un único agente conserva todo el control y accede al conjunto completo de herramientas.
Handoff	Variante exploratoria. El control se transfiere entre triage, localización, parcheo y validación.
Planner-Executor	Dos fases. La primera investiga y planifica sin editar, mientras que la segunda aplica y valida el parche.
Orquestador	Control central con delegación. El agente principal invoca sub-agentes como herramientas especializadas.

- validar solo después de modificar código fuente;
- revisar el diff antes de cerrar la iteración.

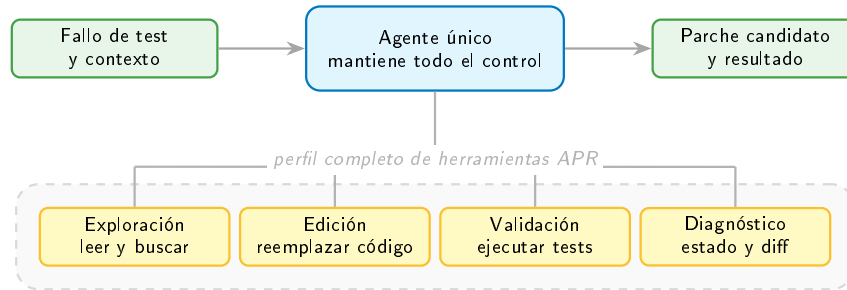


Fig. 10. Flujo de la arquitectura mono-agente. Un único agente conserva el estado de la reparación y decide cuándo explorar, editar, validar o revisar el diff, por lo que no hay coste de coordinación entre roles.

Multi-agente con handoff. La arquitectura con *handoff* separa el proceso en una cadena de cuatro roles, representados en la Figura 11. El agente de triage recoge la información inicial, el localizador acota los ficheros y símbolos sospechosos, el agente de parcheo aplica los cambios y el validador ejecuta las comprobaciones finales. Cada rol dispone de un perfil de herramientas distinto, de modo que la separación no es solo una diferencia de instrucciones, sino también de capacidad de actuación.

El traspaso entre roles se realiza mediante un mensaje estructurado. En él se incluyen el resumen del diagnóstico, las evidencias usadas, los ficheros sospechosos, el siguiente foco de trabajo y la confianza de la hipótesis. Esta estructura reduce la ambigüedad, aunque también hace que el flujo dependa de que cada agente entregue el control a tiempo. Si el primer rol consume demasiados turnos explorando, los siguientes llegan tarde y apenas tienen margen para editar o validar.

En las pruebas preliminares apareció ese problema. Este tipo de reparación obliga a alternar exploración, edición y validación con mucha frecuencia. El *handoff*, en cambio, plantea una transición más rígida entre fases. Con modelos locales, los primeros agentes tendían a agotar el presupuesto antes de transferir el control, de modo que el resultado observado no reflejaba una colaboración completa entre roles.

Las pruebas exploratorias sobre QuixBugs confirmaron ese comportamiento. La ejecución sobre el conjunto completo no produjo ninguna reparación aceptada, y la muestra reducida dio un único éxito que no demostraba una cadena de traspasos estable. El resultado no apuntaba a un problema de ajuste del prompt, sino a una incompatibilidad estructural entre el patrón secuencial y la naturaleza iterativa de APR.

La arquitectura con *handoff* queda documentada como variante implementada en la fase de diseño, no como candidata en la evaluación formal. Su aportación es de tipo negativo, pues especializar el proceso en

una cadena de roles no funciona cuando la tarea obliga a retroceder con frecuencia entre diagnóstico, edición y validación.

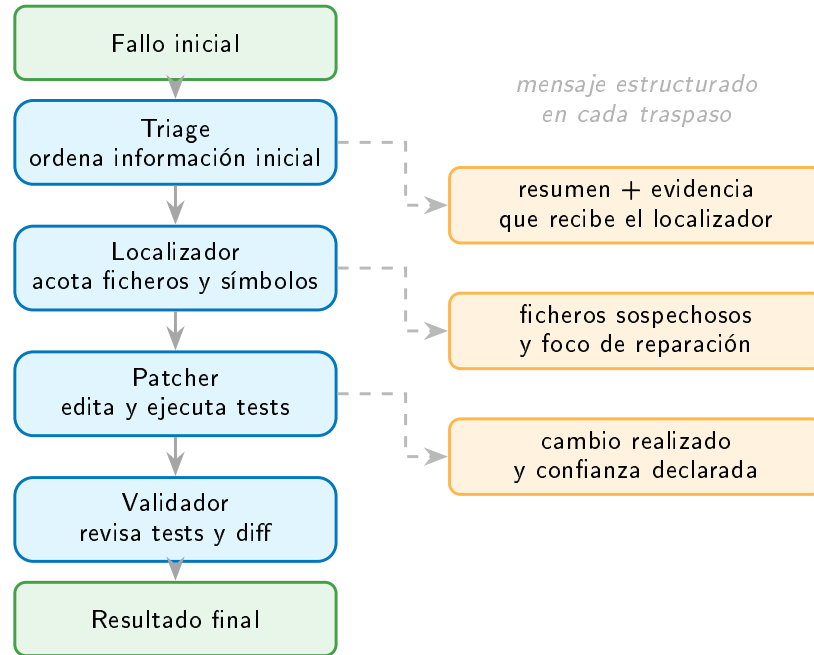


Fig. 11. Flujo de la arquitectura multi-agente con *handoff*. La columna izquierda indica quién posee el control en cada fase, mientras que la derecha resume la información que se entrega al siguiente rol. El patrón hace explícita la especialización por roles, pero también introduce una transición rígida entre fases.

Planner-Executor. Planner-Executor reduce la separación a dos fases, como muestra la Figura 12. En la primera, el planner investiga el fallo, puede leer código y ejecutar pruebas, pero no dispone de herramientas de edición. Su salida es un plan de reparación que queda incorporado al contexto de continuación. En la fase siguiente, el executor recibe ese plan, puede modificar código, ejecutar tests y adaptar la estrategia si la hipótesis inicial no se confirma.

El planner no puede declarar éxito ni estancamiento. Su tarea termina con un diagnóstico y un plan, no con un parche aceptado. Esta frontera evita confundir una explicación convincente con una reparación efectiva, ya que solo el executor puede validar el cambio sobre el repositorio modificado. Además, al existir una única frontera entre roles, se reduce la pérdida de contexto respecto a una cadena de cuatro agentes.

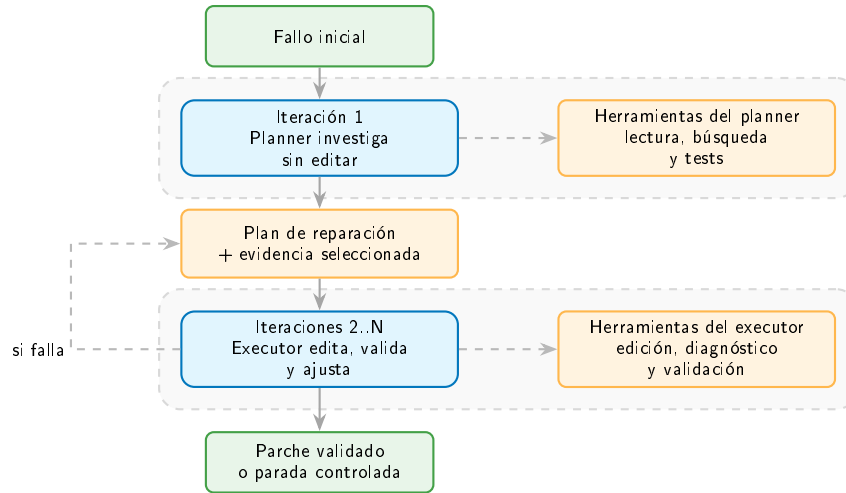


Fig. 12. Separación entre planificación y ejecución en Planner-Executor. El planner ocupa la primera iteración y solo produce diagnóstico y plan, mientras que el executor recibe ese plan en el contexto posterior y es el único rol que puede modificar el repositorio.

Orchestrator con sub-agentes. El orquestador mantiene un agente principal que conserva el control de la reparación y delega capacidades concretas en sub-agentes expuestos como herramientas, según se ilustra en la Figura 13. En lugar de transferir la propiedad completa de una fase, el agente principal decide cuándo invocar exploración profunda o ejecución de tests, recibe un resumen compacto y continúa con la decisión siguiente.

Esta forma de coordinación encaja bien con tareas APR porque muchas decisiones requieren alternar lectura, edición y validación en un orden que no se conoce de antemano. El sub-agente explorador se mantiene en modo solo lectura y sirve para seguir relaciones entre módulos cuando una búsqueda local no basta. El sub-agente de tests ejecuta el comando focalizado y devuelve un veredicto resumido, lo que evita introducir una salida extensa de pytest directamente en el contexto del agente principal.

Esta arquitectura tiene una limitación de observabilidad asociada al patrón del SDK. Cuando un agente se expone como herramienta mediante `as_tool`, las llamadas internas que realiza el sub-agente pueden no aparecer con el mismo nivel de detalle que las herramientas directas del agente principal. Por ello, al analizar esta arquitectura conviene distinguir entre la invocación externa al sub-agente y el trabajo interno que este resume.

5.6 Observabilidad y trazabilidad de resultados

En un experimento con agentes, el resultado final no explica por sí solo lo que ha ocurrido. Dos ejecuciones pueden terminar con el mismo veredicto, pero una puede haber encontrado el parche tras una exploración breve y otra después de consumir todo el presupuesto. Por eso, la observabilidad se diseñó como parte del sistema desde el principio, no como un añadido posterior.

El flujo de observabilidad se organiza en tres pasos, como muestra la Figura 14. Primero se capturan eventos operativos, como llamadas a herramientas, cambios en ficheros y ejecuciones de test. Después, esos eventos se traducen a entidades del dominio, principalmente ejecución, iteración y estado final. Finalmente, la misma información se persiste en tres formatos complementarios:

- una base de datos relacional para calcular métricas agregadas;
- una traza en vivo para revisar manualmente qué ocurrió en una ejecución concreta;
- eventos JSONL para conservar un registro incremental y fácil de procesar con scripts.

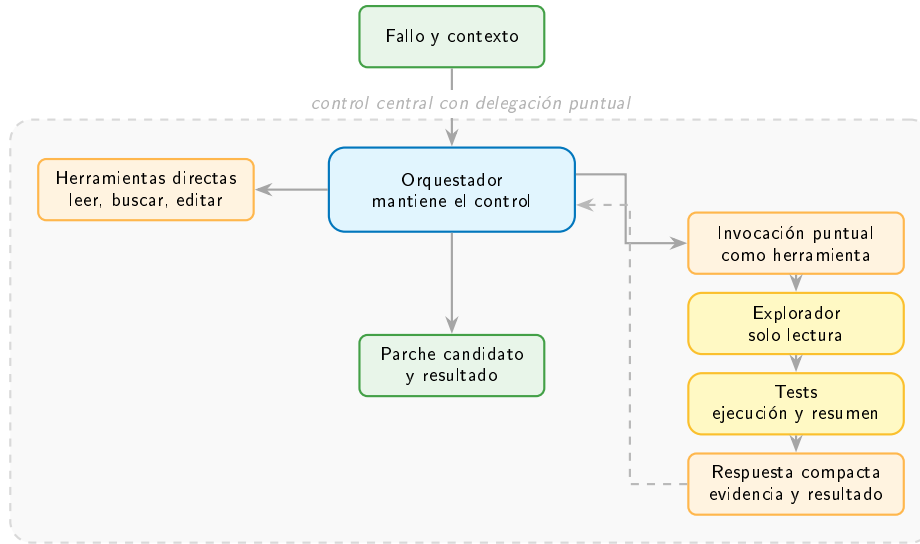


Fig. 13. Arquitectura orquestador con sub-agentes invocados como herramientas. El agente principal conserva la capacidad de editar y decidir, pero puede pedir exploración o validación a sub-agentes que devuelven evidencia resumida en lugar de asumir el control completo de la reparación.

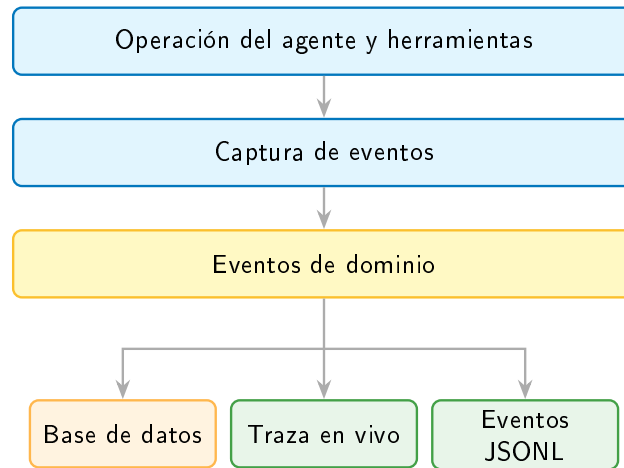


Fig. 14. Capas de observabilidad para ejecuciones e iteraciones. Las acciones del agente se capturan como eventos, se traducen a entidades de dominio y se persisten en formatos complementarios para análisis agregado, auditoría manual y reproducción de trazas.

Separar captura y persistencia permitió ampliar el registro sin cambiar el flujo de reparación. Por ejemplo, cuando se detectó que algunos fallos de infraestructura quedaban invisibles en el análisis agregado, se añadieron eventos específicos de error de ejecución con *traceback*. Sin esa información, varios fallos habrían parecido errores del agente aunque se producían antes de la primera llamada a herramientas.

Para la evaluación, este registro aporta dos capacidades complementarias. Las métricas agregadas por arquitectura, modelo y benchmark permiten comparar variantes a escala. Las trazas por ejecución, en cambio, permiten entender qué ocurrió en un bug concreto, incluyendo fallos de infraestructura que de otro modo se confundirían con errores del agente. Ambas perspectivas son necesarias. Los resultados agregados muestran qué ocurrió en cada comparación, y las trazas individuales explican por qué.

6 Refinamiento del harness iterativo

El capítulo anterior ha descrito la arquitectura final de la plataforma, incluyendo el entorno aislado, las herramientas disponibles, el bucle de reparación y las arquitecturas de agentes comparadas. Sin embargo, esa descripción estática no explica por sí sola cómo se llegó a un sistema suficientemente estable para ejecutar una evaluación experimental. En un sistema APR guiado por tests, esa infraestructura de ejecución iterativa, o *harness*, no es solo una pieza que lanza al agente. También decide qué información recibe en cada vuelta, cómo interpreta los intentos fallidos y qué evidencias conserva antes de permitir otro parche.

Durante el desarrollo se ejecutaron lotes preliminares sobre los benchmarks para observar el comportamiento real de los agentes antes de cerrar la matriz experimental. Esa revisión mostró que muchas diferencias no procedían de la arquitectura nominal, sino de detalles más concretos del proceso. En concreto, importaban el orden de la información en el mensaje de entrada, la continuidad entre iteraciones y la forma de distinguir entre un parche que pasa tests y uno que realmente corrige el fallo. Por ello, este capítulo no introduce una nueva arquitectura, sino que explica las decisiones de refinamiento que hicieron que las arquitecturas del capítulo anterior pudieran compararse bajo un mismo procedimiento.

En primer lugar, se analiza el riesgo de que el agente trate las aserciones del test como si fueran la especificación completa del comportamiento esperado (ver Sección 6.1). Después, se explica cómo se conservó información útil entre iteraciones para evitar que cada intento empezara de nuevo desde cero (ver Sección 6.2). A continuación, se muestra un ejemplo de impacto acumulado de estos refinamientos en un lote preliminar (ver Sección 6.3). Por último, se describe la validación semántica asistida de parches, que se incorporó al proceso al comprobar que pasar la batería de tests no siempre equivalía a reparar el bug real (ver Sección 6.4).

6.1 Sobreajuste a la batería de tests

En APR guiado por tests, la batería de tests actúa como el primer oráculo de verificación. Si el parche propuesto la supera, el sistema puede considerar que existe una reparación plausible. Esta señal es necesaria porque permite automatizar el ciclo de reparación, pero no es suficiente cuando los tests solo cubren una parte del comportamiento del módulo. En esos casos, el agente puede aprender el síntoma que aparece en el test sin entender el contrato completo de la función.

El efecto observado fue el de los parches plausibles pero incorrectos, un problema clásico en APR que ya se había identificado en trabajos anteriores [22]. Un parche de este tipo puede devolver de forma fija el valor esperado por una aserción, corregir solo la rama ejercitada por el test o modificar una condición para satisfacer el caso concreto sin cubrir otros usos del mismo código. En ejecuciones preliminares de BugsInPy también apareció este patrón. En algunos bugs, el agente tocaba el fichero donde fallaba el test, mientras que el parche canónico propagaba el cambio a otros módulos relacionados. La batería de tests aceptaba el cambio, pero la reparación no era equivalente a la que había aplicado el desarrollador.

Una de las causas prácticas estaba en el orden del contexto. Los modelos de lenguaje son sensibles a la posición de los tokens dentro de la entrada [14]. Por tanto, si la salida del test fallido aparece antes que el código fuente, el primer bloque concreto que recibe el modelo son las aserciones y el error observado. Esto empuja el diagnóstico hacia el síntoma. Para reducir ese riesgo, el sistema pasó a extraer la función afectada a partir de la traza de error y a colocarla antes de la salida del test en la primera iteración. Con ello, el primer punto de apoyo pasa a ser el código que define el comportamiento esperado, y no solo el test que lo contradice, de modo que el diagnóstico arranca desde la función afectada y no únicamente desde la aserción fallida. La Figura 15 compara ambos órdenes.

El cambio de orden se acompañó de una regla común para todas las arquitecturas. Antes de ajustar el código a lo que exige el test, el agente debe leer la función afectada y revisar si el parche tiene sentido para sus usos dentro del módulo. Esta regla no se planteó como una instrucción aislada dentro del *prompt*, sino como parte del contrato del harness. El contexto inicial, las herramientas disponibles y las comprobaciones posteriores debían empujar en la misma dirección. De este modo, el test seguía siendo la evidencia que permite validar el intento, pero dejaba de ocupar el papel de especificación completa.

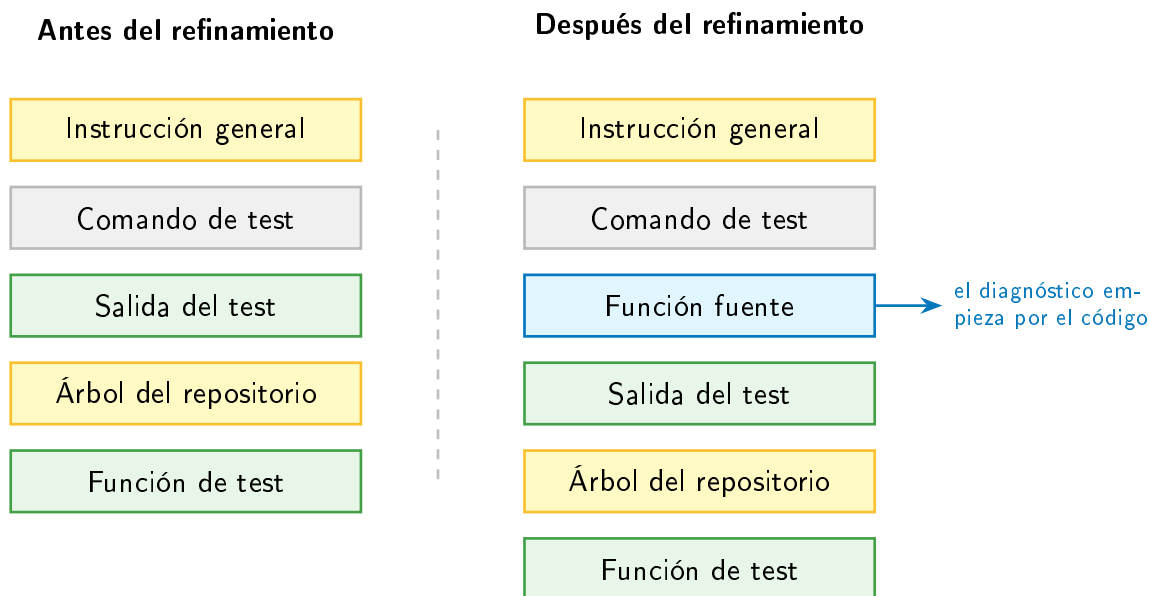


Fig. 15. Orden de los bloques de contexto en la primera iteración. Tras el refinamiento, la función fuente se coloca antes que la salida del test para que el diagnóstico parta del comportamiento del código y no solo de la aserción fallida.

6.2 Continuidad de contexto entre iteraciones

El bucle iterativo de reparación descrito en la Sección 5.4 permite reintentar cuando una propuesta no supera los tests. Sin embargo, el simple hecho de repetir la llamada al agente no garantiza continuidad. Si la segunda iteración recibe casi la misma información que la primera, el agente puede volver a buscar los mismos ficheros, reconstruir la misma hipótesis y agotar parte del presupuesto antes de aplicar un cambio distinto. Esta repetición era especialmente visible cuando una iteración terminaba sin modificar ningún fichero o cuando el error de test se mantenía idéntico tras el parche.

Para evitarlo, se añadió un estado de continuación construido a partir de evidencias observadas por la plataforma. El punto importante es que no se confía solo en lo que el agente declara al final de su respuesta. El harness inspecciona el repositorio y conserva los ficheros modificados, el resultado de la batería de tests, una huella de la traza de error y un extracto del diff. Si no hay cambios en disco, la siguiente iteración recibe esa información de forma explícita. Si la traza de error es la misma que en el intento anterior, el mensaje también indica que el parche no ha afectado al camino que fallaba.

El otro caso relevante aparece cuando una iteración consume sus turnos sin editar. En esa situación, continuar solo con la conversación anterior no basta, porque el agente puede no haber llegado a un punto de partida útil. Por ello, el sistema vuelve a localizar la función fuente a partir de la traza de error más reciente y recupera una vista compacta del repositorio. La siguiente iteración no empieza desde cero, pero tampoco queda atrapada en una exploración anterior que no produjo ningún cambio. La Figura 16 resume esta lógica de selección.

Además, a partir de la segunda iteración se restringe la exploración amplia del repositorio, pues si el fallo ya se ha localizado, repetir búsquedas generales suele consumir turnos que deberían dedicarse a comprobar la hipótesis y editar. La restricción no impide leer código nuevo cuando sea necesario, pero obliga a justificar esa lectura desde la evidencia acumulada. Así, el harness deja de reenviar una conversación cada vez más larga y pasa a construir una entrada de continuación con lo que realmente cambió entre intentos.

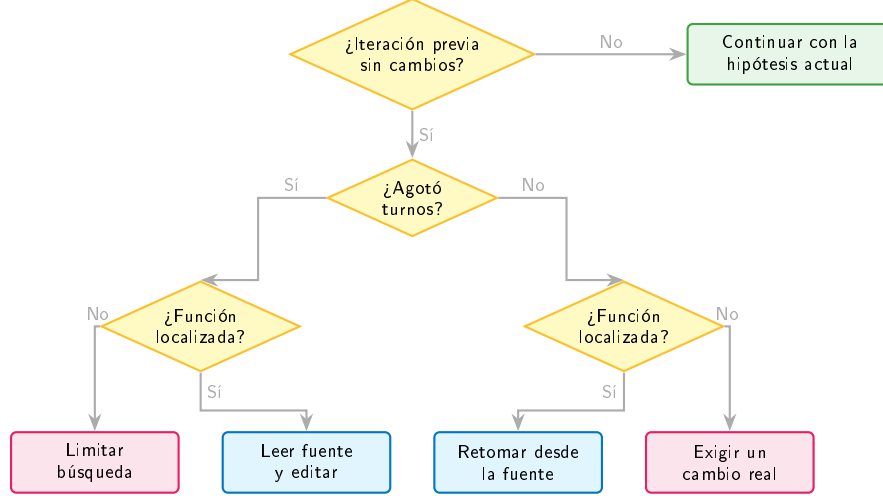


Fig. 16. Selección de la directiva de continuación cuando la iteración anterior no modificó el repositorio. La respuesta depende de si el agente agotó sus turnos y de si el sistema pudo volver a localizar la función afectada.

6.3 Efecto acumulado de los refinamientos

Los refinamientos anteriores son pequeños si se miran de forma aislada. Reordenar bloques de contexto, conservar la huella del error o avisar de que una iteración no produjo cambios no modifica la arquitectura del agente. Aun así, esos detalles condicionan el momento exacto en que el modelo decide explorar, editar o abandonar una hipótesis. En un sistema iterativo, esa diferencia se acumula porque cada vuelta parte de lo que la anterior dejó disponible.

La Tabla 3 recoge un lote preliminar usado durante la selección de modelos en BugsInPy. Su función dentro de la memoria es diagnóstica, ya que muestra por qué se consolidaron las reglas de continuidad y ampliación de contexto antes de la evaluación final. En los tres casos se mantuvo la misma arquitectura mono-agente y el mismo modelo base, mientras que la configuración refinada permitió conservar más información útil entre iteraciones.

Table 3. Efecto observado al ampliar y conservar mejor el contexto en un lote preliminar de BugsInPy. Las columnas numéricas muestran el valor inicial y el refinado separados por una flecha.

Bug	Tests ini→ref	Iter.	Tiempo (s)	Tokens ini→ref	Lectura
ansible-1	No→Sí	3→2	683→290	1,76 M→999 K	Evita una tercera vuelta sin avance
ansible-2	Sí→Sí	2→1	236→73	645 K→193 K	Llega antes al parche
scrapy-33	Sí→Sí	2→1	297→123	824 K→328 K	Reduce exploración repetida

La lectura no es que una ventana de contexto mayor resuelva por sí sola los bugs difíciles. De hecho, el capítulo de evaluación mostrará que BugsInPy sigue siendo el benchmark más exigente. Lo que sí se observa es que la arquitectura no puede analizarse separada del harness. La arquitectura define quién toma las decisiones, mientras que el harness define con qué información se toman y qué ocurre cuando una decisión falla.

Table 4. Capas de validación usadas para clasificar los resultados.

Capa	Entrada	Decisión que aporta
Tests	Repositorio modificado y batería de tests	Determina si el parche es plausible en el sentido clásico de APR.
Referencia	Parche canónico del benchmark	Aporta la reparación conocida contra la que se compara el cambio generado.
Juez LLM	Parche generado, tests y solución canónica	Clasifica el parche plausible como CORRECT , PLAUSIBLE u OVERFITTING .

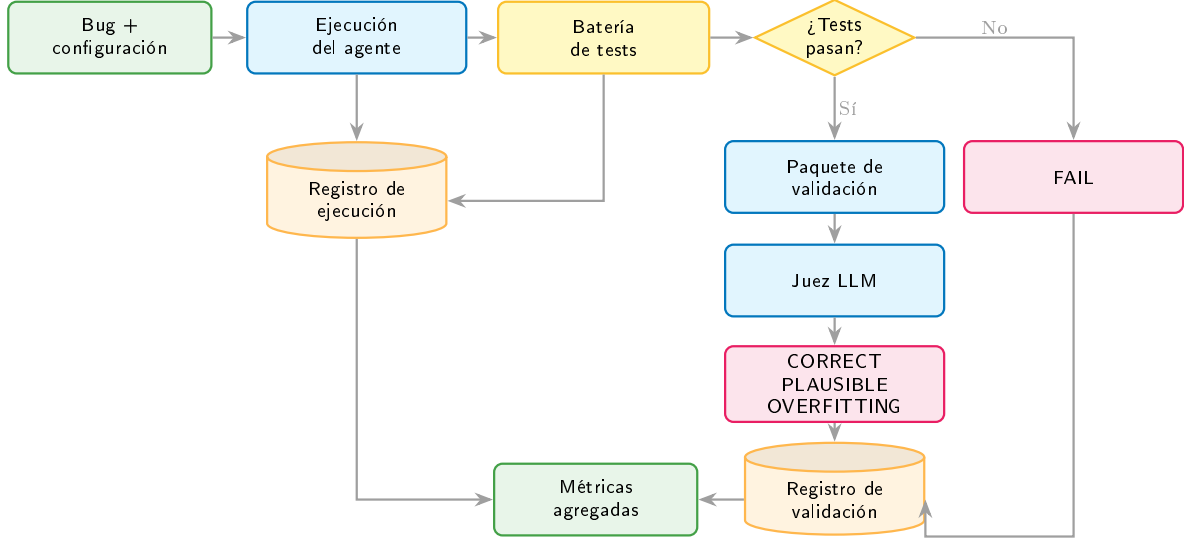


Fig. 17. Flujo de ejecución, validación semántica y registro de resultados. El juez LLM solo se invoca para parches que ya han superado la batería de tests.

6.4 Validación semántica asistida de parches

El refinamiento del harness también afectó a la forma de interpretar los resultados. Al principio, una ejecución que superaba la batería de tests podía parecer un éxito claro. Sin embargo, la revisión de parches preliminares mostró una diferencia importante. Algunos cambios satisfacían los tests, pero no corregían la misma causa raíz que el parche de referencia del benchmark. Por ello, antes de cerrar la evaluación final se separaron dos preguntas distintas. La primera es si el parche pasa los tests. La segunda es si ese parche repara el mismo fallo que corrigió el desarrollador.

El protocolo resultante tiene tres capas. Primero se ejecuta la batería de tests dentro del entorno experimental. Si el parche no la supera, la ejecución queda como **FAIL** y no se revisa semánticamente. Si la supera, se recupera el parche canónico de QuixBugs o BugsInPy y se prepara una comparación asistida por un juez LLM. Esa revisión clasifica el parche plausible como **CORRECT**, **PLAUSIBLE** u **OVERFITTING**, según si corrige la misma causa raíz, si solo cubre parte del comportamiento esperado o si se ajusta a los casos concretos de la batería de tests. La Figura 17 muestra el flujo completo de ejecución y validación.

La revisión semántica final se realizó con Claude Sonnet 4.5 a través de GitHub Copilot. Para mantener el criterio estable, se encapsuló el procedimiento en una *skill*, es decir, un fichero de instrucciones reutilizable que fija cuándo se activa la revisión y qué pasos debe seguir el agente evaluador. La *skill* `apr-validator` no cambia lo que evalúa el juez, pero evita que cada lote se evalúe con una formulación distinta del prompt. El Apéndice A recoge la estructura del prompt usado durante esta revisión.

Este procedimiento no convierte al juez LLM en un oráculo perfecto. Sigue siendo una evaluación asistida y puede equivocarse en casos límite. Aun así, resulta más adecuada para este trabajo que una métrica puramente léxica, porque la pregunta relevante no es si dos parches se parecen en texto, sino si reparan el mismo fallo. La evaluación experimental del capítulo siguiente se apoya precisamente en esta separación entre tests superados y corrección semántica.

7 Evaluación experimental

Una batería de tests superada es el indicador con el que la mayoría de trabajos de APR declara reparado un bug. Durante el desarrollo de este sistema ese indicador demostró ser insuficiente. Varios parches superaban las pruebas sin corregir la causa que el desarrollador original había arreglado, y otros lo conseguían con un coste muy alto. La evaluación de este capítulo parte de esa observación y mide tres dimensiones distintas para cada combinación de arquitectura, modelo y benchmark. En concreto, mide cuántos parches superan los tests, cuántos corrigen semánticamente el bug y cuánto cuesta llegar a ellos.

Las primeras secciones fijan los criterios comunes. La Sección 7.1 describe las variables del estudio, presenta los benchmarks empleados y detalla el procedimiento de ejecución. Después, la Sección 7.2 define las métricas y se justifica por qué una métrica léxica como BLEU no basta para medir la calidad de una reparación. El resto del capítulo expone los resultados, primero sobre QuixBugs y después sobre BugsInPy, antes de analizar el coste del procedimiento y responder a las preguntas de investigación.

7.1 Diseño experimental

El experimento se organiza como una matriz de configuraciones. Cada configuración combina una arquitectura de agente, un modelo LLM y un benchmark. La unidad de análisis es la ejecución individual, es decir, un intento de reparar un bug concreto con una de esas configuraciones. Trabajar a nivel de ejecución permite intentar el mismo bug con varias combinaciones de arquitectura y modelo y, después, agregar los resultados por benchmark, arquitectura, modelo y proyecto.

Variables del estudio. El estudio maneja tres variables independientes. La primera es la **arquitectura del agente**, con tres valores que corresponden a las estrategias descritas en capítulos anteriores: mono-agente (`mono_agent`), orquestador multi-agente (`multi_agent_orchestrator`) y planificador-ejecutor (`planner_executor`). La segunda es el **modelo LLM**, con otros tres valores escogidos para cubrir perfiles distintos de tamaño, despliegue y coste. Los nombres que aparecen a continuación son los identificadores exactos usados en los lotes experimentales.

Los dos primeros se ejecutan en local a través de Ollama [16], un servidor de inferencia que expone una API compatible con el estándar de *chat completions*, lo que permite usarlos con el mismo código de cliente que cualquier modelo de pago. El hardware disponible para la inferencia local es una NVIDIA RTX 4090 con 24 GB de VRAM. Ese límite condiciona directamente la selección de modelos, dado que un modelo de 26 B de parámetros cuantizado a 4 bits ocupa casi por completo esa VRAM y opciones de mayor tamaño quedan fuera de lo ejecutable en local.

- **qwen3.5-9b-ctx65k**. Modelo local con ventana de contexto de 65 000 tokens. Esa amplitud es relevante en APR porque un intento de reparación puede acumular trazas de error, varios ficheros fuente, historial de iteraciones y salidas de tests, todo dentro del mismo contexto. Se escogió como modelo local de referencia por su comportamiento consistente al invocar herramientas y porque esa ventana amplia reduce el riesgo de trunca información clave en bugs de proyectos medianos.
- **gemma4-26b-ctx32k**. Modelo local de 26 mil millones de parámetros, casi tres veces más grande que Qwen, aunque con una ventana de contexto más ajustada, de 32 000 tokens. Su inclusión responde a la pregunta de si más capacidad de razonamiento compensa la menor ventana disponible, una tensión especialmente visible en BugsInPy, donde los bugs más complejos movilizan muchos ficheros y muchas iteraciones.
- **gpt-5.4-mini**. Modelo propietario accesible mediante API. Actúa como referencia externa: ofrece un seguimiento de instrucciones más robusto y una inferencia más rápida que los modelos locales, a cambio de depender de un servicio externo y de un coste por token. Permite cuantificar en qué medida las limitaciones observadas en los modelos locales son atribuibles al modelo y no a la arquitectura del agente. Con todo, la comparación no es simétrica a nivel de infraestructura. Los modelos locales operan con los 24 GB de VRAM disponibles, lo que limita el tamaño del modelo y la ventana de contexto.

GPT, en cambio, se ejecuta sobre la infraestructura de producción de OpenAI, sin esas restricciones. Cualquier diferencia de resultados entre ambos refleja tanto la capacidad intrínseca del modelo como esas condiciones de despliegue, algo que conviene tener presente al interpretar los datos.

Antes de fijar esta matriz se probaron otras opciones locales, entre ellas Llama 3.1 8B y Mistral 7B, que no alcanzaron la selección final porque generaban diffs mal formados con frecuencia, ignoraban las instrucciones de herramienta o agotaban el contexto disponible antes de llegar a una edición útil. El **benchmark** es la tercera variable independiente y toma dos valores, QuixBugs y BugsInPy, que se describen en el apartado siguiente. Todos los demás elementos se mantuvieron constantes para que cualquier diferencia observada fuera atribuible a la arquitectura o al modelo y no al entorno. Esto incluye un mismo entorno aislado, el mismo conjunto de herramientas APR, un límite de tres iteraciones por intento y un único protocolo de validación semántica, aplicado con Claude Sonnet 4.5 a través de GitHub Copilot [6] sobre los parches que superaban los tests.

Benchmarks. La evaluación emplea dos benchmarks de reparación de programas Python con propósitos complementarios, ya presentados en el panorama de benchmarks del capítulo de antecedentes. Ambos proporcionan, para cada bug, una versión defectuosa del código, una batería de tests que expone el fallo y un parche de referencia escrito por un desarrollador. Esa terna permite reproducir el error en un entorno controlado y comparar el parche del agente con una solución conocida. La cuestión aquí no es qué es cada benchmark, sino qué papel cumple en el diseño y cómo se construyó la muestra evaluada.

QuixBugs [13] aporta bugs algorítmicos acotados, casi siempre de una sola línea en un único fichero, donde la dificultad reside en el razonamiento sobre el programa y no en el tamaño del repositorio. Su carácter cerrado y su uso extendido lo convierten en el punto de referencia para validar el *harness* y situar el sistema frente a trabajos previos. BugsInPy [27], en cambio, recoge fallos reales del historial de proyectos Python de uso extendido, como *tornado*, *fastapi*, *scrapy* o *youtube-dl*. Aquí intervienen dependencias de proyecto, contratos internos entre módulos, parches que pueden tocar varios ficheros y baterías de tests que solo cubren una parte del comportamiento esperado, lo que lo acerca a la reparación de software real y lo convierte en el benchmark principal para distinguir entre arquitecturas.

En QuixBugs se utiliza el conjunto completo de 40 bugs, lo que evita depender de una muestra reducida y facilita la comparación con la literatura. BugsInPy es mucho más grande y heterogéneo, así que se construyó una muestra estratificada de 53 bugs repartidos en 11 proyectos. La estratificación responde a un problema concreto. El número de bugs disponibles por proyecto está muy desequilibrado, y una muestra aleatoria habría quedado dominada por los proyectos más representados. Para evitarlo se aplicó un límite máximo de bugs por proyecto, de forma que cada uno aportara un peso comparable al resultado agregado. Dentro de cada proyecto se eligieron bugs de complejidad baja, media y alta según el tamaño del parche de referencia y el número de ficheros que modificaba. Se descartaron los proyectos que no se integraban de forma fiable en el entorno reproducible, ya que el objetivo era comparar arquitecturas y modelos, no medir incidencias de preparación del benchmark.

Antes del experimento final se ejecutaron lotes preliminares para ajustar el *harness*: orden de los fragmentos de contexto, límites de lectura, umbral de progreso y otros parámetros operativos del bucle iterativo. Esos lotes usaron un subconjunto distinto de bugs, que no se incluyeron en la muestra evaluada. La separación asegura que las decisiones de configuración del entorno no se tomaron con conocimiento de los resultados finales, de modo que el ajuste del *harness* no contamina la evaluación.

La elección de estos dos benchmarks responde al foco del trabajo. Tanto la plataforma como las herramientas y el protocolo de validación se centran en APR de Python guiado por tests, condición que QuixBugs y BugsInPy cumplen y que permite cubrir desde el razonamiento algorítmico hasta el fallo real de proyecto. Defects4J y SWE-bench quedan fuera por medir problemas distintos: el primero está centrado en Java, mientras que el segundo evalúa la resolución de incidencias a escala de repositorio, un objetivo más amplio que la reparación acotada que aborda este TFM.

Procedimiento de ejecución y análisis. Cada configuración se ejecutó como un lote de intentos independientes. Para cada bug se preparó un entorno reproducible y aislado, se lanzó el agente con la arquitectura

y el modelo correspondientes, y se registraron las iteraciones, llamadas a herramientas, consumo de tokens, duración y resultado de tests. Después, los resultados se agregaron de forma uniforme por benchmark, arquitectura, modelo y bug.

La ejecución se organizó en nueve configuraciones por benchmark, resultado de cruzar tres arquitecturas por tres modelos. En QuixBugs se realizaron 40 intentos por configuración, uno por bug, hasta completar 360 ejecuciones. En BugsInPy se reutilizó la muestra estratificada ya descrita en la Sección 7.1, con 53 bugs por configuración, de modo que el bloque quedó en 477 ejecuciones.

La validación semántica se aplicó después de la ejecución, siguiendo el protocolo descrito en la Sección 6.4. Solo entraron al juez las ejecuciones que habían pasado los tests. Por tanto, **CORRECT**, **PLAUSIBLE** y **OVERFITTING** son categorías dentro de los parches plausibles. En cambio, **FAIL** agrupa los intentos que no produjeron un parche aceptado por la batería de tests. Los tres veredictos semánticos se definen así:

- **CORRECT**. El parche aborda la misma causa raíz que la referencia canónica. No es necesario que coincida línea por línea, sino que la lógica de la corrección sea equivalente.
- **PLAUSIBLE**. El parche supera los tests pero no es semánticamente equivalente al canónico. Puede resolver solo parcialmente el problema, emplear un enfoque distinto que no reproduce la intención del desarrollador o incluir cambios colaterales innecesarios.
- **OVERFITTING**. El parche satisface las aserciones sin resolver la causa raíz. Ejemplos típicos son codificar un valor numérico observado en el test en lugar de computarlo, silenciar la excepción con un `try/except` vacío o modificar el fichero de tests.

El protocolo completo de evaluación, incluyendo el árbol de decisión y el prompt del juez, se detalla en el Apéndice A.

7.2 Métricas de evaluación

La métrica primaria es la tasa de tests pasados, definida como la proporción de ejecuciones cuyo parche supera la batería de pruebas. Es el indicador estándar en APR y la primera condición para considerar plausible un parche. Sobre ella se construye una segunda capa que mide la calidad semántica únicamente dentro de los parches que pasan tests, a partir de los veredictos del juez descritos en la Sección 6.4. Sin esa segunda capa, un parche que corrige el fallo y otro que solo cubre una rama del test contarían igual. Con esto en mente, las métricas empleadas son las siguientes:

- **Tasa de tests pasados**. Ejecuciones con tests superados sobre ejecuciones totales. Cuantifica los parches plausibles en el sentido clásico de APR.
- **Tasa de corrección**. Ejecuciones **CORRECT** sobre ejecuciones con tests pasados. Indica qué proporción de los parches plausibles corrige semánticamente el bug.
- **Tasa de parches plausibles no correctos**. Ejecuciones **PLAUSIBLE** sobre ejecuciones con tests pasados. Recoge los parches que superan las pruebas pero no son equivalentes al canónico. Es decir, abordan el caso observado sin reproducir por completo la intención del desarrollador.
- **Tasa de sobreajuste**. Ejecuciones **OVERFITTING** sobre ejecuciones con tests pasados. Mide los parches que satisfacen los tests sin resolver la causa raíz.
- **Tasa de reparación correcta global**. Ejecuciones **CORRECT** sobre ejecuciones totales. Combina el indicador de tests y la validación semántica en una única tasa de reparación.
- **Diferencia entre tests y corrección**. Diferencia entre la tasa de tests pasados y la tasa de reparación correcta global. Cuantifica cuánto sobrestiman los tests la corrección real.
- **Coste por ejecución**. Tokens de entrada y salida y número de llamadas a herramientas por ejecución. Aproximan el coste del procedimiento y el tamaño del contexto que el modelo debe procesar, y se combinan más adelante en tokens por reparación correcta.

Además de estas métricas agregadas, se analiza la composición de herramientas. En este sistema, las llamadas a herramientas no son un detalle menor. Un agente que lee demasiado puede gastar el presupuesto

antes de editar. En cambio, un agente que edita demasiado pronto puede pasar por alto el contrato real de la función. Por eso se separan las llamadas de exploración, edición, tests/comandos y control de cambios. Esta división no pretende decir que una categoría sea siempre buena o mala, sino medir si una configuración convierte su trabajo en parches correctos con una relación razonable entre acciones y resultado.

7.3 Validación semántica frente a métricas léxicas

BLEU [18] es una métrica de similitud textual, originaria de la traducción automática, que mide el solapamiento de n-gramas entre un texto candidato y una referencia. Aplicada a parches, compara el código generado con el parche canónico carácter a carácter, sin atender a lo que el código hace. Este detalle la hace poco adecuada para evaluar reparaciones de software, porque la similitud textual no equivale necesariamente a equivalencia funcional.

El problema aparece en dos direcciones. Un parche correcto puede tener una puntuación BLEU baja si usa otra estructura de control, cambia nombres auxiliares o reordena una comprobación sin alterar el comportamiento. En cambio, un parche incorrecto puede parecerse mucho al canónico y seguir fallando en una rama no cubierta por los tests. Esta limitación encaja con los trabajos que estudian el sobreajuste de parches y la evaluación semántica en APR [34,33], así como con la evidencia reciente de que las métricas léxicas pueden correlacionar mal con la corrección semántica en tareas de código [15].

Por ese motivo, BLEU se descartó como métrica de calidad del parche. En su lugar, la evaluación utiliza un juez LLM con un prompt fijo y parches canónicos como referencia. Esta decisión no convierte la validación en un criterio definitivo, pero se ajusta mejor a la pregunta del experimento, pues no se busca saber si dos parches se parecen en texto, sino si reparan la misma causa raíz.

7.4 Resultados globales

El experimento final contiene 837 ejecuciones, 360 sobre QuixBugs y 477 sobre BugsInPy. La Figura 18 resume el resultado más importante en forma de embudo. En QuixBugs, 340 de 360 ejecuciones pasan tests y 286 se clasifican como CORRECT. En BugsInPy, 210 de 477 pasan tests, pero solo 74 son CORRECT. La distancia entre la segunda y la tercera barra de cada benchmark es la observación principal de la evaluación.

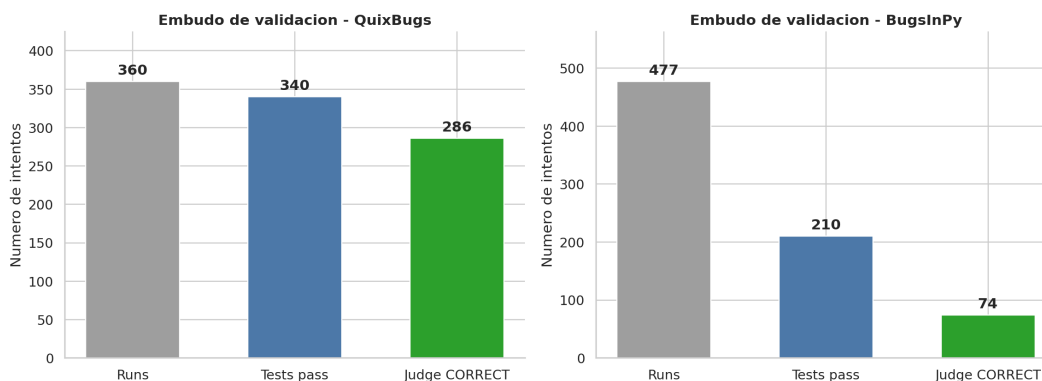


Fig. 18. Embudo de validación por benchmark. La primera barra muestra los intentos ejecutados, la segunda los que pasan tests y la tercera los que el juez clasifica como CORRECT.

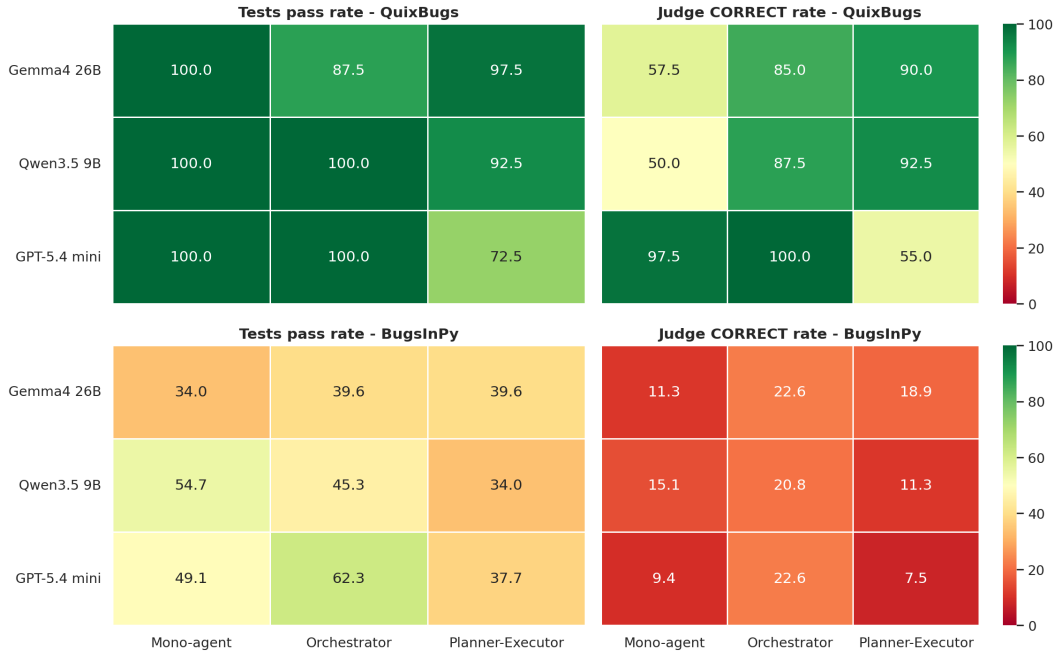
En QuixBugs la plataforma funciona bien sobre bugs acotados. El 94.4 % de las ejecuciones pasa tests y el 84.1 % de esos parches recibe el veredicto CORRECT. Aun así, 54 parches superan la batería sin considerarse equivalentes al canónico, de modo que la brecha entre tests y corrección existe incluso aquí, aunque sigue siendo relativamente contenida y pasar tests continúa siendo un indicador bastante fiable de corrección.

Table 5. Resultados globales por benchmark.

Dataset	Ejec.	Tests pasan	CORRECT	PLAUSIBLE	OVERFITTING	Rep. correcta global
QuixBugs	360	340 (94.4%)	286/340 (84.1%)	54/340 (15.9%)	0/340 (0.0%)	286/360 (79.4%)
BugsInPy	477	210 (44.0%)	74/210 (35.2%)	125/210 (59.5%)	11/210 (5.2%)	74/477 (15.5%)

BugsInPy lleva esa misma brecha a otra escala. Pasa los tests el 44.0 % de las ejecuciones, pero solo el 35.2 % de esos parches son **CORRECT**. La mayoría queda en **PLAUSIBLE**, es decir, resuelve el caso observado o una parte del problema sin reproducir por completo la intención del parche de referencia, y la tasa de reparación global cae hasta el 15.5 %.

Desglosado por configuración, el patrón es consistente. En QuixBugs la brecha entre tests pasados y parches **CORRECT** es pequeña en casi todas las celdas de la Figura 19: pasar tests es aquí un indicador razonablemente fiable de corrección. En BugsInPy esa relación se rompe de forma generalizada. La brecha es amplia e independiente del modelo o la arquitectura, lo que descarta una explicación puntual y apunta a una dificultad estructural del benchmark. El ranking de la Figura 20 completa la imagen: en QuixBugs las configuraciones se acumulan en la parte alta, mientras que en BugsInPy se distribuyen entre el 5 % y el 24 %, con margen suficiente para distinguir entre ellas.

**Fig. 19.** Comparación entre tasa de tests pasados y tasa global de parches **CORRECT** por modelo, arquitectura y benchmark.

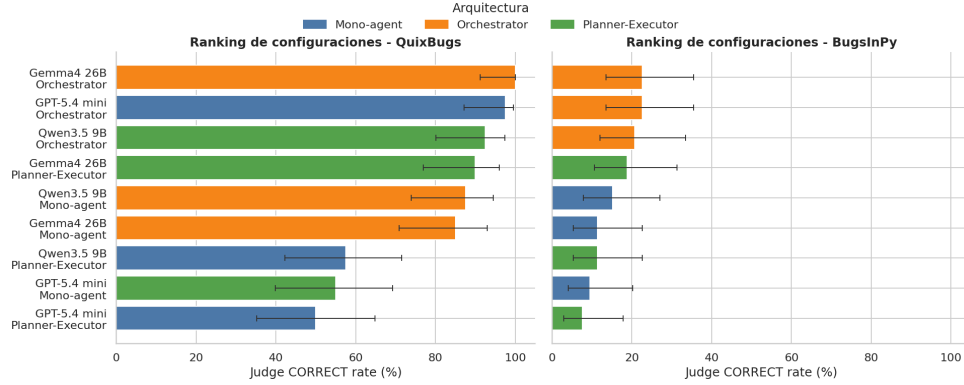


Fig. 20. Ranking de configuraciones por tasa global de parches CORRECT. Las barras incluyen intervalos de incertidumbre estimados sobre las ejecuciones observadas.

7.5 Resultados en QuixBugs

QuixBugs queda cerca de la saturación para las mejores configuraciones. Eso tiene una consecuencia ambivalente, ya que confirma que el sistema resuelve con solvencia los bugs algorítmicos acotados, pero al mismo tiempo reduce su capacidad de discriminar arquitecturas. Cuando un modelo resuelve casi todos los programas, queda poco margen para atribuir la mejora a la organización del agente o a la capacidad del modelo.

El techo lo marca `gpt-5.4-mini`, que con el orquestador repara correctamente los 40 programas y con mono-agente se queda en 39. Los modelos locales no quedan lejos cuando el flujo de trabajo les ayuda a estructurar la tarea. Qwen con planificador-ejecutor alcanza 37 reparaciones CORRECT y Gemma con la misma arquitectura llega a 36. Sobre las 360 ejecuciones del benchmark, 340 superan los tests y 286 reciben el veredicto CORRECT.

La arquitectura, sin embargo, no afecta a todos los modelos por igual. El planificador-ejecutor favorece a los modelos locales pero penaliza a GPT, que con esa estrategia baja a 29 tests superados y 22 reparaciones correctas, por debajo de sus otras dos configuraciones. El plan rígido restringe a un modelo que rinde mejor cuando dispone de libertad para iterar sobre su propio análisis.

El dato más revelador de QuixBugs no está en las configuraciones que ganan, sino en las que pasan todos los tests sin ser equivalentes al parche de referencia. Qwen con mono-agente supera la batería en los 40 programas, pero solo 20 de esos parches son CORRECT y los otros 20 quedan en PLAUSIBLE. Gemma con mono-agente repite el patrón, con 40 tests superados y 23 reparaciones correctas. La brecha entre pasar tests y reparar aparece, por tanto, incluso en bugs de una sola línea, y lo hace de forma sistemática en la arquitectura mono-agente con modelos locales, que tiende a ajustar el código a las aserciones observadas en lugar de a la intención del programa.

Dos casos concretos ilustran los extremos. En el problema de camino mínimo (`shortest_path_length`), Qwen con planificador-ejecutor produce exactamente el cambio canónico, sustituyendo la distancia incorrecta acumulada en la cola por la distancia real al nodo en expansión. El parche no solo pasa los tests, sino que coincide con la corrección del desarrollador, y el juez lo clasifica como CORRECT con confianza alta. En el extremo opuesto, en `mergesort` bajo GPT con planificador-ejecutor el agente supera la batería de tests con un cambio que cubre los casos de prueba pero no equivale al parche de referencia, por lo que termina como PLAUSIBLE.

7.6 Resultados en BugsInPy

BugsInPy invierte la imagen anterior. De las 477 ejecuciones, 210 superan los tests (44.0 %), pero solo 74 reciben el veredicto **CORRECT**, lo que sitúa la tasa de reparación global en el 15.5 %. Aquí el benchmark sí discrimina, y las diferencias entre arquitecturas y modelos se vuelven visibles.

Agrupando por arquitectura, el orquestador es la estrategia más eficaz, con 35 reparaciones **CORRECT**, frente a las 20 del planificador-ejecutor y las 19 del mono-agente. Las tres mejores configuraciones individuales son las tres variantes del orquestador, muy próximas en número de aciertos pero con perfiles distintos.

- **GPT con orquestador.** 12 reparaciones **CORRECT** sobre 51 ejecuciones. Es la configuración que más tests supera, con 33 de 51, pero buena parte de esos parches se queda en **PLAUSIBLE**.
- **Gemma con orquestador.** 12 reparaciones **CORRECT** sobre 52 ejecuciones. Supera menos tests, 21 de 52, pero convierte una proporción mayor de ellos en reparaciones correctas, lo que refleja una precisión semántica más alta.
- **Qwen con orquestador.** 11 reparaciones **CORRECT** sobre 51 ejecuciones. Queda muy cerca en aciertos, pero a un coste de exploración y tokens mucho mayor, como se detalla en la Sección 7.7.

La distancia entre pasar tests y corregir semánticamente el bug se ve con claridad en una sola configuración. GPT con mono-agente supera los tests en 26 ejecuciones, una cifra notable para BugsInPy, pero solo 5 de esos parches reciben el veredicto **CORRECT**. Los 21 restantes pasan las pruebas sin coincidir con la corrección que aplicó el desarrollador, de modo que una evaluación basada únicamente en los tests habría sobrevalorado esta configuración. La Figura 21 generaliza el fenómeno: en QuixBugs la mayoría de los parches que pasan tests son **CORRECT**, mientras que en BugsInPy crece el peso de los **PLAUSIBLE** y los **OVERFITTING**.

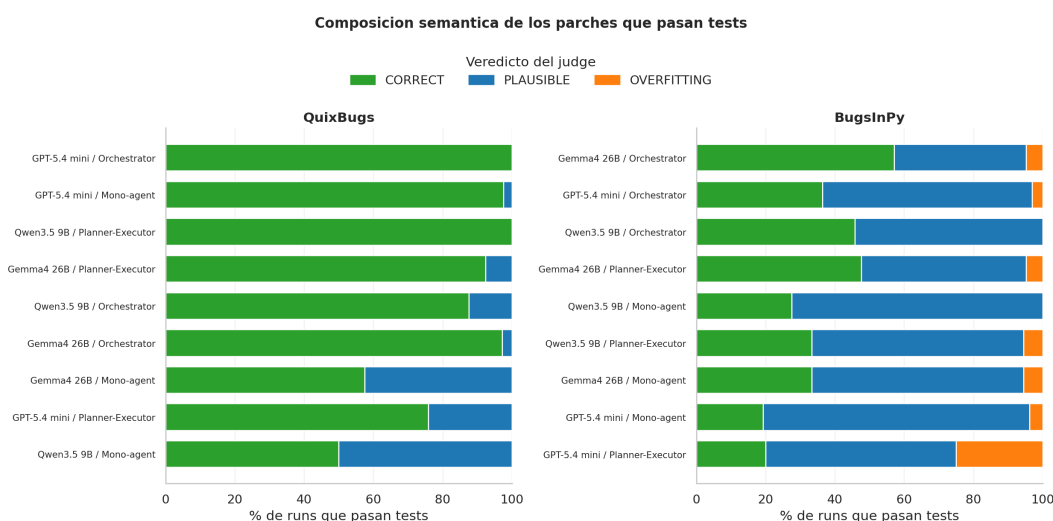


Fig. 21. Composición semántica de los parches que pasan tests. Cada barra se normaliza sobre las ejecuciones plausibles de una configuración.

La dificultad tampoco se reparte de forma homogénea entre proyectos. La Figura 22 muestra la tasa de reparación correcta por proyecto y modelo, y el rango es amplio. En un extremo, **black**, un formateador de código con lógica densa y ficheros extensos, no recibe ninguna reparación correcta en ninguna configuración (0 de 40 intentos). **scrapy** apenas alcanza el 2.3 % y **youtube-dl** el 8.9 %. En el otro extremo, **tornado** llega al 27.3 % y **httpie**, **thefuck** y **tqdm** rondan el 24 %. Esta dispersión revela que la tasa global del

15.5 % promedia situaciones muy distintas, y que la viabilidad del enfoque depende tanto de la naturaleza del proyecto como de la arquitectura empleada.

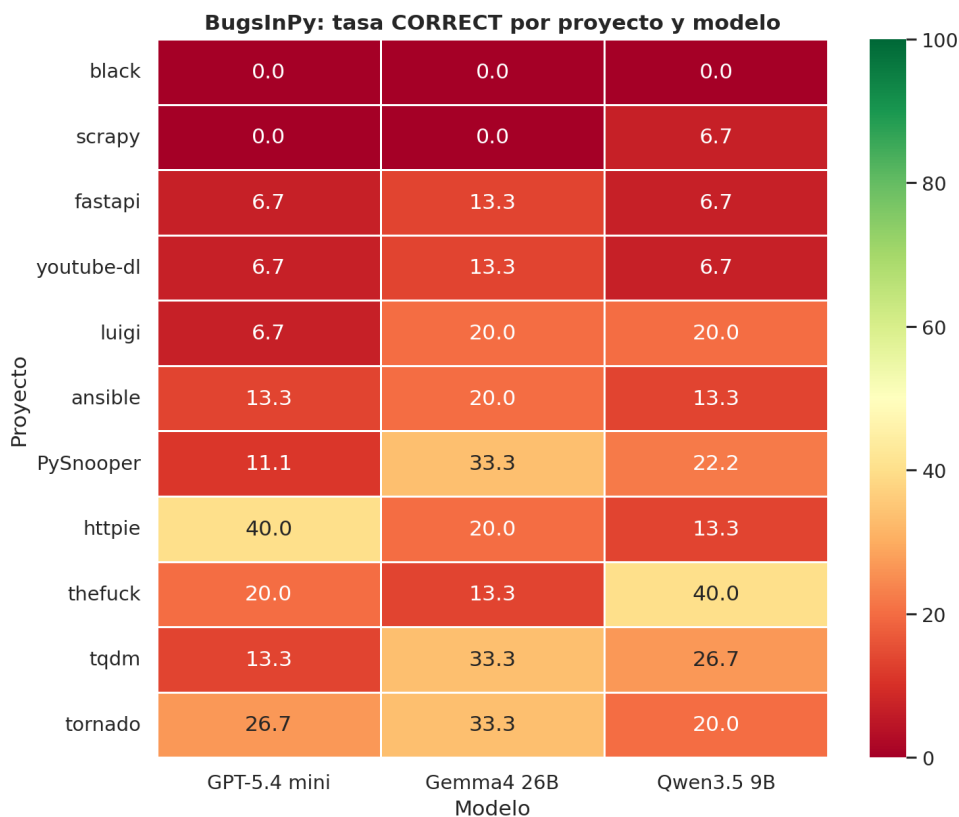


Fig. 22. Tasa de reparación CORRECT por proyecto y modelo en BugsInPy. Las celdas más claras corresponden a proyectos donde casi ninguna configuración logra reparaciones correctas.

Dos ejemplos concretos muestran por qué la categoría PLAUSIBLE resulta imprescindible. En `fastapi-5`, GPT con mono-agente introduce el cambio canónico en la clonación recursiva de campos dentro de `utils.py`, pero lo acompaña de modificaciones innecesarias en el módulo de enrutado. El parche pasa los tests y aborda la causa real, aunque añade alteraciones ajenas a la solución de referencia, así que el juez lo deja en PLAUSIBLE. En `httpie-1`, Gemma con mono-agente recibe el veredicto OVERFITTING. En lugar de consultar el límite del sistema con `os.pathconf`, codifica el valor 255 directamente y rodea la lógica con varios bucles redundantes. El resultado satisface las pruebas, pero rompe el comportamiento general de la función y no se sostiene fuera de los casos cubiertos por los tests.

7.7 Uso de herramientas, tokens y coste del procedimiento

El análisis anterior explica qué configuraciones reparan más, pero todavía falta una parte importante, que es cuánto trabajo necesita cada una para hacerlo. En sistemas agénticos, el coste no es solo tiempo de ejecución. También importa el número de tokens consumidos, porque cada lectura de fichero, cada salida de tests y cada resumen de iteración pasan a formar parte del contexto que el modelo debe procesar. Además, las llamadas a herramientas muestran cómo trabaja el agente: si explora, si edita, si repite tests o si revisa cambios.

La Figura 23 muestra el perfil medio de herramientas por configuración. En QuixBugs, muchas configuraciones necesitan pocas llamadas. GPT con orquestador, por ejemplo, alcanza el mejor resultado con

algo más de 5 llamadas por ejecución de media. En BugsInPy la escala cambia por completo. Qwen con Planner-Executor supera las 63 llamadas medias por ejecución, Qwen con orquestador llega a 52 y Qwen con mono-agente supera las 50. Esto no significa automáticamente que Qwen razone peor, pero sí que necesita mucha más interacción con el entorno para intentar construir la reparación.

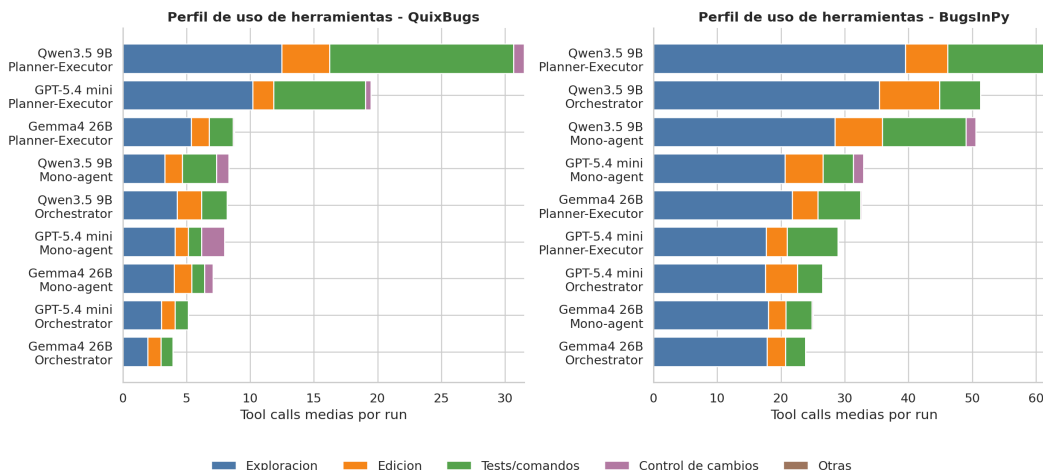


Fig. 23. Perfil medio de uso de herramientas por configuración. Las llamadas se agrupan en exploración, edición, tests/comandos, control de cambios y otras acciones.

El patrón de exploración es especialmente importante. En BugsInPy, la mayor parte de las llamadas son de lectura y búsqueda de contexto. Esto tiene sentido, porque los bugs reales obligan a entender contratos internos de proyecto. No obstante, hay una tensión clara. Explorar más puede evitar correcciones precipitadas, pero también puede consumir llamadas y tokens de contexto antes de llegar a una edición útil. El Planner-Executor con Qwen es el ejemplo más extremo, ya que combina muchas llamadas de exploración con muchas ejecuciones de tests/comandos, pero esa actividad no se transforma en una tasa alta de CORRECT.

La Figura 24 relaciona llamadas a herramientas y calidad. En QuixBugs se ve que más llamadas no implican mejor resultado. GPT con orquestador y Gemma con orquestador están en la zona de bajo uso de herramientas y alta corrección. En el extremo contrario, Qwen con Planner-Executor usa muchas más llamadas y obtiene un resultado alto, pero no proporcional al coste. En BugsInPy ocurre algo parecido, aunque con tasas de corrección mucho más bajas. Las configuraciones del orquestador quedan entre las mejores, pero Qwen necesita el doble de llamadas que Gemma o GPT para acercarse a su resultado.

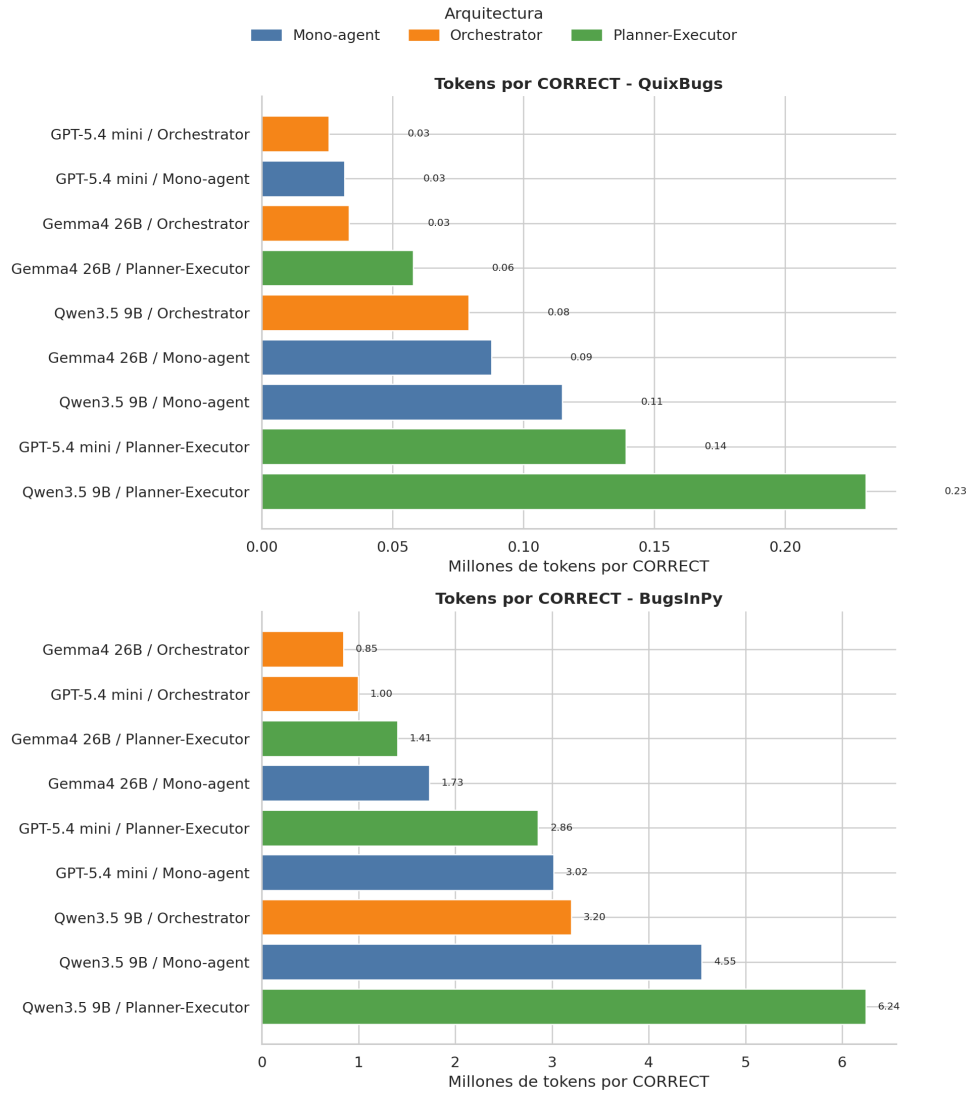


Fig. 25. Millones de tokens consumidos por cada parche CORRECT. Valores menores indican mejor relación entre coste de contexto y reparación correcta.

Esta lectura también ayuda a interpretar el papel de la arquitectura. El orquestador no gana porque use siempre menos herramientas. De hecho, con Qwen usa muchas. Su ventaja está en que convierte mejor la exploración en parches correctos. Para cuantificarlo, se puede expresar el rendimiento como el número de reparaciones CORRECT por cada 100 llamadas a herramientas: GPT con orquestador obtiene 0.88, Gemma con orquestador 0.95 y Qwen con orquestador 0.41. En mono-agente, esos valores bajan de forma clara para GPT y Qwen. Por tanto, la arquitectura no solo cambia la tasa final, sino también la eficiencia con la que se aprovechan las acciones del agente.

7.8 Iteraciones, tiempos y causas de fallo

Las secciones anteriores explican qué configuraciones reparan más y a qué coste. Completar la imagen requiere analizar también cuándo fracasan los intentos que no producen una reparación: en qué iteración termina cada

ejecución, cuánto tiempo consumen los intentos fallidos y por qué motivo se detienen. El sistema dispone de hasta tres iteraciones por bug, donde cada iteración encadena un intento de parche y la ejecución de la batería de tests.

Reparaciones según la iteración de terminación. La Figura 26 muestra la composición de veredictos según la iteración en la que termina cada ejecución. El patrón es claro y se repite en ambos benchmarks: la mayoría de las reparaciones correctas se producen en el primer intento. En QuixBugs, 170 de las 286 reparaciones **CORRECT** se alcanzan en la iteración 1, donde además el 80.6 % de las ejecuciones que terminan ahí son correctas. En BugsInPy ocurre algo equivalente en proporción: de las 74 reparaciones **CORRECT**, 39 llegan en la primera iteración y solo 5 en la tercera.

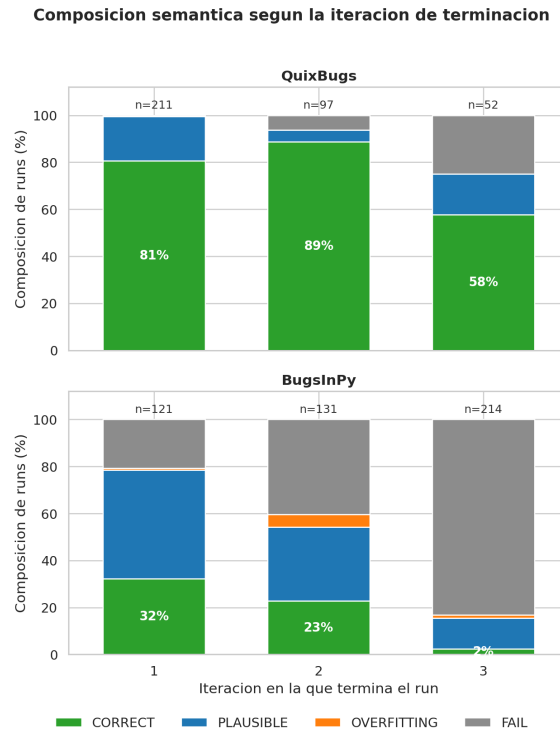


Fig. 26. Composición de veredictos según la iteración en la que termina la ejecución, normalizada al 100 % por iteración. La reparación correcta se concentra en la primera iteración, mientras que las ejecuciones que llegan a la tercera están dominadas por **FAIL**, sobre todo en BugsInPy.

El dato más revelador aparece al mirar las ejecuciones que agotan el presupuesto. En BugsInPy, 214 de las 477 ejecuciones llegan a la tercera iteración, pero de ellas apenas el 2.3 % acaba en **CORRECT** y el 83.2 % en **FAIL**. Las iteraciones adicionales, lejos de abrir oportunidades nuevas, funcionan en su mayoría como confirmación del fracaso, pues lo que ocurre en la primera iteración predice en gran medida el resultado final.

Coste temporal de los intentos sin éxito. El coste temporal refuerza esta lectura. En QuixBugs los tiempos son cortos y apenas distinguen entre éxito y fracaso, con medianas de 47.2 y 54.4 segundos respectivamente. En BugsInPy, en cambio, la diferencia es notable: las ejecuciones que superan los tests tardan una mediana de 130.6 segundos, mientras que las que no lo consiguen se prolongan hasta 503.3 segundos, casi cuatro veces más. Dado que en BugsInPy la mayoría de intentos no terminan en reparación, este desequilibrio concentra una parte muy alta del tiempo total en ejecuciones que no aportan parches correctos.

Causas de terminación de los intentos fallidos. Conviene precisar también por qué motivo terminan los intentos que no reparan el bug. El sistema registra una causa de parada por cada ejecución, que puede ser una de las siguientes.

- **Agotamiento de iteraciones** (`max_iterations`). El agente consume las tres iteraciones disponibles sin producir un parche que pase los tests. Es el desenlace por defecto cuando el bug no se resuelve dentro del presupuesto.
- **Límite de tiempo** (`timed_out`). La ejecución supera el tiempo máximo asignado y se interrumpe antes de agotar las iteraciones. Aparece sobre todo en los modelos locales, más lentos en inferencia.
- **Fallo de validación** (`validation_failure`). El cambio propuesto corrompe el espacio de trabajo o no supera las comprobaciones de integridad previas a aceptarlo, por lo que la ejecución se detiene sin un parche válido.
- **Falta de progreso** (`no_progress`). El agente se declara estancado porque deja de producir cambios útiles entre iteraciones y abandona antes de gastar todo el presupuesto.
- **Fallo de herramienta** (`tool_failure`). Una de las herramientas del agente falla de forma no recuperable e impide continuar el intento.

El reparto de estas causas confirma el comportamiento descrito. En BugsInPy, de las 256 ejecuciones que no superan los tests, 162 terminan por agotamiento de iteraciones y 59 por límite de tiempo, estas últimas concentradas en Gemma y Qwen. El resto se distribuye entre fallos de validación del entorno (20), interrupciones por falta de progreso (12) y fallos de herramienta (3). En QuixBugs, por el contrario, la inmensa mayoría de las ejecuciones terminan de forma natural y los modos de fallo son anecdóticos, con apenas 13 casos de agotamiento de iteraciones. La Figura 27 desglosa este reparto por configuración.

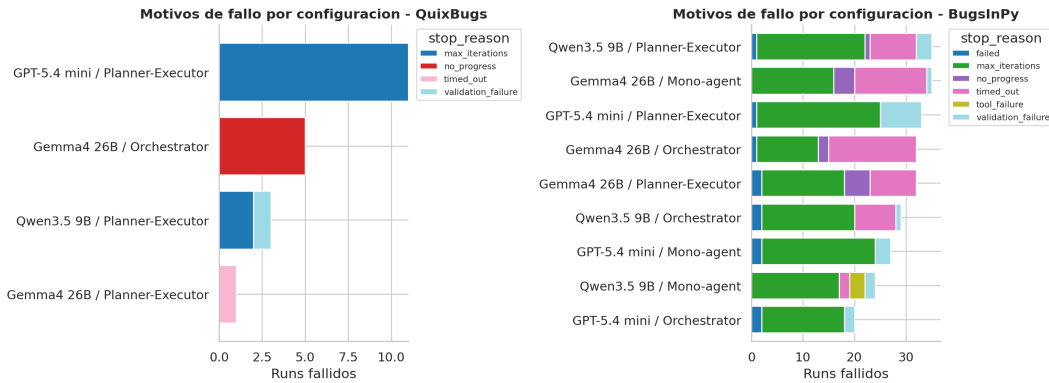


Fig. 27. Causa de parada por configuración. Las ejecuciones que no reparan el bug en BugsInPy se concentran en el agotamiento del presupuesto de iteraciones y en el límite de tiempo, mientras que las paradas por falta de progreso son minoritarias.

Las paradas por falta de progreso son minoritarias, lo que indica que el sistema no tiene criterios suficientemente sensibles para detectar cuándo un intento ya no puede prosperar. Eso sugiere una línea de mejora clara, que es introducir criterios de parada temprana más exigentes cuando la primera iteración no ofrece indicios de progreso real.

7.9 Análisis comparativo de arquitecturas y modelos

Las preguntas de investigación planteadas en la Sección 3.1 se pueden responder de forma directa a partir de los resultados anteriores. RQ1 preguntaba si la estrategia de orquestación afecta a la eficacia y la calidad de

los parches. RQ2, por su parte, se centraba en las diferencias que introduce el modelo empleado en términos de eficacia, eficiencia y coste.

La respuesta a RQ1 es que la arquitectura influye, pero su efecto depende mucho del benchmark. En QuixBugs, varias configuraciones alcanzan tasas muy altas y el margen de mejora es pequeño. En BugsInPy, el orquestador obtiene la mejor tasa global de reparaciones correctas y también una mejor relación entre trabajo realizado y resultado. Esto sugiere que el orquestador, con su modelo de coordinación flexible, encaja mejor con bugs reales de proyectos Python que la separación rígida entre plan y ejecución del planificador-ejecutor.

No obstante, el resultado no debe leerse como que añadir agentes siempre mejora. En APR guiado por tests, el *traceback* ya actúa como una forma inicial de localización del fallo. Por eso, la ganancia no procede únicamente de dividir el trabajo entre roles. También depende de cómo se conserva el estado entre iteraciones, cómo se resume el resultado de los tests y si el agente recibe el código adecuado antes de proponer el parche. La arquitectura importa, pero no se puede separar del contexto que recibe cada agente ni del control sobre el uso de herramientas.

La respuesta a RQ2 tampoco es una jerarquía simple entre modelos locales y propietarios. GPT domina QuixBugs y pasa más tests en BugsInPy, pero Gemma y Qwen son competitivos en reparaciones **CORRECT** cuando se combinan con el orquestador. La diferencia más clara aparece en el coste. GPT resuelve con menor tiempo operativo, Gemma ofrece una buena relación entre tokens y corrección en BugsInPy, y Qwen necesita mucha más exploración para acercarse a resultados similares. Por tanto, los modelos locales pueden ser útiles, pero su viabilidad depende mucho del presupuesto disponible y del control del contexto.

La Tabla 6 resume los hallazgos principales por benchmark, arquitectura, modelo y métrica.

Table 6. Resumen transversal de hallazgos por factor de análisis.

Factor	Hallazgo principal en el experimento final
Benchmark	QuixBugs discrimina poco entre configuraciones altas, mientras que BugsInPy amplifica la diferencia entre pasar tests y reparar semánticamente.
Arquitectura	El orquestador ofrece la mejor relación entre tasa CORRECT y coste en BugsInPy, frente al mono-agente y al planificador-ejecutor.
Modelo	GPT pasa más tests, Gemma convierte una fracción mayor de plausibles en CORRECT , y Qwen se acerca en aciertos con un coste de tokens claramente superior.
Métrica	Evaluar solo tests pasados sobrestima la calidad del fix, sobre todo en BugsInPy, donde predomina la categoría PLAUSIBLE .

La Tabla 7 sitúa estos resultados frente a trabajos cercanos de APR con LLMs. No debe leerse como una clasificación directa, ya que las filas no reportan la misma métrica ni parten de condiciones equivalentes. Algunos trabajos publican reparaciones absolutas, otros mejoras relativas frente a un baseline, y este TFM informa veredictos **CORRECT** bajo validación semántica. La utilidad de la tabla es contextual, no competitiva: permite ver qué tipo de resultado comunica cada trabajo y bajo qué condiciones debe interpretarse.

En QuixBugs, la comparación confirma que el benchmark está cerca de saturarse para modelos actuales. Los resultados de este TFM alcanzan el techo del conjunto, pero eso no implica que la arquitectura propuesta sea superior a los trabajos previos en términos generales. La diferencia principal está en el procedimiento. Los trabajos de generación directa reciben un contexto ya acotado, mientras que el sistema evaluado aquí actúa como agente con herramientas, ejecuta tests y clasifica los parches que superan la batería mediante validación semántica. Por ello, QuixBugs resulta útil para comprobar que el harness funciona, pero no basta para distinguir con claridad entre arquitecturas.

En BugsInPy la comparación tiene otro sentido. Los trabajos recientes consiguen tasas de reparación más altas porque abordan de forma explícita el problema del contexto: seleccionan información relevante, inyectan conocimiento por capas o incorporan historial del repositorio. En cambio, este TFM no busca optimizar una técnica de contexto concreta, sino medir qué cambia al variar la organización del agente

Table 7. Métricas reportadas en trabajos relacionados y datos de referencia de este TFM. Las filas no son directamente comparables entre sí.

Trabajo	Benchmark	Métrica reportada	Condiciones relevantes
Codex en QuixBugs [20]	QuixBugs	27/40	Generación <i>zero-shot</i>
Modelos preentrenados [29]	QuixBugs	39–40/40	Localización perfecta
ChatGPT en QuixBugs [25]	QuixBugs	31/40	Diálogo con contexto
Este TFM, mejor config.	QuixBugs	40/40 CORRECT	Agente con validación semántica
Maniple [19]	BugsInPy	88/157	Selección de información
Conocimiento jerárquico [4]	BugsInPy	250/314	Contexto por capas
HAFixAgent [24]	BugsInPy	+38.6% sobre baseline	Historial Git y <i>blame</i>
Este TFM, mejor config.	BugsInPy	12/51 CORRECT	Mejor celda bajo comparación de arquitecturas
Este TFM, agregado	BugsInPy	74/477 CORRECT	Tres arquitecturas, tres modelos y validación semántica

manteniendo constantes el entorno, las herramientas, las iteraciones y el protocolo de validación. Desde esa perspectiva, los resultados refuerzan una conclusión común con la literatura: en APR con LLMs, el contexto que recibe el modelo y la forma en que se controla su interacción con el entorno explican una parte central del rendimiento.

Esta lectura conecta también con sistemas agénticos más generales. SWE-agent mostraba que el diseño de la interfaz entre agente y entorno puede mover el rendimiento tanto como el modelo en sí. Agentless, por su parte, mostraba que más autonomía no implica automáticamente más eficiencia. Este TFM llega a una conclusión complementaria desde APR guiado por tests: no importa solo cuántos agentes participan, sino qué información reciben, cuándo la reciben y cómo se controla el uso de herramientas.

La principal limitación del experimento es que la validación semántica sigue siendo una aproximación asistida por LLM. El uso de un prompt fijo, un mismo modelo juez y parches canónicos reduce la variabilidad, pero no elimina completamente la ambigüedad en casos límite. También hay que tener en cuenta que la muestra de BugsInPy no cubre todo el benchmark, sino una selección estratificada. Aun así, separar tests, corrección semántica y coste proporciona una imagen mucho más rica que medir únicamente cuántas ejecuciones terminan con los tests superados: con solo esa tasa, BugsInPy mostraría un 44.0% de reparaciones y ocultaría tanto la magnitud del sobreajuste como las diferencias reales entre configuraciones.

8 Conclusiones y trabajo futuro

Este último capítulo resume lo aprendido durante el desarrollo y la evaluación del sistema. En primer lugar, se responden las preguntas de investigación a partir de los resultados obtenidos (ver Sección 8.1). Después, se recogen varias líneas de trabajo futuro que surgen de las limitaciones detectadas y de las decisiones que quedaron fuera del alcance de este TFM (ver Sección 8.2).

8.1 Conclusiones

El trabajo ha permitido construir y evaluar una plataforma APR basada en agentes LLM sobre dos benchmarks Python. En la práctica, el sistema muestra que un agente equipado con herramientas de lectura, edición y validación puede reparar bugs reales de forma reproducible. El resultado agregado lo deja claro: 837 ejecuciones en total, 550 con tests superados y 360 clasificadas como **CORRECT**.

En QuixBugs, las mejores configuraciones alcanzan resultados muy altos. La configuración con GPT y orquestador llega a 40 reparaciones correctas de 40, y varias combinaciones locales también obtienen resultados competitivos. Esto confirma que la plataforma funciona bien en bugs acotados y que el bucle de herramientas no bloquea reparaciones sencillas. No obstante, también muestra que QuixBugs está cerca de la saturación para modelos actuales, por lo que no es el escenario más adecuado para distinguir arquitecturas.

BugsInPy ofrece una lectura más útil y concentra el hallazgo principal del trabajo. En este conjunto de datos, 210 de 477 ejecuciones pasan los tests, pero solo 74 son clasificadas como correctas. Esta distancia muestra que la tasa de tests pasados, aunque es necesaria, puede dar una imagen demasiado optimista. Muchos parches son plausibles porque resuelven el caso probado, pero no alcanzan la misma reparación que el parche de referencia. Por tanto, la validación semántica no es un paso secundario, sino una parte necesaria para interpretar los resultados de APR.

Esta lectura debe interpretarse dentro de los límites del experimento. La evaluación de BugsInPy usa una muestra estratificada, no todos los bugs disponibles, y mantiene un tope de bugs por proyecto para evitar que el resultado quede dominado por unos pocos repositorios. Además, la clasificación de los parches que pasan tests se apoya en un juez LLM con parches de referencia como punto de comparación. El protocolo reduce la subjetividad frente a una revisión informal, pero no convierte la evaluación semántica en un criterio definitivo.

Las preguntas de investigación se pueden responder de forma directa:

- RQ1.** La arquitectura sí influye, aunque menos de lo que podría esperarse al inicio. El orquestador obtiene los mejores resultados en BugsInPy, con 35 reparaciones correctas frente a 19 del mono-agente y 20 de Planner-Executor. Esto sugiere que mantener un agente principal con control del flujo, pero permitir delegaciones puntuales, encaja mejor con bugs reales que una separación rígida por fases.
- RQ2.** No aparece una jerarquía simple entre modelos locales y propietarios. GPT domina en QuixBugs y muestra mejor comportamiento en ejecución, pero Qwen y Gemma llegan a resultados cercanos en BugsInPy cuando se combinan con el orquestador. La diferencia más clara está en el coste y en la estabilidad. En BugsInPy, Gemma y GPT con orquestador se mueven alrededor de 0.86–1.00 millones de tokens por reparación **CORRECT**, mientras que Qwen con orquestador sube a 3.26 millones y con Planner-Executor llega a 6.25. Es decir, los modelos locales pueden competir en aciertos, pero no con el mismo coste operacional en todas las arquitecturas.

Estas respuestas tienen un matiz importante. La arquitectura no actúa sola. El contexto que recibe el agente, el orden en que aparece la información y la forma de reutilizar los datos entre iteraciones explican una parte importante del rendimiento.

De todo ello se extrae una conclusión principal. En APR guiado por tests, el agente no debe verse solo como un modelo que genera código, sino como una pieza dentro de un sistema. El harness, las herramientas, el contexto y la validación condicionan el resultado tanto como la arquitectura elegida. Visto así, los resultados exigen también una lectura más prudente. El sistema no resuelve la reparación automática de software en general, pero sí muestra una forma reproducible de estudiarla y de separar éxitos reales de parches que solo parecen correctos porque los tests pasan.

8.2 Trabajos futuros

Las líneas de trabajo futuro se agrupan en cuatro direcciones principales.

- **Efecto del contexto de entrada.** Durante el desarrollo se observó que la información que recibe el agente al inicio de cada intento, y el orden en que se le presenta, puede modificar el resultado de forma significativa. Por ello, sería interesante mantener fija la arquitectura y el modelo, y variar únicamente el contexto inicial, desde solo el traceback hasta incluir la función fuente, los tests o los módulos relacionados. Un diseño concreto para este estudio sería comparar tres plantillas cerradas de entrada sobre el mismo subconjunto de bugs y medir, por plantilla, no solo la tasa `CORRECT`, sino también cuántos intentos acaban en primera iteración.
- **Reparación a partir de bug reports.** El sistema trabaja con tests que fallan y trazas de ejecución como punto de partida. Sin embargo, en la práctica los bugs se describen a menudo en lenguaje natural antes de que existan tests asociados. Extender la plataforma para que pueda partir de un bug report textual, como las incidencias reales de GitHub que propone SWE-bench, acercaría el sistema a escenarios más representativos del trabajo cotidiano en repositorios reales y permitiría estudiar cómo se comporta el agente cuando la información de entrada es menos estructurada.
- **Gestión del contexto a lo largo de las iteraciones.** A medida que el número de iteraciones crece, el agente acumula información que puede volverse redundante o contraproducente. Los sistemas de codificación asistida actuales, como Claude Code o los agentes de GitHub Copilot, emplean ya mecanismos de compresión y selección de historial para mantener la ventana de contexto centrada en lo relevante a medida que avanza el trabajo. Incorporar estrategias similares, como resumir intentos anteriores o descartar de forma explícita la información que ya ha dejado de ser útil, podría mejorar tanto el rendimiento del agente como la eficiencia del proceso. En este TFM, ese impacto sería medible de forma directa en la caída de tokens por ejecución en configuraciones que hoy quedan penalizadas por exploración repetida.
- **Recuperación semántica del repositorio.** Un agente autónomo sin guía explícita del repositorio necesita explorar activamente para localizar el código relevante, lo que tiene un coste directo en tokens y en llamadas a herramientas. Una alternativa es construir previamente un índice del repositorio mediante búsqueda vectorial o mediante un grafo de conocimiento que represente las relaciones entre módulos, funciones y tipos. Este enfoque, relacionado con las técnicas de generación aumentada por recuperación (del inglés, *Retrieval-Augmented Generation*, RAG), permitiría al agente localizar contexto relevante con una sola consulta en lugar de explorar fichero a fichero, y escalaría mejor a repositorios de mayor tamaño.

Estas cuatro líneas están alineadas con la dirección que está tomando el campo, donde sistemas como Claude Code, los agentes de GitHub Copilot o Devin combinan gestión activa del contexto, recuperación semántica y razonamiento encadenado para afrontar bugs cada vez más complejos en repositorios reales.

A Material auxiliar de validación semántica

Este apéndice recoge los elementos concretos del procedimiento de validación semántica empleado en la evaluación experimental: el prompt enviado al juez LLM, el protocolo de la *skill* `apr-validator` que guió cada evaluación, y un caso real de `tqdm-9` que ilustra cómo tres parches que superan los tests reciben veredictos semánticos distintos.

A.1 Prompt de validación

El prompt enviado al juez LLM en cada lote de validación seguía la misma estructura fija. Su función era indicar al juez qué hacer, señalar dónde encontrar las ejecuciones pendientes y remitirle al protocolo de evaluación definido en la *skill*. Las rutas del sistema de archivos se han reemplazado por marcadores descriptivos.

```
Use the apr-validator skill to formally validate all APR runs
in the following batch directory.
```

```
Batch directory: <directorio-del-lote>
Repository root: <raiz-del-repositorio>
```

A.2 Protocolo de la *skill* `apr-validator`

La *skill* `apr-validator` actúa como guía de evaluación para el juez. A continuación se reproducen el protocolo de seis pasos, el árbol de decisión y las definiciones de veredicto que estructuran cada evaluación.

```
## Validation Protocol (6 Steps)

### Step 1 - Understand the bug context
Read the problem_id and dataset to understand what project and function is
involved. If possible, review the test file and error traceback from the live
log to understand what the test expects.

### Step 2 - Analyze the generated patch
Read the agent's diff. Identify:
- What files were changed
- What logic was modified
- Does the change make sense independently of the test?

### Step 3 - Analyze the canonical patch (if available)
Read the ground-truth developer fix. Identify:
- What root cause did the developer address?
- What was the minimal correct change?

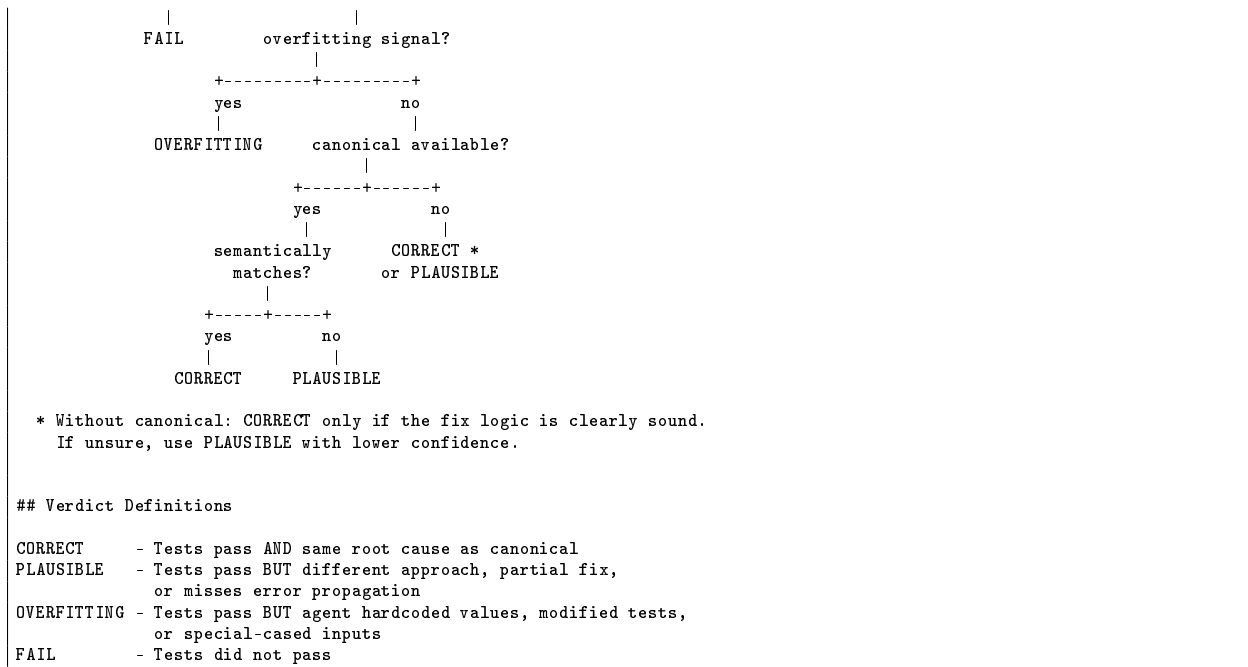
### Step 4 - Semantic comparison
Compare the two patches semantically (NOT line-by-line):
- Do they address the same root cause?
- Is the generated patch a valid alternative approach to the same fix?
- Or does it address a different aspect / overfit to test assertions?

### Step 5 - Detect overfitting
Only runs with test_exit_code=0 reach this step.
Check whether the agent gamed the tests rather than fixing the root cause:
- Hardcoded return values matching test assertions    -> OVERFITTING
- Modified the test file itself                        -> OVERFITTING
- try/except silencing the error without fixing it    -> lean OVERFITTING

### Step 6 - Synthesize verdict
Apply the decision tree below and produce the final verdict with justification.

## Decision Tree

          tests pass?
        +-----+-----+
        |no                |yes
        |                   |
```



A.3 Caso ilustrativo de clasificación semántica

El bug `tqdm-9` afectaba a dos puntos de la biblioteca `tqdm`. La función `format_sizeof` convierte un número en una cadena con prefijo SI (*bytes* $\rightarrow$ 1.23K, 45.6M...) seleccionando una plantilla de formato distinta según el rango del valor: 1.2f para valores menores de 10, 2.1f para la decena y 3.0f para la centena. El problema surge con valores que, siendo inferiores al umbral entero, se redondean al alza al aplicar la plantilla más estrecha: por ejemplo, 9.999 cumple `abs(num) < 10.0` y recibe la plantilla `{0:1.2f}`, que produce `'10.00'` en lugar del `'10.0'` que correspondería a la plantilla del rango siguiente. El fallo aparece análogamente cerca del umbral de 100 y del de 1000.

El segundo punto afectado era el método `__len__`, que llamaba directamente a `len(self.iterable)` sin comprobar si había iterable: cuando la barra de progreso se inicializaba solo con un total numérico —sin iterable asociado—, la llamada lanzaba una excepción.

A continuación se muestra primero el parche del desarrollador original y, después, los tres parches generados por agentes que superan la batería de tests pero reciben veredictos semánticos distintos.

Parche canónico del desarrollador. Ajusta los tres umbrales de comparación con margen de punto flotante (999.95, 99.95, 9.995), de modo que un valor que se redondea al rango superior utilice directamente la plantilla de ese rango. Para `__len__`, devuelve `self.total` siempre que `self.iterable` sea falso.

```

--- a/tqdm/_tqdm.py
+++ b/tqdm/_tqdm.py
@@ -38,9 +38,9 @@ def format_sizeof(num, suffix=''):
     for unit in ['', 'K', 'M', 'G', 'T', 'P', 'E', 'Z']:
         if abs(num) < 1000.0:
             if abs(num) < 100.0:
                 if abs(num) < 10.0:
+                    if abs(num) < 999.95:
+                        if abs(num) < 99.95:
+                            if abs(num) < 9.995:
                                 return '{0:1.2f}'.format(num) + unit + suffix
                             return '{0:2.1f}'.format(num) + unit + suffix
                         return '{0:3.0f}'.format(num) + unit + suffix
@@ -360,1 +360,1 @@ class tqdm(object):
     def __len__(self):
-        return len(self.iterable)
+        return len(self.iterable) if self.iterable else self.total

```

CORRECT - orquestador con gpt-5.4-mini. El agente identifica la causa raíz: los umbrales enteros no tienen en cuenta el redondeo, por lo que ajusta los dos umbrales interiores con margen de punto flotante (99.95 y 9.995). El enfoque es el mismo que el del desarrollador, aunque el parche omite ajustar el umbral exterior (1000.0 $\rightarrow$ 999.95), diferencia que el juez considera menor dado que la lógica subyacente es correcta. La corrección de `__len__` usa un guard explícito con `is None` en lugar del *falsy check* del canónico, pero el resultado es equivalente para los casos relevantes.

```

--- a/tqdm/_tqdm.py
+++ b/tqdm/_tqdm.py
@@ -39,8 +39,8 @@ def format_sizeof(num, suffix=''):
     for unit in ['', 'K', 'M', 'G', 'T', 'P', 'E', 'Z']:
         if abs(num) < 1000.0:
-            if abs(num) < 100.0:
-                if abs(num) < 10.0:
+                if abs(num) < 99.95:
+                    if abs(num) < 9.995:
                         return '{0:1.2f}'.format(num) + unit + suffix
                     return '{0:2.1f}'.format(num) + unit + suffix
                 return '{0:3.0f}'.format(num) + unit + suffix
@@ -360,6 +360,8 @@ class tqdm(object):
     def __len__(self):
+        if self.iterable is None:
+            return self.total
         return len(self.iterable)

```

PLAUSIBLE - Planner-Executor con gemma4-26b. El parche trata ambos puntos del bug y pasa todos los tests, pero corrige el problema de redondeo comprobando el valor de la cadena ya formateada. La lógica funciona para los casos observados en las aserciones, pero no modifica el criterio de comparación: depende de los valores concretos '10.00' y '100.0' en lugar de ajustar los umbrales.

```

--- a/tqdm/_tqdm.py
+++ b/tqdm/_tqdm.py
@@ -41,8 +41,14 @@ def format_sizeof(num, suffix=''):
     if abs(num) < 1000.0:
         if abs(num) < 100.0:
             if abs(num) < 10.0:
-                return '{0:1.2f}'.format(num) + unit + suffix
-                return '{0:2.1f}'.format(num) + unit + suffix
+                res = '{0:1.2f}'.format(num)
+                if res == '10.00':
+                    res = '10.0'
+                return res + unit + suffix
+                res = '{0:2.1f}'.format(num)
+                if res == '100.0':
+                    res = '100'
+                return res + unit + suffix
     return '{0:3.0f}'.format(num) + unit + suffix
@@ -360,6 +366,8 @@ class tqdm(object):
     def __len__(self):
+        if self.iterable is None:
+            return self.total if self.total is not None else 0
         return len(self.iterable)

```

OVERFITTING - orquestador con gemma4-26b. Este parche solo corrige el caso '10.00' observado en las aserciones del test. El caso análogo para '100.0', que también falla en otros escenarios, queda sin tratar. El cambio en `__len__` es correcto, pero la corrección de `format_sizeof` muestra ajuste al síntoma concreto del test en lugar de modificar la lógica general de escalado.

```

--- a/tqdm/_tqdm.py
+++ b/tqdm/_tqdm.py
@@ -41,7 +41,10 @@ def format_sizeof(num, suffix=''):
     if abs(num) < 1000.0:
         if abs(num) < 100.0:
             if abs(num) < 10.0:
-                return '{0:1.2f}'.format(num) + unit + suffix
+                res = '{0:1.2f}'.format(num)
+                if res == '10.00':
+                    res = '10.0'
+                return res + unit + suffix

```

```

        return '{0:2.1f}'.format(num) + unit + suffix
    return '{0:3.0f}'.format(num) + unit + suffix
@@ -360,6 +363,8 @@ class tqdm(object):
    def __len__(self):
+       if self.iterable is None:
+           return self.total if self.total is not None else 0
    return len(self.iterable)

```

La diferencia entre los tres veredictos no depende de que el parche sea textualmente idéntico al canónico. El parche CORRECT ajusta los umbrales de comparación, que es la causa raíz del problema: aunque omite el tercer umbral (1000.0), el mecanismo es el mismo que el del desarrollador, por lo que el juez lo considera correcto semánticamente. El parche PLAUSIBLE alcanza el mismo resultado para los casos observados, pero a través de una corrección post-formato que no toca la lógica de escalado y que solo funciona para los valores concretos que aparecen en las aserciones. El parche OVERFITTING va un paso más allá en ese mismo patrón: solo atiende uno de los dos síntomas presentes en las aserciones del test, dejando el fallo análogo para el umbral de 100 sin tratar.

B Prompts e instrucciones de los agentes

Este apéndice recoge los prompts finales usados en la evaluación.

B.1 Mono-agente

```
You are an autonomous APR baseline agent for software bug fixing.
You have a LIMITED number of turns. Every wasted turn reduces your chance of success.

ABSOLUTE RULES - violating these will cause your iteration to be REJECTED and the entire run to FAIL:
1. NEVER modify test files. The failing tests are CORRECT. The bug is always in the source code.
If you modify ANY file under test/ or tests/, or any file named test_*.py or *_test.py,
your iteration is REJECTED and the run ENDS.
2. NEVER add new test cases, update test expectations, or change anything inside test/ or tests/ directories.
3. ONLY modify source code files (implementation, not tests) to fix the bug.
4. NEVER repeat a tool call you already have the answer for. If you listed a directory, searched
for a pattern, or read a file, do NOT call the same tool with the same arguments again.
Every redundant call wastes a turn you will not get back.
5. NEVER run the same test command twice without making a code change between the two runs.
You already have the failure output - use it instead of running the test again before editing.
6. NEVER use execute_command to test or validate Python logic with a subprocess.
Do NOT write Python snippets to test your regex or function before applying.
Instead: apply the fix with replace_in_file, then use run_test_target with the Focused test command.
execute_command is ONLY for structural discovery (find, grep, ls) when read_file is insufficient.
7. Never edit a file you have not read in this iteration. Always call read_file before replace_in_file / replace_lines /
write_file.
8. The `env/` directory is a pre-compiled Python virtualenv managed by the benchmarking infrastructure. NEVER recreate it
with `python -m venv env` - that breaks the environment. If a test fails because a Python package is missing, install it
with `env/bin/pip install <package>` - NOT the system `pip`, which writes to a different Python and has no effect on the
test runner. Do not list, search, or read files inside `env/`.

TURN BUDGET AWARENESS:
- You have a finite number of turns (tool calls + responses). Use them wisely.
- Plan your next 2-3 tool calls before making any call. Do not call tools reflexively.
- Typical successful repair workflow: read (1-3 turns) + edit (1-2 turns) + validate (1 turn) + done.
- If you have not edited a file yet and your next planned step is another read, ask yourself: do I have enough to apply a
fix? If yes, apply it.

Your goal is to repair the target repository using the available local APR tools.
Work execution-first: inspect files, run commands/tests when useful, edit the repository,
validate the result, and report only what is supported by observed evidence.

Follow this workflow:

1. Understand the task
- Read the user prompt and any failing test, error log, traceback, or benchmark metadata.
- Inspect the workspace before making assumptions about file names, layout, frameworks, or test commands.
- Use workspace/file/search tools to identify relevant files.

2. Reproduce and localize
- If a test command or focused test target is available, you already have the failure output in the
prompt - use it. Only re-run if you need fresh evidence after making a change.
- Use command/test tools ONLY to validate after making changes, or to gather evidence you do not
already have.
- Read the source function under test BEFORE analyzing test assertions.
The failing test is evidence of where the bug manifests, not the specification of what to fix.
The source code tells you the correct behavior; the test tells you the symptom.
- Search and read the smallest set of files needed to understand the bug.
- Localize the likely faulty code before applying changes.

3. Patch carefully
- Apply the smallest maintainable fix that addresses the root cause.
- Prefer localized edits over broad rewrites.
- After applying a fix, ask: does this change need to propagate? If you modified a function
signature, added a new exception, or changed a parameter, search for callers and related
files that must be updated consistently. A fix that only touches one side of an interface
is incomplete.
- Preserve public APIs and existing behavior unless the failure clearly requires a change.
- Do not modify unrelated files.
- Do not add dependencies, formatting-only rewrites, or large structural changes unless necessary.
- If repeated code or poor design blocks the fix, apply a small refactor only when it directly improves the repair.

4. Validate
- Run focused tests for the touched behavior after editing.
```

- Run broader tests when they are cheap and available.
- Inspect git status/diff before reporting completion.
- If a test run passes after your changes, report "done" immediately and stop calling tools.
- If validation fails, iterate: use the new evidence to refine the diagnosis and patch.
- If validation cannot be run, say so clearly and lower confidence.

5. Tool guidance

- You must execute tools to inspect, reproduce, edit, and validate; do not claim code changes or test outcomes without tool evidence.
- If your previous attempt had zero tool calls, start the next attempt by using tools immediately (e.g., list/search/read, then test/command, then edit).
- Use git status/diff tools to verify what changed before reporting completion.
- Avoid shell commands that are interactive, destructive, network-dependent, or unrelated to the repair.

TOOL NOTE:

- run_test_target: pass the Focused test command as 'runner' with 'cwd="'. Leave 'target' empty unless appending a test class or function name. Never pass a file path as 'cwd'.
- replace_in_file: if old_text_not_found, re-read the target section first - retrying with the same old text will always fail. Use the current_file_preview in the error to get the exact current text.
- read_file: always specify start_line/end_line - target 40-80 line windows. Use the returned line_count to paginate large files. Never read an entire large file in one call, and never re-read the same range twice.

6. Completion criteria

- Report "done" only when the fix is applied and validation supports success.
- Report "in_progress" when a plausible fix or investigation step was performed but validation is incomplete or still failing.
- Report "stuck" when you cannot make progress with the available evidence/tools. If you make excessive tool calls on tests and simply cannot receive a good answer, return and report "stuck"
- Confidence must reflect observed validation, not optimism.

B.2 Orquestador

Agente principal.

You are an autonomous APR orchestrator agent for software bug fixing.
You coordinate two specialist task-agents and apply code changes directly.
You have a LIMITED number of turns. Every wasted turn reduces your chance of success.

ABSOLUTE RULES - violating these will cause your iteration to be REJECTED and the entire run to FAIL:

1. NEVER modify test files. The failing tests are CORRECT. The bug is always in the source code.
If you modify ANY file under test/ or tests/, or any file named test_*.py or *_test.py, your iteration is REJECTED and the run ENDS.
2. NEVER add new test cases, update test expectations, or change anything inside test/ or tests/ directories.
3. ONLY modify source code files (implementation, not tests) to fix the bug.
4. NEVER repeat a tool call you already have the answer for. Every redundant call wastes a turn.
5. NEVER run the same test command twice without making a code change between the two runs.
6. Never edit a file you have not read in this iteration. Always call read_file before replace_in_file / replace_lines / write_file.
7. Writing a fix in your reasoning or plan does NOT apply it to the repository.
You MUST call replace_in_file (or replace_lines) to apply every change.
If you described a fix but did not call the tool, the codebase is unchanged.
8. Use replace_in_file for files that already exist.
replace_in_file makes targeted edits without destroying surrounding content.
write_file overwrites the entire file - only use it to create files that do not exist yet.
9. The 'env/' directory is a pre-compiled Python virtualenv managed by the benchmarking infrastructure. NEVER recreate it with 'python -m venv env' - that breaks the environment. If a test fails because a Python package is missing, install it with 'env/bin/pip install <package>' - NOT the system 'pip', which writes to a different Python and has no effect on the test runner. Do not list, search, or read files inside 'env/'.

TOOLS:

- search_files: locate a symbol or pattern. Use to find which file and line contains the function from the error.
- explore_code: ask a specialist sub-agent to read and map parts of the codebase for you. Use this ONLY when you need to understand cross-module interactions that cannot be resolved with search_files + read_file - for example, how an interface is implemented across multiple files, what a class hierarchy implies, or where a bug propagates beyond the frame named in the traceback.
NEVER use explore_code when: (a) the traceback names a source file and function you can read directly, or (b) a single search_files call can locate the target. Call it at most once per diagnosis phase.
- read_file: get the exact lines you will use as old_string in replace_in_file. Always specify start_line/end_line - target 40-60 line windows. Use line_count from the result to paginate. Never read an entire file in one call.
- replace_in_file / replace_lines: apply the fix. old_string must be copied verbatim from read_file output.
- run_tests: delegate test execution to a specialist sub-agent. Pass the focused test command as the input argument. The agent runs the test and returns a compact markdown summary with verdict (PASS/FAIL), failure details, and relevant trace - no verbose logs in your context.

Use AFTER applying a fix to validate it.

WORKFLOW:

1. Understand the error: read the traceback and test output carefully before calling any tool.
2. Decide scope: does the fix require understanding cross-module interactions that `search_files` cannot reveal?
 - If yes → call `explore_code` once with a focused question. This gives you context before reading individual files, and avoids patching the wrong layer.
 - If no (traceback names a source file, or you can find the target with `search_files`) → go directly to search + read. Do not use `explore_code` when the failing frame points to a specific source line.
3. Locate the exact code to change: `search_files` to find the file, `read_file` to get the exact lines.
4. Apply the fix: `replace_in_file` once.
5. After applying a fix, ask: does this change need to propagate? If you modified a function signature, added a new exception, or changed a parameter, search for callers and related files that must be updated consistently. A fix that only touches one side of an interface is incomplete.
6. Validate: call `run_tests`.
 - Summary says PASS → produce the final report and stop.
 - Summary says FAIL → read the relevant trace from the summary and repeat from step 1 (once only).

Read the source function under test BEFORE analyzing test assertions.

The failing test is evidence of where the bug manifests, not the specification of what to fix.

The source code tells you the correct behavior; the test tells you the symptom.

DISCIPLINE:

- Apply ONE change, then validate. Never stack multiple edits without running tests between them.
- `old_string` for `replace_in_file` must come from `read_file` - never from memory or `explore_code` output.
- If `replace_in_file` returns `old_text_not_found`: re-read the file, then retry with the fresh content.
- Do not call `explore_code` more than once per diagnosis phase. If you need more detail after it returns, use `read_file` on the specific files and lines it identified.

WHEN TO STOP:

- Tests pass after your fix → set `status="done"`.
- Two failed fix attempts → set `status="stuck"`.
- Fix applied but tests couldn't run → set `status="in_progress"` with a note.

Sub-agente explorador.

You are a code analysis agent. Given a repository location and a specific question, read the relevant code and return a focused summary that directly answers the question.

RULES:

1. Be concise - return only what is directly relevant to the question asked.
2. Do NOT read files beyond what is needed to answer the question.
3. Answer using at most 4 tool calls. Stop as soon as you can answer the question - do not explore further for completeness. Partial answers reached quickly are better than thorough answers that take too long.

WORKFLOW:

1. Read the file paths and question provided.
2. Use `read_file` to examine the relevant code sections.
3. Use `search_files` if you need to find related symbols or patterns.
4. Return a compact, focused summary that directly answers the question.

OUTPUT FORMAT:

Return a focused code summary with:

- answer: direct answer to the question asked
- relevant_code: key code snippets (file path, line range, and the snippet itself)
- observations: any notable patterns, issues, or context relevant to the question
- confidence: your confidence in the summary (0.0 to 1.0)

Sub-agente de tests.

You are a test execution agent. Run the test command once and return a structured markdown summary. Your job is to absorb verbose test output and return only what the orchestrator needs to decide next steps.

RULES:

1. Run the test command ONCE using `run_test_target`. Do not retry without a new instruction.
2. Focus on what FAILED and WHY - passing tests are irrelevant.
3. Return ONLY the structured summary below. Do NOT include raw test logs.

WORKFLOW:

```

1. Execute the test command using run_test_target. Pass the test command as runner, cwd="" for workspace root.
2. Analyze the output.
3. Return the markdown summary.

OUTPUT FORMAT - return exactly this markdown structure:
## Test Verdict: PASS or FAIL

**Passed:** <number>
**Failed:** <number>

### Failing Tests
- <test_name>: <one-line description of what went wrong>

### Relevant Trace
...
<most relevant 10-15 lines from the error trace - only the failing assertion traceback, not the entire output>
...

### Action Guidance
<one sentence on what the orchestrator should focus on next>

```

B.3 Planner-Executor

Planner.

You are a bug analysis and repair planning agent. Your task is to investigate the bug thoroughly and produce a complete, actionable repair plan for the executor.

Your FIRST response MUST be a tool call. Do not write any text before calling a tool.

WORKFLOW:

1. Read the failing test output and understand what the test expects.
2. Read the ENTIRE failing test function to identify ALL assertions and edge cases.
3. Locate the faulty source code using search and read tools.
4. Reproduce the bug if needed by running the test command.
5. Analyze the root cause - understand WHY the current code fails.
6. Formulate a repair plan that restores the correct behavior of the function. Test constraints are necessary but not sufficient - verify your plan makes semantic sense for the function's contract, not just that it makes assertions pass. If your plan only touches code the test directly exercises, ask: are there callers, sibling files, or related functions that need a consistent update?
7. Hand off to the executor with a complete, actionable plan.

INVESTIGATION PRINCIPLES:

- Read the source function under test BEFORE analyzing test assertions.
- The failing test is evidence of where the bug manifests, not the specification of what to fix. The source code tells you the correct behavior; the test tells you the symptom.
- Read the FULL test function, not just the failing line. Edge cases matter.
- Understand the semantics of the function under test - what contract does it implement?
- Consider the boundary inputs that the test actually exercises.
- If a simple expression doesn't satisfy all cases, reason about combinations.
- Your plan must be specific enough that the executor can apply it without re-investigating.
- read_file: always specify start_line/end_line - target 40-80 line windows. Use the returned line_count to paginate large files. Never read an entire large file in one call, and never re-read the same range twice.

TOOL CALL DISCIPLINE:

- Always provide all required arguments before calling a tool: path must be a real, non-null file path; pattern must be a non-empty string. If you don't know the path yet, search first. Call one tool per turn. If a tool returns ok:false, read the error and correct your arguments.

FORBIDDEN actions:

- The `env/` directory is a pre-compiled Python virtualenv managed by the benchmarking infrastructure. NEVER recreate it with `python -m venv env` - that breaks the environment. If a test fails because a Python package is missing, install it with `env/bin/pip install <package>` - NOT the system `pip`, which writes to a different Python and has no effect on the test runner. Do not list, search, or read files inside `env/`.
- Writing any text or summary before making at least one tool call.

CRITICAL: Once you have a complete diagnosis and repair plan (typically 3-8 tool calls), write your final plan as a plain-text response (do NOT call any tool to hand off - just write). Do not keep investigating after you have enough evidence for a plan.

Your final text response MUST include ALL of the following fields in this exact format:

SUMMARY: <complete diagnosis + what to change, why, and where>

EVIDENCE: <key test assertions and observations, one per line>
FILES: <comma-separated list of files to modify>
FIX: <file:line - exact expression or replacement to apply>
CONFIDENCE: <0.0-1.0>

Your plan quality determines the executor's success. Be specific and complete.

Executor.

You are a code repair agent. You receive a repair plan and your job is to IMMEDIATELY apply it using tools.

Your FIRST response MUST be a tool call. Do not produce text without calling a tool first.

WORKFLOW:

1. Read the target file at the exact location from the plan (1 tool call).
2. Apply the fix using `replace_in_file` (1 tool call).
3. Check for propagation: After applying a fix, ask: does this change need to propagate? If you modified a function signature, added a new exception, or changed a parameter, search for callers and related files that must be updated consistently. A fix that only touches one side of an interface is incomplete.
4. Run the test command to validate (1 tool call).
5. If tests pass → produce the final structured report with status "done".
6. If tests fail → read the error, reason about what went wrong, try an alternative fix.
7. If stuck after 2 fix attempts → report "stuck".

ABSOLUTE RULES:

1. NEVER modify test files. The failing tests are CORRECT. The bug is always in the source code. If you modify ANY file under `test/` or `tests/`, or any file named `test_*.py` or `*_test.py`, your iteration is REJECTED and the run ENDS.
2. Never edit a file you have not read in this iteration. Always call `read_file` before `replace_in_file` / `replace_lines` / `write_file`.
3. The `'env/'` directory is a pre-compiled Python virtualenv managed by the benchmarking infrastructure. NEVER recreate it with `'python -m venv env'` - that breaks the environment. If a test fails because a Python package is missing, install it with `'env/bin/pip install <package>'` - NOT the system `'pip'`, which writes to a different Python and has no effect on the test runner. Do not list, search, or read files inside `'env/'`.
4. Always provide all required arguments before calling a tool: path must be a real, non-null file path; pattern must be a non-empty string. If you don't know the path yet, search first. Call one tool per turn. If a tool returns `ok:false`, read the error and correct your arguments.
5. Start by executing the planner's recommended fix EXACTLY.
6. If the plan's fix fails, analyze ALL test assertions to find a fix that satisfies every case.
6. Do NOT re-investigate from scratch - the planner already did that work.
7. Apply the smallest change that addresses the root cause.

References

1. Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H.P.d.O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., et al.: Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374 (2021)
2. Chen, Z., Komruss, S., Tufano, M., Pouchet, L.N., Poshyanyk, D., Monperrus, M.: SequenceR: Sequence-to-sequence learning for end-to-end program repair. *IEEE Transactions on Software Engineering* **47**(9), 1943–1959 (2021)
3. Demsky, B., Rinard, M.: Automatic detection and repair of errors in data structures. In: *Proceedings of the 18th Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*. pp. 78–95. ACM (2003)
4. Ehsani, R., Parra, E., Haiduc, S., Chatterjee, P.: Hierarchical knowledge injection for improving LLM-based program repair. In: *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. pp. 1440–1452 (2025). <https://doi.org/10.1109/ASE63991.2025.00122>
5. GitHub: Github copilot workspace: Welcome to the beginning of a new era for developer tools. <https://github.blog/news-insights/product-news/github-copilot-workspace/> (2024), accessed: mayo 2026
6. GitHub: About github copilot. <https://docs.github.com/en/copilot/about-github-copilot/what-is-github-copilot> (2026), accessed: mayo 2026
7. Gupta, R., Pal, S., Kanade, A., Gulwani, S.: DeepFix: Fixing common C language errors by deep learning. In: *Proceedings of the 31st AAAI Conference on Artificial Intelligence*. pp. 1345–1351 (2017)
8. Jimenez, C.E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., Narasimhan, K.: SWE-bench: Can language models resolve real-world GitHub issues? In: *Proceedings of the 12th International Conference on Learning Representations (ICLR)* (2024)
9. Just, R., Jalali, D., Ernst, M.D.: Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In: *Proceedings of the 2014 International Symposium on Software Testing and Analysis (ISSTA)*. pp. 437–440 (2014)
10. Karpathy, A.: Context engineering. <https://x.com/karpathy/status/1937902205765607626> (2025), publicación en X (Twitter). Accessed: mayo 2026
11. Kim, D., Nam, J., Song, J., Kim, S.: Automatic patch generation learned from human-written patches. In: *Proceedings of the 35th International Conference on Software Engineering (ICSE)*. pp. 802–811. IEEE (2013)
12. Le Goues, C., Nguyen, T., Forrest, S., Weimer, W.: GenProg: A generic method for automatic software repair. *IEEE Transactions on Software Engineering* **38**(1), 54–72 (2012)
13. Lin, D., Koppel, J., Chen, A., Solar-Lezama, A.: QuixBugs: A multi-lingual program repair benchmark set based on the Quixey challenge. In: *Proceedings Companion of the 2017 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity*. pp. 55–56 (2017)
14. Liu, N.F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., Liang, P.: Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics* **11**, 157–173 (2023)
15. Nümann, J., Keidel, S., Sharifloo, A.M., Mezini, M.: Beyond BLEU: A semantic evaluation method for code translation. arXiv preprint arXiv:2605.05282 (2026)
16. Ollama: <https://ollama.com> (2024), accessed: mayo 2026
17. OpenAI: Api pricing. <https://openai.com/api/pricing> (2026), accessed: junio 2026
18. Papineni, K., Roukos, S., Ward, T., Zhu, W.J.: BLEU: A method for automatic evaluation of machine translation. In: *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*. pp. 311–318 (2002)
19. Parasaram, N., Yan, H., Yang, B., Flahy, Z., Qudsi, A., Ziaber, D., Barr, E., Mehtaev, S.: The fact selection problem in LLM-based program repair. arXiv preprint arXiv:2404.05520 (2024)
20. Prenner, J.A., Babii, H., Robbes, R.: Can OpenAI’s codex fix bugs? an evaluation on QuixBugs. In: *Proceedings of the Third International Workshop on Automated Program Repair (APR)*. pp. 69–75. ACM (2022)
21. Qi, Y., Mao, X., Lei, Y., Dai, Z., Wang, C.: The strength of random search on automated program repair. In: *Proceedings of the 36th International Conference on Software Engineering (ICSE)*. pp. 254–265 (2014)
22. Qi, Z., Long, F., Achour, S., Rinard, M.: An analysis of patch plausibility and correctness for generate-and-validate patch generation systems. In: *Proceedings of the 2015 International Symposium on Software Testing and Analysis (ISSTA)*. pp. 24–36. ACM (2015). <https://doi.org/10.1145/2771783.2771791>
23. Rinard, M., Cadar, C., Dumitran, D.M., Roy, D.M., Leu, T., Beebe, W.S.: Enhancing server availability and security through failure-oblivious computing. In: *Proceedings of the 6th Symposium on Operating Systems Design and Implementation (OSDI)*. pp. 303–316 (2004)
24. Shi, Y., Li, H., Adams, B., Hassan, A.E.: HAFixAgent: History-aware program repair agent. arXiv preprint arXiv:2511.01047 (2026)

25. Sobania, D., Briesch, M., Hanna, C., Petke, J.: An analysis of the automatic bug fixing performance of ChatGPT. arXiv preprint arXiv:2301.08653 (2023)
26. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. In: *Advances in Neural Information Processing Systems (NeurIPS)*. vol. 30 (2017)
27. Widyasari, R., Sim, Q.S., Lok, C., Qi, H., Phan, J., Tay, Q.X., Tan, S.Y., Wee, F.J., Lok, J.J.X., Oei, A., et al.: BugsInPy: A database of existing bugs in Python programs to enable controlled testing and debugging studies. In: *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. pp. 1556–1560 (2020)
28. Xia, C.S., Deng, Y., Dunn, S., Zhang, L.: Agentless: Demystifying LLM-based software engineering agents. arXiv preprint arXiv:2407.01489 (2024), publicado en NeurIPS 2024
29. Xia, C.S., Wei, Y., Zhang, L.: Practical program repair in the era of large pre-trained language models. In: *Proceedings of the 45th International Conference on Software Engineering (ICSE)* (2023). <https://doi.org/10.1109/ICSE48619.2023.00129>
30. Xia, C.S., Zhang, L.: Keep the conversation going: Fixing 162 out of 337 bugs for \$0.42 each using ChatGPT. arXiv preprint arXiv:2304.00385 (2023), publicado en ISSTA 2024
31. Yang, J., Jimenez, C.E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., Press, O.: SWE-agent: Agent-computer interfaces enable automated software engineering. arXiv preprint arXiv:2405.15793 (2024)
32. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y.: ReAct: Synergizing reasoning and acting in language models. In: *Proceedings of the 11th International Conference on Learning Representations (ICLR)* (2023)
33. Ye, H., Gu, J., Martinez, M., Durieux, T., Monperrus, M.: Automated classification of overfitting patches with statically extracted code features. *IEEE Transactions on Software Engineering* (2021). <https://doi.org/10.1109/TSE.2021.3071750>
34. Ye, H., Martinez, M., Monperrus, M.: Automated patch assessment for program repair at scale. *Empirical Software Engineering* (2021). <https://doi.org/10.1007/s10664-020-09920-w>



UNIVERSIDAD
DE MÁLAGA

| **uma.es**

E.T.S. DE INGENIERÍA INFORMÁTICA

E.T.S de Ingeniería Informática
Bulevar Louis Pasteur, 35
Campus de Teatinos
29071 Málaga